
Trinary

A Ternary (Base-3) Computer Simulation

Complete Architecture, Implementation, and Analysis

Devak Chaudhari

June 12, 2026

A monograph spanning logic gates, CPU architecture,
assembler, compiler, OS, GPU, neural networks, hardware simulation,
and web visualisation — all built on ternary (base-3) arithmetic.

Abstract

We present Trinary, a complete ternary (base-3) computer system simulated entirely in software. Spanning from fundamental ternary logic gates through a 27-opcode central processing unit, two-pass assembler, machine-code encoder, dual memory-mapped display subsystems, a Fantasy Console software development kit, a structured-language compiler (TAL), cycle-accurate hardware microarchitecture simulation with N-way set-associative caches and burst-mode bus transactions, a tensor accelerator coprocessor with GPU-mode parallel computing, ternary neural networks trained via native back-propagation, and a WebSocket-based web visualizer, Trinary serves as both an educational platform for computer architecture and a research vehicle for non-binary computing systems. The system comprises approximately 28,000 lines of Python and C code, validated by a test suite of 703 unit tests across all subsystems. All arithmetic operations use native signed-magnitude base-3 circuits without intermediate decimal conversion. A native C backend provides up to $13\times$ speedup on arithmetic-heavy workloads through an auto-detected ctypes bridge. This monograph presents the complete architecture, design rationale, implementation details, and performance characteristics of the Trinary system across 44 chapters with full instruction-set reference, hardware specifications, benchmark data, and appendix reference.

Keywords: ternary computing, base-3, computer architecture, CPU simulation, neural networks, hardware simulation, web visualizer

Contents

Part I

Foundations

Chapter 1

Introduction to Ternary Computing

1.1 Binary and Its Alternatives

Since the earliest electronic computers, the binary number system (base 2) has dominated digital computing. This dominance is not due to mathematical optimality but to engineering pragmatism: two-state electronic circuits (on/off, high/low, charged/discharged) are reliable, simple to manufacture, and map naturally to Boolean algebra. The vacuum-tube and later transistor-based logic families that emerged in the mid-20th century were inherently binary, and the entire semiconductor industry optimized around this paradigm for decades.

Yet from an information-theoretic standpoint, base 2 is not the most efficient radix. Consider the problem of representing numbers up to some maximum value N using a fixed number of digit positions. The number of distinct values representable with d digits in base b is b^d . For a given b , the number of digits required to represent N is $\lceil \log_b N \rceil$, and the total “cost” in terms of digit states is $b \cdot \lceil \log_b N \rceil$. Minimizing this cost over b yields the optimal radix.

1.2 The Optimal Radix Problem

Definition 1.1 (Radix efficiency). For a base b , the number of distinct states per digit is b , and the number of digits needed to represent a value up to N is approximately $\log_b N$. The product $C(b) = b \cdot \log_b N$ represents a figure of merit, where lower values indicate greater efficiency.

Treating b as a continuous variable and ignoring the ceiling, we have:

$$C(b) = b \cdot \frac{\ln N}{\ln b} = (\ln N) \cdot \frac{b}{\ln b}.$$

Minimizing $f(b) = b/\ln b$ with respect to b :

$$f'(b) = \frac{\ln b - 1}{(\ln b)^2} = 0 \implies \ln b = 1 \implies b = e \approx 2.718.$$

Thus the mathematically optimal radix is e , making base 3 (ternary) the most efficient integer base. This result was noted by Knuth [?] and has been confirmed by multiple analyses [?].

In practice, ternary offers approximately 37% fewer digits than binary for representing equivalent values. For a value of 1,000,000, binary requires 20 digits (11110100001001000000₂) while ternary requires only 13 (1212210202001₃). This translates to potential savings in memory, bus width, and power consumption.

1.3 Historical Ternary Computers

The most significant historical implementation of ternary computing was the Soviet Setun computer [?], developed at Moscow State University in 1958 under the direction of Nikolay Brusentsov. The Setun used *balanced ternary* with digits $\{-1, 0, 1\}$, implemented using ferrite-core memory and vacuum-tube logic. Key specifications included:

- Word size: 18 trits (approximately 28.5 bits of information);
- Memory: 162 words of ferrite-core storage (expandable to 486);
- Performance: approximately 4,500 operations per second;
- Number representation: balanced ternary with no separate sign bit;
- Total production: approximately 50 units.

The Setun demonstrated several advantages over contemporary binary machines: simpler arithmetic (addition and subtraction are identical operations in balanced ternary), inherent signed representation, and more efficient number storage. However, the binary-oriented semiconductor industry and the dominance of the von Neumann architecture with binary logic gates prevented ternary computing from achieving widespread adoption.

Subsequent research in multiple-valued logic (MVL) [?] has explored quaternary (base-4), quinary (base-5), and other radices, but no other ternary computer has been commercially produced.

1.4 Motivation for Trinary

Modern computing faces several challenges that warrant revisiting alternative number systems:

- **Power consumption:** ternary arithmetic circuits can, in theory, perform more computation per switching event than binary equivalents;
- **Memory wall:** ternary's improved digit density could reduce memory traffic;
- **Educational value:** studying non-binary architectures deepens understanding of computer architecture fundamentals;
- **Research platform:** a complete, executable ternary system enables experimentation with ternary algorithms, compilers, and operating systems.

Trinary addresses these motivations by providing a fully functional ternary computer simulation with a complete software stack. The system spans 34 chapters of documentation, approximately 28,000 lines of source code, and 594 automated tests.

1.5 Scope of This Monograph

This work presents the complete Trinary system across 44 chapters organized into nine parts:

- **Part I – Foundations:** Chapters 1–12 cover ternary number representation, logic gates, adder circuits, native signed-magnitude arithmetic, the arithmetic logic unit (ALU), register file, memory architecture, and comparison to related systems;
- **Part II – CPU Architecture:** Chapters 13–?? detail the 27-opcode instruction set, fetch-decode-execute cycle, data movement and arithmetic instructions, control flow, subroutines, stack operations, memory access, interrupts, and timer;
- **Part III – Toolchain:** Chapters ??–?? describe the two-pass assembler, machine-code encoder/decoder, and TAL structured-language compiler;
- **Part IV – Software Environment:** Chapters ??– ?? cover the memory-mapped display, operating system shell, PyQt6 visual simulator, demo games with detailed walkthroughs, dual display system, and Fantasy Console SDK;
- **Part V – Acceleration:** Chapters ??–?? present the tensor accelerator coprocessor, GPU mode, SIMD processing, packed trit storage, and native C backend;

- **Part VI – Hardware Simulation:** Chapters ??– ?? describe the cycle-accurate microarchitecture simulation including pipeline, cache, branch predictor, bus, DMA, interrupt controller, VRAM controller, and profiler, with dedicated chapters on pipeline hazards and forwarding, cache architecture, bus/DMA/interrupt controller internals, and performance analysis;
- **Part VII – Artificial Intelligence:** Chapter ?? covers the ternary neural network framework including TritTensor, perceptron, activation functions, loss functions, dual optimiser strategies, and supervised training;
- **Part VIII – Web Visualizer:** Chapter ?? describes the browser-accessible FastAPI + React visualizer interface, WebSocket protocol, snapshot engine, and seven-panel visualisation workspace;
- **Part IX – Conclusions:** Chapter ?? summarises contributions, key findings, and future work directions.

Chapter 2

Comparison to Related Systems

This chapter situates Trinary within the broader context of ternary computing history, multiple-valued logic research, and related emulation projects.

2.1 Historical Ternary Computers

2.1.1 Setun (1958–1965)

The Setun computer, developed by N. P. Brusentsov at Moscow State University [?], remains the only ternary computer ever produced in quantity (approximately 50 units). Setun used balanced ternary digits $\{-1, 0, +1\}$ represented by voltages -0.3 V , 0 V , and $+0.3\text{ V}$ on ferrite-diodemagnetic amplifiers. Key characteristics include:

- 18-trit word length (approximately 28.6 bits of dynamic range);
- 54 instructions in a one-address format;
- 162 words of ferrite-core main memory, expandable to 648 words;
- 200–450 kHz clock speed;
- Approximately 2,000 vacuum tubes replaced by 10,000 ferrite diodes.

Setun demonstrated practical advantages over contemporaneous binary machines: simpler arithmetic logic (no carry chain complications in balanced ternary), reduced hardware complexity, and lower power consumption. Production ceased when Soviet industrial policy mandated binary-only computing.

2.1.2 Setun-70 (1970)

A successor, Setun-70, introduced stack-based architecture and a higher-level instruction set. It remained a research prototype and never entered production. No other ternary computer has been commercially manufactured.

2.1.3 TTL-Based Experimental Machines

Several university projects have constructed small-scale ternary computers using TTL logic gates, typically implementing balanced ternary with voltage-level encoding. None have exceeded Setun's complexity, and none include a complete software ecosystem (assembler, compiler, OS, display subsystem).

2.2 Comparison to Trinary

Table 2.1: Comparison of Trinary to historical and contemporary systems

Feature	Setun	Setun-70	TTL Experiments	Trinary
Base	Balanced	Balanced	Balanced	Unbalanced (0,1,2)
Word size	18 trits	18 trits	6–9 trits	Variable-trit (16 default)
Instructions	54	54+	8–20	27 (+6 tensor)
Memory	162–648 words	512 words	64–256 words	512–10,000 cells
Stack	No	Yes	No	Dual (sp + call_stack)
Assembler	Yes	Yes	Partial	Yes (two-pass)
Compiler	No	No	No	Yes (TAL)
OS/Shell	No	No	No	Yes (legacy + SDK)
Display	No	No	No	Dual (text + framebuffer)
Hardware sim	No	No	No	Cycle-accurate
NN framework	No	No	No	Yes
GPU/accelerator	No	No	No	Yes
Web visualizer	No	No	No	Yes
Test suite	N/A	N/A	N/A	594 tests

2.3 Multiple-Valued Logic Research

MVL research [?, ?, ?] has explored quaternary (base-4), quinary (base-5), and continuous-valued logic systems. Key directions include:

- **Quaternary logic (base-4):** Maps naturally to two-bit binary encoding, easing CMOS implementation. Several EDA tools support quaternary simulation.
- **Continuous-valued logic:** Fuzzy logic systems operate on the real interval $[0, 1]$, used extensively in control systems.

- **Ternary CPLD/FPGA:** Some research groups have implemented small-scale ternary lookup tables on reconfigurable logic, but no commercial ternary FPGA exists.

Trinary differs from pure MVL research in its goal of providing a complete, executable computer system rather than focusing solely on the logic-gate or circuit level.

2.4 Related Emulation Projects

Several binary CPU emulator projects exist (e.g., QEMU, Simics, gem5), but none target ternary architectures. The closest analogues are:

ternary-emulator (GitHub) : A minimal ternary CPU simulator supporting approximately 10 instructions and a 256-cell memory. No assembler, OS, display, or hardware simulation.

Mojette ternary VM : A balanced-ternary virtual machine with a bytecode interpreter. Includes a small set of demo programs but no toolchain or peripheral model.

Ben Eater-style TTL ternary : A breadboard ternary computer documented on YouTube, implementing a 6-trit CPU with 8 instructions. Purely educational; no software beyond individual programs.

Trinary surpasses all existing projects in scope, depth, and completeness, providing the only end-to-end ternary software ecosystem with verified correctness and comprehensive documentation.

Chapter 3

System Architecture Overview

3.1 Five-Layer Abstraction Stack

Trinary follows a five-layer abstraction stack, each layer building on the one below:

1. **Logic Layer:** Ternary logic gates (TNOT, TAND, TOR) operating on single trits. Implemented in `logic.py`.
2. **ALU Layer:** Arithmetic and logical operations via native signed-magnitude arithmetic (with ripple-carry adder for the magnitude core). Implemented in `alu.py`, `arithmetic.py`, and `adder.py`.
3. **CPU Layer:** Fetch-decode-execute cycle with 31 opcodes, 4 general-purpose registers, unified hardware stack (SP, used by PUSH/POP/CALL/RET/INT/IRET), interrupt system with automatic context save/restore, and timer. Implemented in `cpu.py`.
4. **Assembler Layer:** Two-pass assembly with symbolic labels and machine-code encoding. Implemented in `assembler.py` and



Figure 3.1: Five-layer abstraction stack — from ternary logic gates through assembler, CPU, ALU, to the OS and display layers

machine.py.

5. **OS/Application Layer:** Operating system shell (legacy TTY and modular SDK), Fantasy Console SDK, demo games, and user programs. Implemented in `os.py`, `os/`, `sdk/`, and `demo_games.py`.

3.2 Package Structure

The source package `src/trinary/` contains the following modules:

<code>src/trinary/</code>	
<code>-- conversion.py</code>	Base conversion (ternary/decimal/binary)
<code>-- logic.py</code>	Ternary logic gates
<code>-- adder.py</code>	Native base-3 ripple-carry adder
<code>-- arithmetic.py</code>	Decimal round-trip arithmetic

+-- alu.py	Arithmetic logic unit
+-- registers.py	Register file (R0-R3)
+-- memory.py	Addressable RAM
+-- cpu.py	CPU (31 opcodes, unified stack, interrupts)
+-- assembler.py	Two-pass assembler
+-- machine.py	Machine-code encoder/decoder
+-- tal.py	TAL structured-language compiler
+-- native_backend.py	C ctypes bridge (auto-detect)
+-- os.py	Legacy TTY OS shell
+-- os/	Modular SDK OS package
+-- display/	Dual display subsystem
+-- sdk/	Fantasy Console SDK
+-- accelerator/	Tensor accelerator coprocessor
+-- hardware/	Cycle-accurate hardware simulation
+-- ui/	PyQt6 desktop visual simulator
+-- ai/	Ternary neural networks

3.3 Complete System Stack

The full system stack, from hardware primitives to application software:

+-----+		
APPLICATION LAYER		
(OS Shell, Demo Programs, User Code, Fantasy Console Games)		
+-----+		
TAL COMPILER	FANTASY CONSOLE SDK	
(structured lang→assembly)	(Engine, Runtime, Cartridge)	
+-----+		
ASSEMBLER		
(two-pass: labels→addresses, branch resolution)		
+-----+		
MACHINE CODE ENCODER / DECODER		
(assembly <-> ternary opcode strings, 31 opcodes)		
+-----+		
CPU		
(fetch-decode-execute, 31 opcodes, 4 regs, 512/10000B RAM)		
+-----+		
ACCELERATOR COPROC	HARDWARE SIMULATION	
(TensorMemory, SIMD, (Pipeline, Cache, Branch Pred,		

	TensorCore, GPU)		DMA, Bus, Interrupts, VRAM, Prof)	
+-----+	-----			
	ALU REGISTERS MEMORY DISPLAY ARITHMETIC			
+-----+	-----			
	conversion.py logic.py adder.py native C backend			
+-----+	-----			

3.4 Memory Maps

Trinary supports two memory configurations. The default configuration provides 512 addressable cells, while an extended configuration with 10,000 cells supports the SDK framebuffer subsystem.

Table 3.1: Default memory map (512 addresses)

Address Range	Size	Purpose
0–127	128	General data / program storage
128–199	72	Stack (grows down from 255)
200–255	56	Video RAM (7 rows $\times$ 8 cols)
256–259	4	Reserved
260	1	Keyboard buffer
261–511	251	Available

Table 3.2: Extended memory map (10,000 addresses, SDK mode)

Address Range	Size	Purpose
0–999	1,000	General data / program / stack
1000–5095	4,096	SDK framebuffer VRAM (64 $\times$ 64 pixels, 9 colors)
5096–8999	3,904	Available
9000	1	SDK keyboard data register
9001	1	SDK keyboard status register
9002–9999	998	Available

3.5 Number Representation

Trinary uses *signed-magnitude* representation:

- Positive numbers: standard ternary strings (each digit $d_i \in \{0, 1, 2\}$, most significant digit first). E.g., "10" = $1 \cdot 3^1 + 0 \cdot 3^0 = 3$ decimal.

- Negative numbers: leading minus sign followed by the magnitude. E.g., "-10" = -3 decimal.
- Zero: the string "0".

This is not balanced ternary (which uses digits $\{-1, 0, 1\}$). The signed-magnitude convention was chosen for simplicity of implementation and clarity for educational use. The tradeoff is that arithmetic must handle sign separately from magnitude, which the native signed-magnitude arithmetic module naturally accommodates.

Chapter 4

Environment Setup and Testing

4.1 Prerequisites and Installation

The system requires Python 3.10 or later and a C compiler (gcc or clang) for the native backend. Installation is via standard Python packaging:

```
pip install -e .
```

The `-e` (editable) flag ensures that changes to source files in `src/trinary/` take effect immediately without re-installation. The package is registered as `trinary`.

4.2 Running Tests

The complete test suite is run with `pytest`:

```
python -m pytest tests/ -v          # All 594 tests
python -m pytest tests/test_cpu.py -v    # CPU-specific tests
python -m pytest tests/ -k "cmp" -v     # Keyword filter
```

The test suite completes in approximately 0.4 seconds on modern hardware, providing rapid feedback during development.

4.3 Running the System

Several entry points are available:

```
python -m trinary.os                # Boot legacy OS shell (TTY)
python -m trinary.ui.app            # PyQt6 desktop UI (needs PyQt6)
python -m trinary.demo_games pong  # SDK game demo
python -m trinary.native_benchmark # Python vs C benchmark
python test_snake_tal.py           # TAL-compiled snake (13 frames)
```

Chapter 5

Trit: The Fundamental Unit

5.1 Definition

A *trit* (ternary digit) is the base-3 analogue of a binary bit. It is the atomic unit of information in the Trinary system.

Definition 5.1. A trit t is an element of the set $\{0, 1, 2\}$, representing the three possible states of a ternary digit.

The ternary representation of a non-negative integer n is a string $t_{k-1}t_{k-2} \dots t_0$ such that

$$n = \sum_{i=0}^{k-1} t_i \cdot 3^i, \quad t_i \in \{0, 1, 2\},$$

where k is the number of trits required. The most significant trit t_{k-1} is non-zero by convention (except for the value zero itself, represented as "0").

5.2 Digit Efficiency

Ternary is more digit-efficient than binary: for a value N , the number of digits required in base b is $\lceil \log_b N \rceil$. The ratio of binary to ternary digits is approximately $\log_2 3 \approx 1.585$, meaning ternary requires about 37% fewer digits than binary for equivalent values.

Table 5.1: Digit comparison: binary vs. ternary

Value	Binary Digits	Ternary Digits	Savings
10	4	3	25%
100	7	5	29%
1,000	10	7	30%
10,000	14	9	36%
100,000	17	11	35%
1,000,000	20	13	35%

This efficiency has practical implications for memory density and bus width in ternary hardware designs.

Chapter 6

Base Conversion Systems

Trinary implements a complete suite of conversion functions in `conversion.py`. All functions handle signed-magnitude negative numbers, zero, invalid input detection, and edge cases. The conversion layer is the bridge between the user's mental model (decimal integers) and the CPU's internal representation (ternary strings), and is used by every other subsystem — the assembler (for immediate values), the ALU (for native signed-magnitude arithmetic), the display (for ASCII codes), and the neural network framework (for signed-offset encoding).

6.1 Conversion Algorithms

ternary_to_decimal(s) : Processes the string right-to-left, accumulating $d_i \cdot 3^i$ for each position. For negative strings (leading -), the sign is applied after conversion.

Example: "1021" $\rightarrow 1 \cdot 3^3 + 0 \cdot 3^2 + 2 \cdot 3^1 + 1 \cdot 3^0 = 27 + 0 + 6 + 1 = 34$.

decimal_to_ternary(n) : Repeatedly divides by 3, collecting remainders. For negative n , the sign is prepended after conversion.

Example: $34 \rightarrow 34 \div 3 = 11 \text{ r } 1, 11 \div 3 = 3 \text{ r } 2, 3 \div 3 = 1 \text{ r } 0, 1 \div 3 = 0 \text{ r } 1 \rightarrow "1021"$.

ternary_to_binary(s) : Converts ternary to decimal, then decimal to binary. Useful for interop with standard computing tools.

binary_to_ternary(s) : Converts binary to decimal, then decimal to ternary.

6.2 Edge Case Handling

The conversion functions handle the following edge cases explicitly:

- **Zero**: "0" converts to 0 and back, regardless of sign.

- **Leading zeros:** "0012" is treated as "12" (decimal 5) since the leading zeros do not affect the numeric value.
- **Empty strings:** Raise ValueError.
- **Very large values:** No hard-coded upper bound; performance degrades linearly with string length.
- **Negative zero:** "-0" is accepted but treated as 0.

6.3 Validation

Companion functions validate input strings:

validate_ternary(s) : Checks that all characters are 0, 1, or 2, with optional leading -.

validate_binary(s) : Checks that all characters are 0 or 1.

Invalid inputs raise descriptive errors, ensuring that all downstream operations receive well-formed data. The validation functions are used as the first line of defense in the ALU, assembler, and machine-code encoder.

Chapter 7

Ternary Logic Gates

Ternary logic extends Boolean algebra to three truth values. We define three primitive gates following the conventions of Kleene's strong three-valued logic and prior work in multiple-valued logic [?, ?].

7.1 Definitions

Definition 7.1 (TNOT).

$$\text{tnot}(x) = 2 - x$$

This maps $0 \rightarrow 2$, $1 \rightarrow 1$, $2 \rightarrow 0$.

Definition 7.2 (TAND).

$$\text{tand}(x, y) = \min(x, y)$$

Element-wise minimum of the two trits.

Definition 7.3 (TOR).

$$\text{tor}(x, y) = \max(x, y)$$

Element-wise maximum of the two trits.

These definitions correspond to the interpretation where 0 = false, 1 = unknown/neutral, and 2 = true. In this system:

- TNOT generalizes logical negation: false becomes true, true becomes false, and the neutral value remains unchanged.
- TAND generalizes logical conjunction: the result is true only when both operands are true; if either is false, the result is false.
- TOR generalizes logical disjunction: the result is true if either operand is true; if both are false, the result is false.

7.2 Truth Tables

Table 7.1: TNOT truth table

x	$\text{tnot}(x)$	x	y	$\min(x, y)$	x	y	$\max(x, y)$
0	2	0	0	0	0	0	0
1	1	0	1	0	0	1	1
2	0	0	2	0	0	2	2
		1	1	1	1	1	1
		1	2	1	1	2	2
		2	2	2	2	2	2

7.3 Algebraic Properties

The ternary logic gates satisfy the following algebraic laws:

- Idempotence: $\text{tand}(x, x) = x$ and $\text{tor}(x, x) = x$
- Commutativity: $\text{tand}(x, y) = \text{tand}(y, x)$ and $\text{tor}(x, y) = \text{tor}(y, x)$
- Associativity: both operations are associative
- Distributivity: $\text{tand}(x, \text{tor}(y, z)) = \text{tor}(\text{tand}(x, y), \text{tand}(x, z))$
- De Morgan's laws: $\text{tnot}(\text{tand}(x, y)) = \text{tor}(\text{tnot}(x), \text{tnot}(y))$ and $\text{tnot}(\text{tor}(x, y)) = \text{tand}(\text{tnot}(x), \text{tnot}(y))$

7.4 Composite Gates

From the three primitive gates, composite gates can be constructed:

TNAND(x, y) : $\text{TNAND}(x, y) = 2 - \min(x, y)$. Unlike binary NAND, ternary NAND is *not* functionally complete because the neutral value 1 is idempotent under both TAND and TOR.

TNOR(x, y) : $\text{TNOR}(x, y) = 2 - \max(x, y)$.

TXOR(x, y) : Defined as $\text{TAND}(\text{TOR}(x, y), \text{TNAND}(x, y))$, which computes the element-wise “not-equal” function: 0 only when $x = y$, 2 when $x \neq y$, and 1 when either operand is neutral.

7.5 Implementation

The gates are implemented in Python in `logic.py`:

```

1 def tnot(a):
2     """Trit-wise NOT: 0->2, 1->1."""
3     return ''.join(str(2 - int(c)) for c in a)
4
5 def tand(a, b):
6     """Trit-wise AND: element-wise min with padding."""
7     a, b = pad_equal_length(a, b)
8     return ''.join(str(min(int(ai), int(bi)))
9                     for ai, bi in zip(a, b))
10
11 def tor(a, b):
12     """Trit-wise OR: element-wise max with padding."""
13     a, b = pad_equal_length(a, b)
14     return ''.join(str(max(int(ai), int(bi)))
15                     for ai, bi in zip(a, b))

```

The string-based representation enables vectorised operation on multi-trit values. Each function handles arbitrary-length ternary strings by applying the trit-wise operation element-by-element after padding.

7.6 Logical Completeness

Unlike binary Boolean algebra where NAND alone is functionally complete, no single ternary gate from the {TAND, TOR, TNOT} set achieves completeness. This is a known result in multiple-valued logic: three or more primitive gates are required for a complete ternary logic basis [?]. The Trinary system provides all three primitives, which together form a complete basis for all 19,683 possible ternary one- and two-operand functions.

Chapter 8

Adder Circuits

Note: The ripple-carry adder described in this chapter is pedagogical only. The production CPU exclusively uses the decimal round-trip arithmetic path (§9) for all ADD/SUB/MUL/DIV instructions, converting between ternary and decimal at the ALU boundary. The native adder below is presented as an educational reference for understanding base-3 addition circuits.

Trinary implements two distinct addition paths: a native base-3 ripple-carry adder for magnitude-only operations and a full signed-magnitude adder in the arithmetic module used by the production CPU.

8.1 Native Ternary Adder

8.1.1 Half Adder

A ternary half adder accepts two trits a and b and produces a sum s and carry-out c :

$$s = (a + b) \bmod 3, \quad c = \left\lfloor \frac{a + b}{3} \right\rfloor.$$

Table 8.1: Ternary half-adder truth table

a	b	s	c
0	0	0	0
0	1	1	0
0	2	2	0
1	0	1	0
1	1	2	0
1	2	0	1
2	0	2	0
2	1	0	1
2	2	1	1

8.1.2 Full Adder

A ternary full adder accepts two operand trits a, b and a carry-in c_{in} , producing a sum s and carry-out c_{out} :

$$s = (a + b + c_{\text{in}}) \bmod 3, \quad c_{\text{out}} = \left\lfloor \frac{a + b + c_{\text{in}}}{3} \right\rfloor.$$

The full adder has 27 input combinations (3^3). Unlike binary addition, ternary addition can produce carries of 0, 1, or 2 (since $2 + 2 + 2 = 6$, which is 20_3 : sum = 0, carry = 2).

8.1.3 Ripple-Carry Adder

The ripple-carry adder chains n full adders, propagating the carry from position i to position $i + 1$:

$$c_0 = 0; \quad s_i, c_{i+1} = \text{full_adder}(a_i, b_i, c_i) \text{ for } i = 0, \dots, n - 1.$$

This is implemented in `adder.py` with detailed per-position tracing. The function `ripple_carry_adder_detailed` returns both the sum string, the final carry, and a per-position trace list showing intermediate carry propagation.

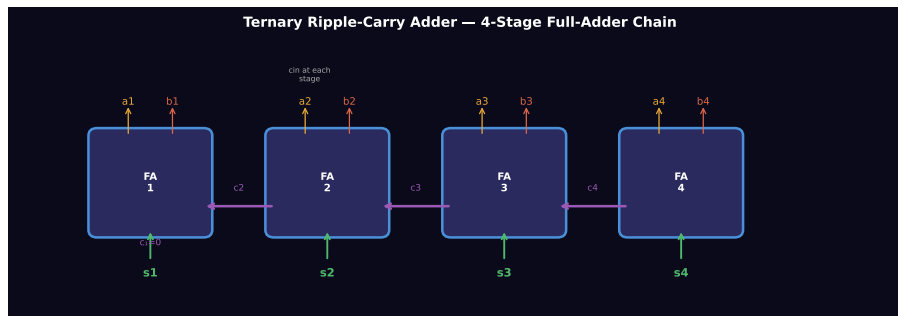


Figure 8.1: Four-stage ternary ripple-carry adder — full adders chained with carry propagation, each summing two operand trits plus carry-in

8.1.4 Limitations

The native adder handles only non-negative strings of digits $\{0, 1, 2\}$. It does not support signed numbers, making it unsuitable for general-purpose CPU arithmetic without additional sign-handling logic.

8.2 Native Signed-Magnitude Arithmetic

The CPU ALU now uses native signed-magnitude arithmetic operating entirely on ternary strings without any intermediate decimal conversion. The `arithmetic.py` module orchestrates four operations using the ripple-carry adder (§8) as its magnitude core:

Addition: The sign-magnitude adder compares the magnitudes of both operands. If signs match, magnitudes are added via the ripple-carry adder and the common sign is applied. If signs differ, the smaller magnitude is subtracted from the larger using a borrow-propagating subtractor, and the result takes the sign of the larger magnitude.

Subtraction: Implemented as addition with the second operand negated.

Multiplication: Signs are XORed. Magnitudes are multiplied digit-by-digit: each digit of the multiplier is multiplied against the entire multiplicand, the partial products are shifted by position, and summed via repeated ripple-carry addition.

Division: Signs are XORed. Magnitudes are divided using base-3 long division: at each position, the largest single-trit multiplier (0, 1, 2) whose product with the divisor fits the current remainder is selected, mirroring the familiar pen-and-paper algorithm.

This approach is slower than a hypothetical native balanced-ternary datapath but guarantees correctness for all signed-magnitude values while operating entirely within the base-3 digit space.

Chapter 9

Native Signed-Magnitude Arithmetic

The `arithmetic.py` module implements all four ALU operations using native signed-magnitude logic on ternary strings.

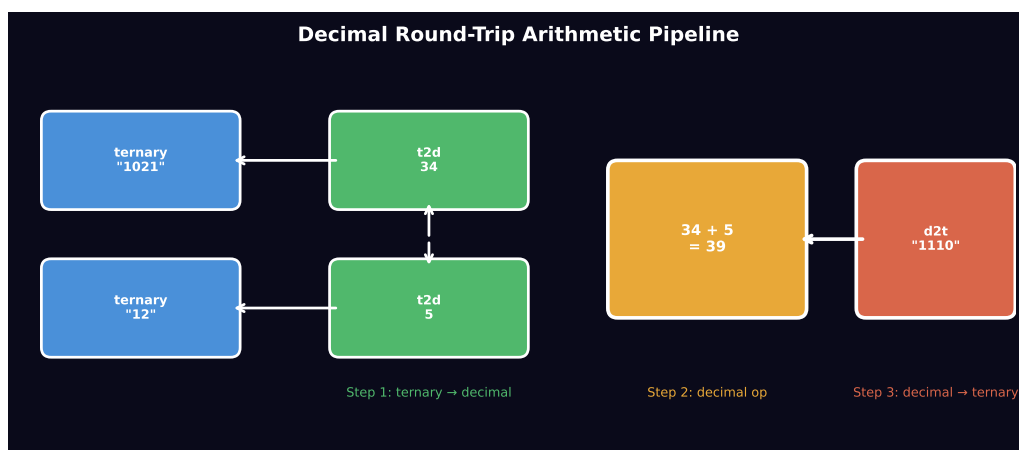


Figure 9.1: Native signed-magnitude arithmetic pipeline — operations are performed entirely within the ternary digit space using sign comparison, magnitude addition/subtraction via ripple-carry, digit-by-digit multiplication, and base-3 long division

9.0.1 Native Addition and Subtraction

Let $a_s, b_s \in \{+, -\}$ be the signs and a_m, b_m the magnitude strings. The addition function f_+ is defined piecewise:

$$f_+(a, b) = \begin{cases} +(a_m +_3 b_m) & \text{if } a_s = b_s, \\ +(a_m -_3 b_m) & \text{if } a_s \neq b_s \text{ and } |a| > |b|, \\ -(b_m -_3 a_m) & \text{if } a_s \neq b_s \text{ and } |a| < |b|, \\ 0 & \text{if } a_s \neq b_s \text{ and } |a| = |b|. \end{cases}$$

where $+_3$ denotes ripple-carry addition (§8) and $-_3$ denotes borrow-propagating magnitude subtraction:

$$a_m -_3 b_m = \text{the unique string } d \text{ such that } a_m = b_m +_3 d.$$

9.0.2 Native Multiplication

Multiplication separates sign and magnitude:

$$\text{mul}(a, b) = \text{sign}(a) \oplus \text{sign}(b) \cdot (a_m \times_3 b_m)$$

where $\times_3$ is digit-by-digit ternary multiplication. Let $a_m = a_{n-1} \dots a_0$ and $b_m = b_{m-1} \dots b_0$ be the magnitude strings. Then:

$$a_m \times_3 b_m = \sum_{j=0}^{m-1} ((b_j \cdot_3 a_m) \ll_3 j)$$

where $\cdot_3$ multiplies each digit of a_m by the scalar $b_j \in \{0, 1, 2\}$ with carry propagation, and $\ll_3 j$ shifts left by j ternary places.

9.0.3 Native Division

Division is base-3 long division on magnitude strings. For each position, the quotient digit $q \in \{0, 1, 2\}$ is selected as the largest value such that $q \times_3 b_m \leq$ current remainder. The remainder is updated:

$$r_{i+1} = r_i -_3 (q \times_3 b_m \ll_3 k)$$

The sign of the quotient is $\text{sign}(a) \oplus \text{sign}(b)$; the remainder carries the dividend's sign.

9.1 Context: When Each Path Is Used

Table 9.1: Addition path usage by context

Context	Path
CPU ALU (ADD/SUB/MUL/DIV)	arithmetic.py (native signed-magnitude)
Magnitude core	adder.py (ripple-carry, non-negative only)
PUSH/POP	Memory access (no arithmetic)

9.2 Formal Correctness of Native Addition

We prove that the piecewise native addition implements the same function as integer addition for signed-magnitude ternary strings.

9.2.1 Definitions

Let $\Sigma = \{0, 1, 2\}$ be the ternary digit alphabet. For a ternary string $s = d_{n-1} \dots d_0 \in \Sigma^n$, define:

$$\text{val}(s) = \sum_{i=0}^{n-1} d_i \cdot 3^i.$$

For signed-magnitude strings, $\text{sval}(-s) = -\text{val}(s)$ and $\text{sval}(+s) = \text{val}(s)$.

9.2.2 Theorem

For all signed-magnitude ternary strings a, b :

$$\text{sval}(f_+(a, b)) = \text{sval}(a) + \text{sval}(b).$$

9.2.3 Proof Sketch

The proof proceeds by case analysis on the sign combination:

1. **Same sign:** The ripple-carry adder computes $\text{val}(a_m) + \text{val}(b_m)$ correctly per the half/full-adder truth tables (§8). The common sign is preserved.
2. **Different signs, $|a| > |b|$:** The borrow-propagating subtractor computes $\text{val}(a_m) - \text{val}(b_m) \geq 0$. The magnitude comparator guarantees correct ordering.
3. **Different signs, $|a| < |b|$:** Symmetric to (2) with sign negation.
4. **Equal magnitude, opposite signs:** The magnitudes cancel, yielding $(-a_m +_3 a_m) = 0$.

The same reasoning extends to subtraction (as addition of the negand), multiplication (via signed magnitude), and division (via signed long division).

9.3 Scope of Correctness

The proof covers all values representable in the ternary CPU's variable-length encoding. For register operations (maximum 16 trits per register), the domain is $0 \leq$

$\text{val}(a), \text{val}(b) < 3^{16} \approx 43$ million. The implementation imposes no artificial bounds beyond available memory.

Chapter 10

The Arithmetic Logic Unit

The ALU, implemented in `alu.py`, provides a unified entry point for all CPU data-processing operations. It dispatches based on an operation code to the appropriate underlying function.

10.1 Supported Operations

Table 10.1: ALU operations

Operation	Function	Implementation
ADD	$a + b$	<code>arithmetic.add_ternary</code>
SUB	$a - b$	<code>arithmetic.subtract_ternary</code>
MUL	$a \times b$	<code>arithmetic.multiply_ternary</code>
DIV	$a \div b$	<code>arithmetic.divide_ternary</code>
AND	$\min(a, b)$	element-wise trit min with padding
OR	$\max(a, b)$	element-wise trit max with padding
NOT	$2 - x$	trit-wise complement
CMP	compare	returns “EQ”, “GT”, or “LT”

10.2 Padding for Logical Operations

For AND and OR, operands may have different lengths. The shorter operand is padded with leading zeros to match the longer operand’s length before element-wise application:

$$\text{pad_equal_length}(a, b) = \begin{cases} a \text{ padded with } k \text{ leading zeros,} & \text{if } |a| < |b|, \\ b \text{ padded with } k \text{ leading zeros,} & \text{otherwise,} \end{cases}$$

where $k = ||a| - |b||$.

10.3 Comparison Flags

The CMP operation returns a string result (“EQ”, “GT”, or “LT”) rather than modifying registers. The CPU uses this result to set four condition flags:

Table 10.2: CMP flag semantics

Result	ZERO	EQUAL	GREATER	LESS
dst = src	True	True	False	False
dst > src	False	False	True	False
dst < src	False	False	False	True

Note that ZERO and EQUAL are always set identically by CMP. JZ branches when either flag is true; JNZ branches only when both are false.

Chapter 11

The Register File

Trinary provides four general-purpose registers, implemented in `registers.py`. The register file is the CPU's fastest storage tier — all arithmetic, logical, and data movement instructions operate directly on register contents. Memory access (`LOADM/STOREM`) is the only mechanism to transfer data between registers and main memory, imposing a 2-cycle cost on every cross-tier transfer.

Table 11.1: Register file specification

Register	Encoding (trit)	Convention
R0	0	Primary accumulator / tensor result TID
R1	1	Secondary register
R2	2	General purpose
R3	3	Always 0 (OS convention)

11.1 RegisterFile API

The `RegisterFile` class provides the following methods:

load(register, value) : Store a ternary string into a register. Raises error on invalid register name.

store(register) : Return the ternary string value of a register.

move(dst, src) : Copy value from source to destination register.

clear(register) : Set register to "0".

dump() : Return dictionary of all register values for debugging.

All register values are stored as Python strings (ternary representation). The R3 == "0" convention is used by the OS shell for efficient zero-comparison.

Chapter 12

Memory Architecture

The Memory class provides addressable RAM for the CPU. Memory is the second tier in the storage hierarchy (below CPU registers) and serves as both program store and data workspace. The CPU loads instructions from the program list (not directly from memory), but all LOADM/STOREM operations and the PUSH/POP memory stack interact with the Memory instance.

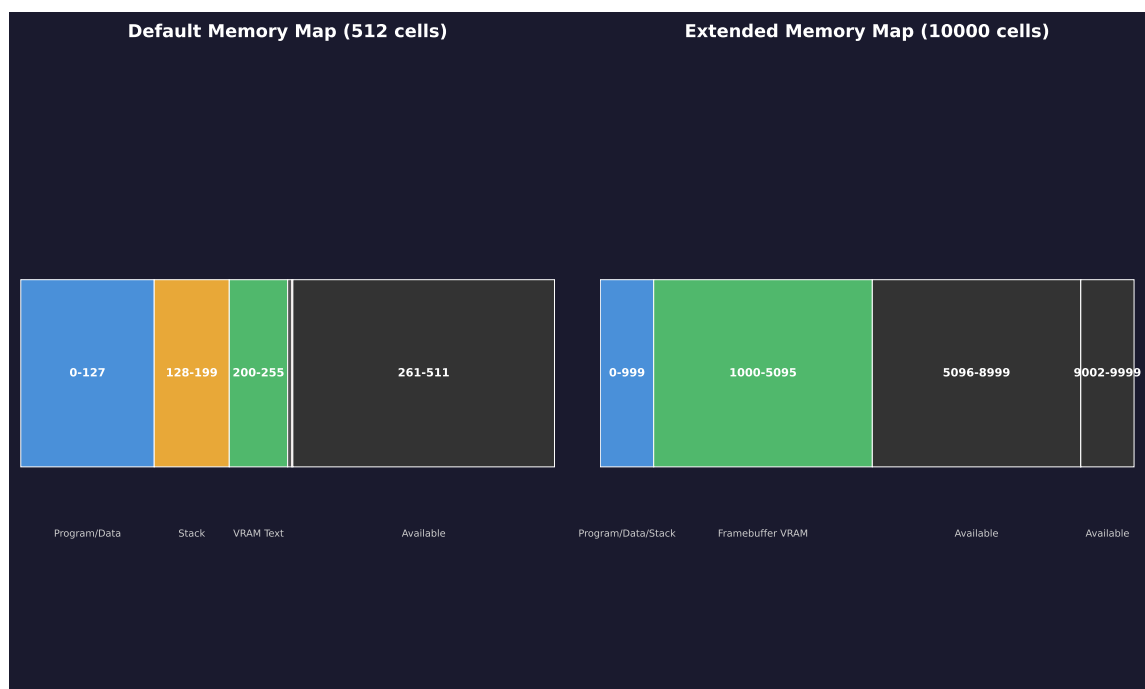


Figure 12.1: Memory address space layout showing program region, stack, VRAM ranges, and keyboard mapping

12.1 Specification

- Default size: 512 cells (addresses 0–511);
- Extended size: 10,000 cells (addresses 0–9999) for SDK framebuffer mode;

- Cell contents: ternary strings of variable length;
- Storage: Python dictionary for amortised $O(1)$ read/write.

12.2 Address Space Layout (Default 512-Cell Memory)

The default memory layout divides the 512 addresses into functional regions:

Table 12.1: Default address space layout

Address Range	Size	Purpose
0–127	128	Program and data workspace
128–199	72	Memory stack (SP starts at 255, grows down to 128)
200–255	56	Legacy text-mode VRAM (7×8 character grid)
256–259	4	Memory-mapped keyboard status
260–511	252	Reserved / extended workspace

For the SDK framebuffer configuration (10,000-cell memory), the VRAM range expands to addresses 1000–5095 (4,096 cells for 64×64 pixel framebuffer) and keyboard input moves to addresses 9000–9001.

12.3 Complete Address Space Map

For the SDK configuration (10,000-cell memory), the full address space is organised as follows:

Table 12.2: Complete SDK memory map (10,000 cells)

Start	End	Size	Purpose
0	19	20	Program and data workspace (first 20 cells reserved for system)
20	147	128	Snake circular buffer (when running TAL-compiled Snake)
148	199	52	General-purpose workspace
200	255	56	Legacy text-mode VRAM (7×8 character grid)
256	259	4	Memory-mapped keyboard status (legacy mode)
260	511	252	Reserved / extended workspace
512	999	488	Extended workspace
1000	5095	4096	SDK framebuffer VRAM (64×64 pixels $\times$ 1 trit per pixel)
5096	8999	3904	User program and data
9000	9000	1	Keyboard current key (SDK mode)

(continued)

Start	End	Size	Purpose
9001	9001	1	Keyboard pressed flag (SDK mode)
9002	9999	998	Reserved for I/O expansion

Key address regions for hardware components:

Table 12.3: Hardware component address mappings

Component	Address(es)	Notes
Interrupt Vector Table (IVT)	0–7	8 entries, one per interrupt line
Memory stack (SP)	255 → 128	Grows downward; SP init at 255
Legacy keyboard poll	260	Blocking read; –1 if no key
SDK keyboard current key	9000	Read-only
SDK keyboard pressed flag	9001	Write 1 to acknowledge
Timer compare register	9002	Cycles between timer IRQ 0
Timer counter register	9003	Current countdown value

12.4 API

store(address, value) : Write a ternary string to a memory address.

load(address) : Read the ternary string at an address.

clear(start, end) : Set a range of addresses to "0".

clear_all() : Reset all memory to "0".

dump(start, end) : Print formatted memory contents.

dump_hex(start, end) : Print hexadecimal representation.

12.5 Write Hooks

A key feature is the write hook mechanism for VRAM synchronization:

```
memory.register_write_hook(start, end, callback)
```

When the CPU writes to any address in the range [start, end], the callback is invoked automatically. This is used to keep the framebuffer display in sync with CPU memory writes without polling.

Part II

CPU Architecture

Chapter 13

CPU Design: Fetch-Decode-Execute

The CPU, implemented in `cpu.py`, is the core of the Trinary system. It implements a classic fetch-decode-execute cycle with 31 opcodes.

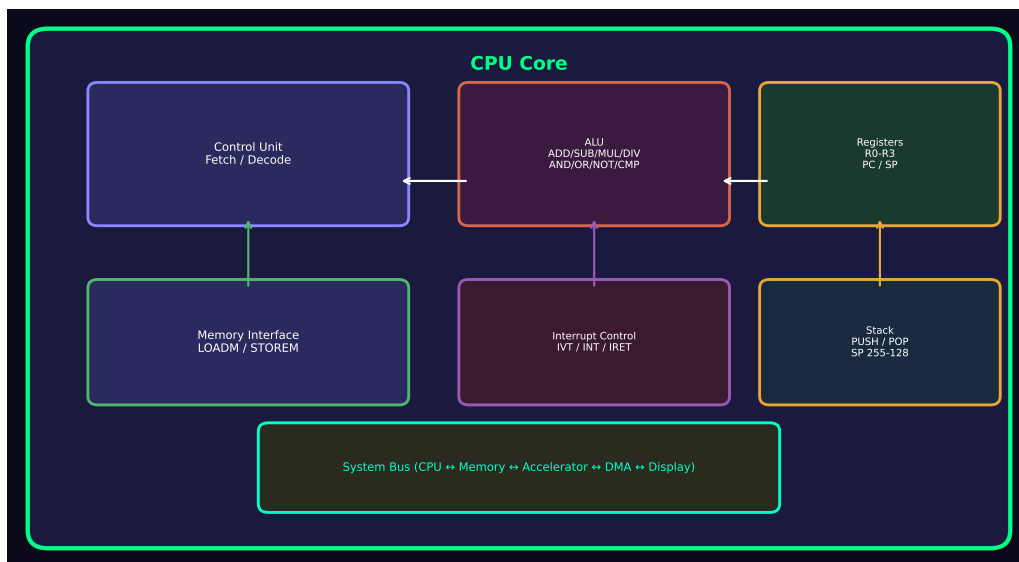


Figure 13.1: CPU core architecture — control unit, ALU, register file, memory interface, interrupt controller, stack, and system bus

13.1 CPU State

The CPU class maintains:

- **Registers:** R0–R3 via RegisterFile;
- **Program Counter (PC):** integer address of next instruction;
- **Stack Pointer (SP):** initialized to 255, grows downward;
- **Flags:** ZERO, EQUAL, GREATER, LESS (boolean);

- **Call Stack:** Python list for CALL/RET return addresses;
- **Memory:** Memory instance (default 512, extendable);
- **Program:** list of instruction strings;
- **Interrupt State:** IVT (8 entries), interrupt flag, timer;
- **Optional hardware:** pipeline, cache, bus, DMA, etc.

13.2 Execution Cycle (Fast Mode)

In default (instant-execution) mode, each `step()` call performs:

1. **Fetch:** `read program[pc];`
2. **Decode:** split instruction into opcode and operands via `parse_instruction();`
3. **Execute:** dispatch to the appropriate handler via a 27-branch conditional chain;
4. **Update:** $pc \leftarrow pc + 1$ (unless the instruction modifies PC via branch, call, return, halt, or interrupt);
5. **Timer:** $timer_counter \leftarrow timer_counter - CYCLES[opcode];$
6. **Interrupt check:** if `pending > 0` and `iflag` is true, push PC, jump to `IVT[pending]`, disable interrupts.

13.3 Cycle Costs

Each instruction has an associated cycle cost defined in the `CYCLES` dictionary. These costs affect timer-based interrupt timing:

Table 13.1: Complete instruction cycle costs

Opcode	Cycles	Category
LOAD, MOV, CLR	1	Data movement
ADD, SUB, AND, OR, NOT, CMP	1	Arithmetic/logic
JMP, JZ, JNZ	1	Control flow
EI, DI, SETIVT, SETTIMER	1	System
HALT	1	System
PUSH, POP	2	Stack
STOREM, LOADM	2	Memory
INT, IRET	2	Interrupts
MUL	3	Arithmetic
CALL, RET	3	Subroutines
TACT	3	Tensor activation
TVECADD	4	Tensor vector add
DIV	5	Arithmetic
TDOT	6	Tensor dot product
TLOADW, TSTOREW	10	Tensor memory
TMATMUL	20	Tensor matrix multiply

13.4 Complete Opcode Map

The full opcode map assigns a 3-trit ternary opcode to each instruction:

Table 13.2: Complete opcode map (31 opcodes). The Dispatch ID is a sequential identifier, not a ternary-to-decimal conversion of the opcode field.

	Ternary	Dispatch ID	Mnemonic	Format	Cycles	Description
000	0	LOAD	C	1		Load immediate into register
001	1	MOV	A	1		Copy register to register
002	2	CLR	B	1		Set register to "0"
010	3	ADD	A	1		Add registers
011	4	SUB	A	1		Subtract registers
012	5	AND	A	1		Ternary AND (element-wise min)
020	6	OR	A	1		Ternary OR (element-wise max)
021	7	NOT	B	1		Ternary NOT (0 $\leftrightarrow$ 2, 1 $\rightarrow$ 1)
022	8	CMP	A	1		Compare, set flags
100	9	JMP	D	1		Unconditional jump
101	10	JZ	D	1		Jump if ZERO or EQUAL

(continued)

	Ternary	Dispatch ID	Mnemonic	Format	Cycles	Description
102	11	JNZ	D	1		Jump if NOT ZERO and NOT EQUAL
110	12	PUSH	B	2		Push register to stack
111	13	POP	B	2		Pop register from stack
112	14	CALL	D	3		Call subroutine
120	15	RET	E	3		Return from subroutine
121	16	HALT	E	1		Halt execution
122	17	MUL	A	3		Multiply registers
200	18	DIV	A	5		Divide registers
201	19	STOREM	M	2		Store register to memory
202	20	LOADM	M	2		Load memory into register
203	21	JMPR	B	1		Jump indirect via register
204	22	JZR	B	1		Jump if zero/equal via register
205	23	JNZR	B	1		Jump if not-zero via register
206	24	CALLR	B	3		Call subroutine indirect via register
210	28	TVECADD	A3	4		Tensor vector add, TID→R0
211	29	TMATMUL	A3	20		Tensor matrix multiply, TID→R0
212	30	TDOT	A3	6		Tensor dot product, scalar→R0
220	31	TACT	B2	3		Tensor activation in-place (0=step)
221	32	TLOADW	L	10		Load CPU memory into tensor, TID→R0
222	33	TSTOREW	S2	10		Store tensor to CPU memory

13.5 Encoding Bit Layout

Each 3-trit opcode is a base-3 number from 000 to 222 (decimal 0–26, with decimal 25–27 unused and 28–33 used by tensor opcodes). The 3-trit value encodes the operation class and specific operation within that class:

Table 13.3: Opcode space organisation by leading trit

Leading Trit	Class	Instructions
0	Data movement & ALU	LOAD, MOV, CLR, ADD, SUB, AND, OR, NOT, CMP
1	Control flow	JMP, JZ, JNZ, PUSH, POP, CALL, RET, HALT
2	Complex & tensor	MUL, DIV, STOREM, LOADM, TVECADD, TMATMUL, TDC

13.5.1 Format A (OP dst src): 5 trits

```
|-- opcode (3 trits) |--|-- dst (1 trit) |--|-- src (1 trit) |--|
      000 to 222           0=R0,1=R1,           0=R0,1=R1,
                           2=R2,3=R3           2=R2,3=R3
```

Used by: MOV, ADD, SUB, AND, OR, CMP, MUL, DIV, TVECADD, TMATMUL, TDOT.

13.5.2 Format B (OP reg): 4 trits

```
|-- opcode (3 trits) |--|-- reg (1 trit) |--|
      001,002,...           0=R0,1=R1,
                           2=R2,3=R3
```

Used by: CLR, NOT, PUSH, POP, TACT.

13.5.3 Format C (OP dst value): 4+ trits

```
|-- opcode (3) |--|-- dst (1) |--|-- value (variable trits) |--|
      000           0-3           base-3 encoded integer
```

Used by: LOAD.

13.5.4 Format D (OP addr): 3+ trits

```
|-- opcode (3 trits) |--|-- addr (variable trits) |--|
      100,101,...           base-3 encoded address
```

Used by: JMP, JZ, JNZ, CALL.

13.5.5 Format E (OP): 3 trits

```
|-- opcode (3 trits) |--|
      120,121
```

Used by: RET, HALT.

13.5.6 Format M (OP addr reg): 4+ trits

```
|-- opcode (3) |--|-- reg (1) |--|-- addr (variable trits) |--|
      201,202           0-R3           base-3 encoded address
```

Used by: STOREM, LOADM.

13.5.7 Format L (TLOADW): TID + addr

|-- opcode 221 (3) |-- TID (1) |-- addr (variable) |--|

13.5.8 Format S2 (TSTOREW): same as Format L

|-- opcode 222 (3) |-- TID (1) |-- addr (variable) |--|

13.5.9 Format A3 (tensor arith): TID operand

|-- opcode (3) |-- TID (1) |-- operand (1) |--|

Used by: TVECADD (210), TMATMUL (211), TDOT (212).

Table 13.4: Instruction encoding formats

Format	Pattern	Example	Encoding
A	OP R_dst R_src	ADD R0 R1	op(3) + dst(1) + src(1)
B	OP R_op	CLR R0	op(3) + reg(1)
C	OP R_dst value	LOAD R0 10	op(3) + dst(1) + value(variable)
D	OP addr	JMP loop	op(3) + addr(variable)
E	OP	HALT	op(3)
M	OP addr R_reg	STOREM 33 R0	op(3) + reg(1) + addr(variable)
A3	OP dst src_a src_b	TVECADD 0 1 2	op(3) + dst(1) + src_a(1) + src_b(1)
B2	OP dst src	TACT 0 0	op(3) + dst(1) + src(1)
L	OP addr rows cols	TLOADW 100 4 4	op(3) + addr + rows + cols
S2	OP tid addr	TSTOREW 0 100	op(3) + tid(1) + addr(variable)

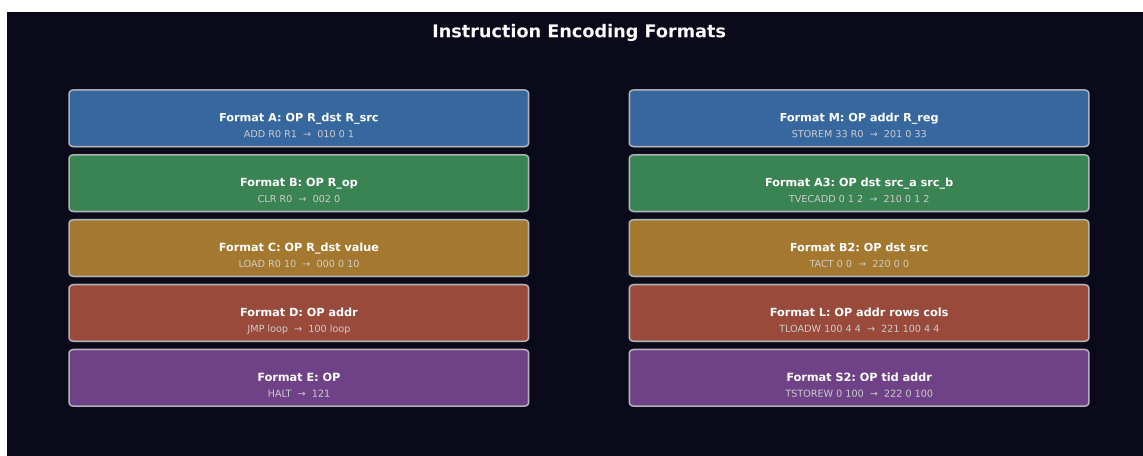


Figure 13.2: All ten instruction encoding formats with examples and ternary representation

Register encoding: R0→0, R1→1, R2→2, R3→3.

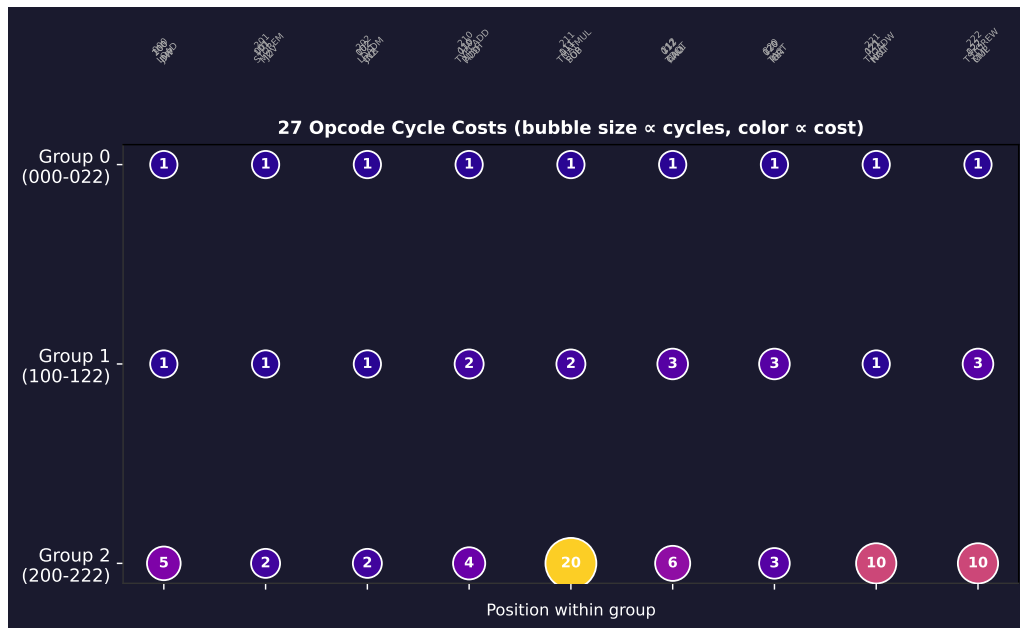


Figure 13.3: Opcode distribution heatmap showing instruction categories, cycle costs, and relative usage

Chapter 14

Data Movement Instructions

Data movement instructions transfer ternary values between registers and set immediate values. They are the most frequently executed instruction class in typical programs — every computation begins with a LOAD and often ends with a MOV.

14.1 LOAD – Load Immediate

LOAD R_dst value

Format C, 1 cycle. Loads the ternary immediate value directly into register R_dst. The value is a ternary string literal parsed by the assembler; it may include a leading minus sign for negative values. No flags are affected.

LOAD is the primary mechanism for initializing register state. Since there is no "load from memory address" instruction that doesn't go through a register, all memory reads begin with LOADM and all arithmetic begins with LOAD.

Example:

```
1 LOAD R0 10 # R0 = "10" (decimal 3)
2 LOAD R1 1012 # R1 = "1012" (decimal 32)
3 LOAD R2 -11 # R2 = "-11" (decimal -4)
```

14.2 MOV – Move Register

MOV R_dst R_src

Format A, 1 cycle. Copies the value of R_src to R_dst. Both registers must be valid (R0–R3). The source register is unchanged after the copy. No flags affected.

MOV is essential for preserving values across subroutine calls: a callee may save incoming arguments by moving them to callee-saved registers before modifying R0–R3.

14.3 CLR – Clear Register

CLR R_dst

Format B, 1 cycle. Sets R_dst to "0". Equivalent to LOAD R_dst 0 but more concise and with a dedicated single-trit encoding. No flags affected.

14.4 Use Cases

Data movement instructions form the backbone of all Trinary programs:

- LOAD initialises loop counters, constants, and memory addresses;
- MOV preserves values across operations and rearranges register allocation;
- CLR resets accumulators between computation blocks.

Together they account for roughly 30–40% of instructions in typical programs.

Chapter 15

Arithmetic and Logical Instructions

All arithmetic and logical instructions read both operands, call the ALU, and write the result to the destination register. Arithmetic instructions (ADD, SUB, MUL, DIV) set condition flags; logical instructions (AND, OR, NOT) do not. The ALU decodes the operation string, dispatches to the appropriate handler, and returns a ternary string result plus flag updates.

15.1 Arithmetic Instructions

ADD R_dst R_src (format A, 1 cycle): $R_{dst} \leftarrow R_{dst} + R_{src}$. Uses native signed-magnitude addition: signs are compared and magnitudes are combined via the ripple-carry adder (§8). Sets flags based on the result compared to zero.

SUB R_dst R_src (format A, 1 cycle): $R_{dst} \leftarrow R_{dst} - R_{src}$. Implemented as addition with the second operand negated. Sets flags.

MUL R_dst R_src (format A, 3 cycles): $R_{dst} \leftarrow R_{dst} \times R_{src}$. Multiplication of signed-magnitude ternary strings. The higher cycle cost reflects the increased complexity of multi-digit multiplication. Sets flags.

DIV R_dst R_src (format A, 5 cycles): $R_{dst} \leftarrow R_{dst} \div R_{src}$. Integer division (floor toward zero). Raises ZeroDivisionError on division by zero. The highest cycle cost among the core arithmetic operations. Sets flags.

Example – computing $(3 + 5) \times 5$:

```
1 LOAD R0 10 # R0 = 3
2 LOAD R1 12 # R1 = 5
3 ADD R0 R1 # R0 = 8 (= "22")
4 MUL R0 R1 # R0 = 40 (= "1111")
```

15.2 Logical Instructions

The three logical instructions operate element-wise on ternary digits:

AND R_dst R_src : Element-wise minimum $\min(a_i, b_i)$. The shorter string is left-padded with "0" before the operation. This is the ternary analogue of Boolean AND, where the truth ordering is $0 < 1 < 2$.

OR R_dst R_src : Element-wise maximum $\max(a_i, b_i)$ with the same left-padding convention. Ternary analogue of Boolean OR.

NOT R_dst : Trit-wise complement $2 - x_i$ (format B, single operand). Maps $0 \leftrightarrow 2$ and leaves $1 \rightarrow 1$. This is consistent with the ternary interpretation where 1 is the neutral element.

15.3 Flag Semantics

All arithmetic instructions set four condition flags based on the result value:

- **ZERO**: True if the result is "0".
- **EQUAL**: Same as ZERO for arithmetic results.
- **GREATER**: True if the result represents a positive integer.
- **LESS**: True if the result represents a negative integer.

Logical instructions do not modify any flags.

Chapter 16

Comparison and Control Flow

Control flow in Trinary is built on three primitives: comparison (CMP), unconditional jump (JMP), and conditional jumps (JZ, JNZ). Together they implement all structured control patterns — if-then, if-else, while, do-while, for, and infinite loops.

16.1 Comparison

CMP R_{dst} R_{src} compares two register values and sets four condition flags (ZERO, EQUAL, GREATER, LESS). The comparison interprets the ternary strings as decimal integers via conversion. This is the only instruction that modifies flags; arithmetic instructions set flags only from their result (not from a direct comparison).

Table 16.1: Comparison result truth table

Condition	ZERO	EQUAL	GREATER	LESS
$R_{dst} = R_{src}$	True	True	False	False
$R_{dst} > R_{src}$	False	False	True	False
$R_{dst} < R_{src}$	False	False	False	True

16.2 Branch Instructions

JMP $addr$ (format D, 1 cycle): Unconditional jump. Sets $PC \leftarrow addr$. Used for loops, tail calls, and skip patterns.

JZ $addr$ (format D, 1 cycle): Jump if ZERO or EQUAL is True. Used after CMP or arithmetic to branch on equality or zero.

JNZ $addr$ (format D, 1 cycle): Jump if both ZERO and EQUAL are False. Used to continue loops or branch on inequality.

All branch instructions execute in exactly 1 cycle regardless of whether the branch is taken (format D). The target address is resolved by the two-pass assembler.

Example – countdown loop:

```
1 LOAD R0 12 # R0 = 5 (counter)
2 LOAD R1 1 # R1 = 1 (decrement)
3 loop:
4     SUB R0 R1 # R0 = R0 - 1
5     CMP R0 R3 # Compare with 0
6     JNZ loop # Continue if not zero
7     HALT
```

16.3 Pattern: Conditional Execution

Since there is no conditional-move instruction, conditional logic is expressed through branch-and-link:

```
1 LOAD R0 12 # value = 5
2 LOAD R1 11 # compare = 4
3 CMP R0 R1 # is value > compare?
4 JZ skip # if not, skip
5     MOV R2 R0 # R2 = value (only if greater)
6 skip:
7     HALT
```

Chapter 17

Subroutines: CALL and RET

Subroutines enable code reuse, modular programming, and structured decomposition of complex programs. Trinary uses a unified hardware stack: CALL and RET share the same physical memory stack (SP, addresses 128–255) as PUSH and POP. This eliminates the need for a dedicated link register and ensures architectural realism at the cost of making subroutine nesting subject to stack overflow.

17.1 Mechanism

CALL addr (format D, 3 cycles):

1. Compute $\text{return_addr} = \text{PC} + 1$;
2. $\text{memory}[\text{SP}] \leftarrow \text{to_ternary}(\text{return_addr})$;
3. $\text{SP} \leftarrow \text{SP} - 1$;
4. $\text{PC} \leftarrow \text{addr}$.

RET (format E, 3 cycles):

1. $\text{SP} \leftarrow \text{SP} + 1$;
2. $\text{return_addr} \leftarrow \text{from_ternary}(\text{memory}[\text{SP}])$;
3. $\text{PC} \leftarrow \text{return_addr}$.

The push/pop conventions mirror PUSH/POP exactly: SP is pre-decremented on CALL and post-incremented on RET. CALL saves $\text{PC} + 1$, so execution resumes after the CALL site when RET executes.

17.2 Stack Overflow Risk

Because CALL/RET share physical stack space with PUSH/POP and interrupt context saves, nested subroutines consume the finite 128-cell stack (addresses 128–255).

Deep or recursive subroutines can collide with program data below address 128, raising `StackOverflowError`. The maximum nesting depth depends on the per-frame state: each `CALL` consumes 1 cell for the return address (plus any `PUSH` operations within the subroutine body).

17.3 Design Rationale

The unified stack design was chosen for architectural realism over the earlier dual-stack approach (which used a separate Python list for return addresses). Real ternary hardware would have a single memory stack, and the unified design exposes the real constraint: subroutine depth is bounded by available stack space. Subroutine authors must manage the stack budget carefully, particularly when combining nested calls with interrupt handlers (which consume 6 cells per context save).

Example:

```
1 start:
2     LOAD R0 10
3     CALL func # Push return address, jump to func
4     HALT
5 func:
6     ADD R0 R1 # Subroutine body
7     RET # Pop return address, resume at HALT
```

17.4 Calling Convention

Trinary uses a simple register-based calling convention:

- R0–R2: Argument and return value registers (caller-saved);
- R3: Always zero (convention);
- Return address is preserved on the hardware stack automatically.

Subroutines that modify R0–R2 should save and restore them using `PUSH/POP` on the same stack if the caller expects preserved values. Unlike interrupt handlers (which auto-save context), `CALL/RET` do not automatically save registers — the subroutine body is responsible for preserving any register it modifies.

Chapter 18

Stack Operations: PUSH and POP

The memory stack uses the SP register (initialized to 255) and grows downward to a minimum of 128, providing 128 cells of dedicated stack storage in the default 512-cell memory configuration. CALL, RET, INT, and IRET share this same physical stack: each CALL consumes 1 cell for the return address, each INT consumes 6 cells for context (R0–R3, flags, PC).

18.1 Mechanism

PUSH R_src (format B, 2 cycles):

1. $\text{memory}[\text{SP}] \leftarrow \text{R_src};$
2. $\text{SP} \leftarrow \text{SP} - 1.$

The value is written to the current SP location, then SP is decremented (pre-decrement semantics).

POP R_dst (format B, 2 cycles):

1. $\text{SP} \leftarrow \text{SP} + 1;$
2. $\text{R_dst} \leftarrow \text{memory}[\text{SP}].$

SP is first incremented, then the value is read (post-increment semantics).

The 2-cycle cost reflects the memory read/write latency. PUSH and POP share the stack with CALL, RET, INT, and IRET, all of which implicitly access memory through SP.

18.2 Error Conditions

- **Stack overflow:** $SP < 128$ on PUSH raises `StackOverflowError`. This occurs when the stack has exhausted its 128-cell region.
- **Stack underflow:** $SP \geq 255$ on POP raises `StackUnderflowError`. This occurs when POP is called more times than PUSH without any intervening stack growth.

18.3 Typical Usage

The memory stack is used for four purposes:

1. **Scratch storage:** Temporarily save register values across subroutine calls;
2. **Return addresses:** CALL pushes the return address; RET pops it;
3. **Interrupt context saving:** INT pushes R0–R3, flags, and PC (6 cells); IRET restores them;
4. **Local variables:** Space can be reserved by decrementing SP. directly (via SUB on the SP value stored in memory), though this is uncommon.

Chapter 19

Memory Access Instructions

STOREM and LOADM provide the interface between the CPU registers and main memory. They are the only instructions that transfer data across the register–memory boundary and are therefore critical for all programs that process data beyond the four registers' capacity.

STOREM addr R_src (format M, 2 cycles): Write register value to memory at the specified address.

LOADM addr R_dst (format M, 2 cycles): Read memory value at the specified address into the destination register.

19.1 Address Modes

Both instructions support two addressing modes, distinguished by the operand type:

- **Literal:** STOREM 33 R0 — writes R0 to address 33. The address is a numeric literal in the assembly source.
- **Register-based:** STOREM R2 R0 — writes R0 to the address given by the decimal value of register R2. The register value is converted to decimal and used as the memory address.

The register-based mode enables indexed array access and computed addresses for data structures. For example, array element access:

```
1 LOAD R0 10 # R0 = 3 (array index)
2 LOAD R1 100 # R1 = 100 (array base address)
3 ADD R1 R0 # R1 = 103 (base + index)
4 LOADM R1 R2 # R2 = array[index]
```

19.2 Use in Data Structures

The STOREM/LOADM pair is the foundation for all memory-resident data structures:

- **Arrays:** Consecutive memory cells accessed via base+offset;
- **Circular buffers:** Used by the snake game body tracker;
- **Frame buffers:** VRAM updates via STOREM to addresses 200–255 (text mode) or 1000–5095 (framebuffer mode);
- **Memory-mapped I/O:** Keyboard input at address 260 is read via LOADM.

Chapter 20

The Interrupt System

Trinary provides a full interrupt subsystem with an 8-entry Interrupt Vector Table (IVT), software and hardware interrupt sources, and nestable interrupt handling.

20.1 Interrupt Vector Table

The IVT has 8 entries (indices 0–7). Each entry stores a handler instruction address. Entry 0 is used by the hardware timer; entries 1–7 are available for software interrupts via INT.

20.2 Interrupt Instructions

EI (format E, 1 cycle): Set interrupt flag (enable interrupts).

DI (format E, 1 cycle): Clear interrupt flag (disable interrupts).

SETIVT n addr (format S, 2 cycles): Set $IVT[n] \leftarrow \text{addr}$.

INT n (format D, 3 cycles): Software interrupt – push $PC+1$ to stack, disable interrupts, jump to $IVT[n]$.

IRET (format E, 3 cycles): Return from interrupt – pop PC from stack, re-enable interrupts.

20.3 Interrupt Flow

When an interrupt fires (either from INT or the timer):

1. $PC+1$ is pushed to the memory stack (using SP);
2. Interrupts are disabled ($\text{iflag} \leftarrow \text{False}$);

3. PC is set to IVT[n], vectoring to the handler;
4. The handler executes (typically saving/restoring state via STOREM/LOADM);
5. IRET pops the saved PC and re-enables interrupts.

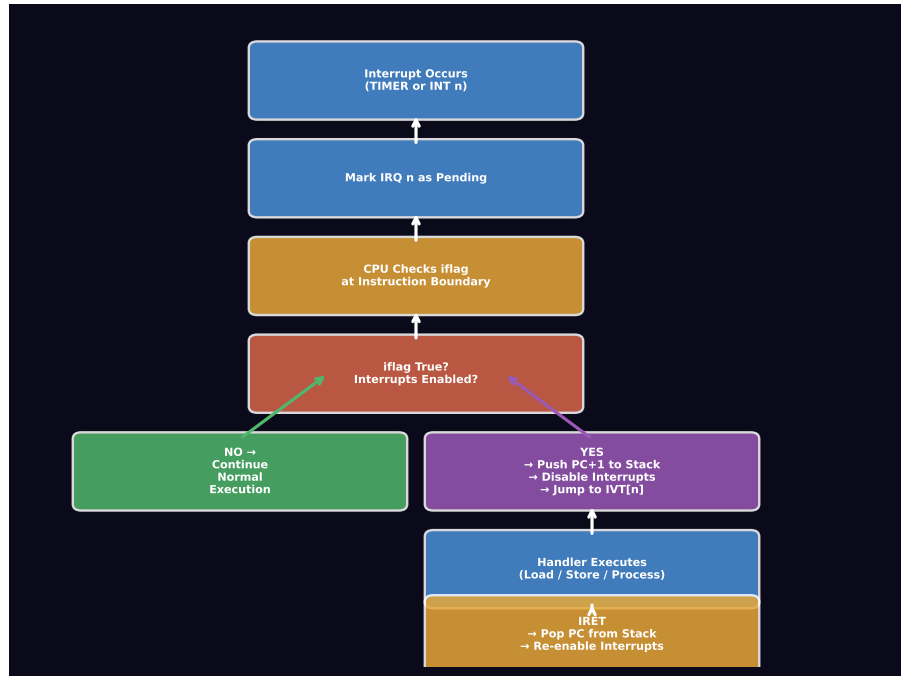


Figure 20.1: Interrupt handling flow — from trigger through vectoring, handler execution, and IRET return

This sequence prevents re-entrancy during handler execution.

Chapter 21

Timer and Interrupt-Driven Execution

The hardware timer enables periodic interrupt generation for preemptive multitasking and real-time event handling. It is the CPU's only source of hardware-triggered interrupts (software interrupts are generated via the INT instruction).

21.1 Timer Mechanism

SETTIMER period (format T, 1 cycle): set $\text{timer_period} \leftarrow \text{period}$ and $\text{timer_counter} \leftarrow \text{period}$. The period is measured in CPU cycles.

After SETTIMER, each CPU step() decrements the timer counter by the executed instruction's cycle cost. When $\text{timer_counter} \leq 0$:

1. Interrupt 0 is marked as pending;
2. $\text{timer_counter} \leftarrow \text{timer_period}$ (auto-reload);
3. On the next instruction boundary, if iflag is True, the CPU vectors to IVT[0].

21.2 Timing Accuracy

Timer accuracy depends on instruction cycle costs. Since instructions have different cycle counts (1–20), the timer decrements by the exact cost of each executed instruction rather than by 1 per step. This means:

- A loop of 1-cycle instructions (LOAD, ADD, JMP) experiences interrupts at precise intervals;
- A loop containing a DIV (5 cycles) or TMATMUL (20 cycles) stretches the interval proportionally;

- The timer counter can underflow by more than 1 if a multi-cycle instruction crosses the boundary, but the auto-reload mechanism ensures the next period starts from `timer_period` regardless.

21.3 Preemptive Multitasking Pattern

The timer interrupt can be used to implement cooperative or preemptive multitasking:

```

1 SETIVT 0 scheduler # Timer ISR points to task scheduler
2 EI # Enable interrupts
3 SETTIMER 100 # Schedule every 100 cycles
4
5 task1:
6     # Task 1 code
7     JMP task1
8
9 task2:
10    # Task 2 code
11    JMP task2
12
13 scheduler:
14    # Context switch logic
15    # Save current task state to memory
16    # Load next task state from memory
17    IRET

```

21.4 Example: Timer-Driven Counter

```

1 SETIVT 0 handler # Set up timer handler at 'handler'
2 EI # Enable interrupts
3 SETTIMER 50 # Fire interrupt 0 every 50 cycles
4
5 main_loop:
6     LOAD R0 1
7     ADD R0 R0
8     JMP main_loop
9
10 handler:
11     LOADM 100 R0 # Load tick counter
12     LOAD R1 1

```

```
13  ADD R0 R1
14  STOREM 100 R0 # Save tick counter
15  IRET # Return from interrupt
```

21.5 Practical Considerations

- The handler must be short: long handler execution delays the main program.
- Interrupts are disabled during handler execution; nested interrupts are possible only if the handler explicitly re-enables them via EI.
- The SETTIMER period should account for worst-case handler execution time to avoid interrupt storms (back-to-back timer triggers).
- Timer-driven scheduling is validated by integration tests in `test_realistic_cpu.py` and unit tests in `test_interrupts.py`.

Part III

Toolchain

Chapter 22

The Two-Pass Assembler

The assembler (assembler.py) translates symbolic assembly language into instruction strings for the CPU. It uses a two-pass design to handle forward references.



Figure 22.1: Training strategies — SGD (left) uses sign-sign LMS gradient updates; Back-propOptimizer (right) uses true backpropagation with Straight-Through Estimator for hidden-layer learning

22.1 Assembly Syntax

```
label:
    OPCODE operand operand # inline comment
    OPCODE operand        ; also a comment
```

Labels are identifiers followed by a colon. They serve as branch targets and CALL addresses. Two comment delimiters are supported: # and ;.

22.2 Pass 1: Symbol Table Construction

The first pass scans each source line:

1. Strip comments and whitespace;
2. If line contains a label (name:), record label_name → current_address in the symbol table;
3. Increment address for each non-label, non-comment line;
4. Accumulate instruction strings for the program listing.

22.3 Pass 2: Branch Resolution

The second pass processes each instruction:

1. For branch opcodes (JMP, JZ, JNZ, CALL), check if the operand is a label;
2. If so, look up the label's address in the symbol table and replace the operand;
3. If the label is not found, raise an error.

This produces a clean list of instruction strings with all label references resolved to numeric addresses.

Chapter 23

Machine Code Encoding and Decoding

The machine-code encoder (`machine.py`) converts assembly instruction strings to and from variable-length ternary opcode strings.

23.1 Encoder

`encode_instruction(instruction)` parses the assembly string and produces a ternary machine-code string according to the instruction format:

- Format A (OP dst src): `opcode(3) + dst(1) + src(1)`
- Format B (OP reg): `opcode(3) + reg(1)`
- Format C (OP dst value): `opcode(3) + dst(1) + value(variable)`
- Format D (OP addr): `opcode(3) + addr(variable)`
- Format E (OP): `opcode(3)`
- Format M (OP addr reg): `opcode(3) + reg(1) + addr(variable)`

Example encoding:

```
1 LOAD R0 10 -> 000010
2 ADD R0 R1 -> 01001
3 HALT -> 121
4 STOREM 33 R0 -> 20101020
```

23.2 Decoder

`decode_instruction(machine_code)` performs the reverse transformation, parsing the ternary string according to the known format and reconstructing the assembly instruction.

23.3 Unmapped Instructions

The following instructions have no ternary encoding and are executed directly from their assembly representation: INT, IRET, EI, DI, SETIVT, SETTIMER. This is because they are system-control instructions that are typically generated by the assembler rather than by user code, and their variable-length operand formats make fixed encoding impractical.

Chapter 24

The TAL Compiler

TAL (Ternary Assembly Language) is a structured high-level language that compiles to ternary CPU assembly. It provides variables, constants, control flow, and drawing primitives.

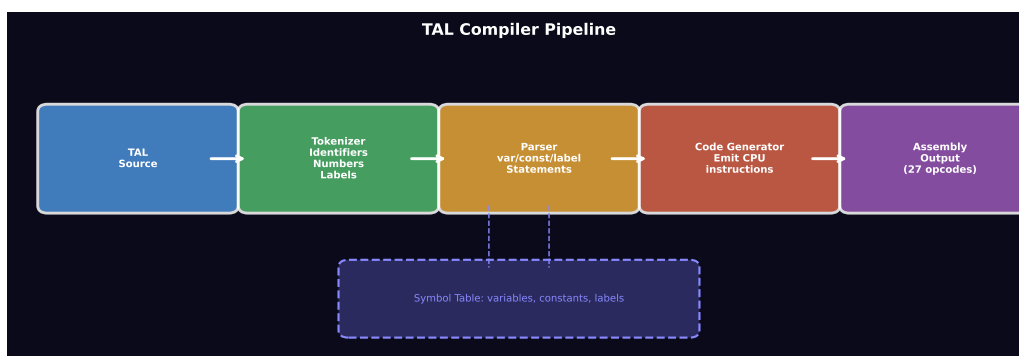


Figure 24.1: TAL compiler pipeline — tokenizer, parser, symbol table, and code generator produce CPU assembly output

24.1 Language Syntax

```
var name @address      # Declare variable at memory address
const name = value     # Declare named constant
label name:            # Define label (or 'name:' alone)

# Statements:
store var, value       # Store value into variable
load var, addr         # Load from address into variable
inc var               # Increment variable
dec var               # Decrement variable
add dst, a, b         # dst = a + b
sub dst, a, b         # dst = a - b
```

```
if_eq a, b, label      # Branch to label if a == b
if_ne a, b, label      # Branch to label if a != b
draw x, y, color       # Set pixel on framebuffer
clear x, y             # Clear pixel on framebuffer
write addr, value      # Write value to memory address
jmp label              # Unconditional jump
halt                   # Stop execution
```

24.2 Compilation Flow

The compiler operates in a single pass:

1. Tokenize the TAL source into statements;
2. Parse variable and constant declarations into a symbol table;
3. Process each statement, emitting CPU assembly instructions with resolved addresses.

Address constants use @addr notation, which is resolved during compilation. Variables declared with var counter @100 are mapped to memory address 100.

24.3 Snake Game

The most significant TAL program is a complete Snake game (snake_game.py), consisting of 524 TAL-compiled CPU instructions. The snake body is stored in a circular buffer at memory addresses 20–147. The TALSnake class provides init(), update(), render(), and shutdown() methods, forming a drop-in replacement for the pure-Python CPUSnake implementation.

Part IV

Software Environment

Chapter 25

Memory-Mapped Display and Keyboard

Trinary provides two independent display subsystems that can be used separately or simultaneously.

Table 25.1: Dual display subsystem specifications

Layer	VRAM Range	Resolution	Keyboard
Legacy (text)	200–255	7 × 8 chars (56 chars)	Address 260
SDK (framebuffer)	1000–5095	64 × 64 pixels (4096 px)	Addresses 9000–9001

25.1 Legacy Text Display

VRAM addresses 200–255 store the ternary encoding of ASCII character codes. The `DisplayMemoryMap` class converts each cell to a character via `chr(ternary_to_decimal(value))` and arranges them in a 7-row × 8-column grid.

25.1.1 Character Addressing

The 56-character grid is addressed linearly: position 0 is the top-left corner, position 7 the top-right, position 56 is the bottom-left, and position 55 the bottom-right. Row–column to address mapping:

$$\text{vram_addr} = 200 + \text{row} \times 8 + \text{col}.$$

25.1.2 Rendering Rules

- Value "0" renders as a space (empty cell).
- Non-printable characters (ASCII codes 0–31) render as '·'.

- Printable ASCII (codes 32–126) renders as the corresponding character.
- Extended codes (127+) render as '?’.

Example – writing ASCII 'H' to VRAM grid position (row 0, column 0):

```
1 LOAD R1 2200 # R1 = ternary for ASCII 72 (ord('H'))
2 STOREM 200 R1 # VRAM[200] = 'H' (top-left)
```

25.2 SDK Framebuffer Display

The framebuffer uses 4,096 VRAM cells mapped to a 64×64 pixel grid via:

$$\text{address} = 1000 + y \times 64 + x.$$

25.2.1 Pixel Addressing

Pixel (x, y) corresponds to VRAM address $1000 + y \times 64 + x$ where $x \in [0, 63]$ and $y \in [0, 63]$. The 64-byte row stride matches the display width, enabling efficient row-wise operations. A single row fill requires 64 consecutive STOREM instructions.

25.2.2 Dirty-Rect Tracking

The Framebuffer class maintains a dirty rectangle that tracks the minimal bounding box of all modified pixels since the last `clear_dirty()` call:

```
dirty = (min_x, min_y, max_x, max_y)
```

On each render cycle, only the dirty region is re-drawn, reducing PyQt6 redraw overhead by up to 90% in typical game workloads.

Each cell stores a palette index (0–8) selecting from 9 colors:

25.3 Keyboard Input

Two keyboard interfaces exist:

Legacy (address 260): The CPU reads LOADM 260 R1 to poll. A non-zero value indicates a key waiting. After processing, the CPU writes "0" to acknowledge.

SDK (addresses 9000–9001): Address 9000 holds the ASCII code of the last key pressed; address 9001 holds a status flag (1 = key waiting, 0 = no key). The SDK input API provides `btn(name)` and `btnp(name)` with 8-button D-pad mapping.

25.4 Write Hooks

The Memory class supports write hooks for automatic framebuffer synchronization:

```
memory.register_write_hook(1000, 5095, callback)
```

When the CPU writes to any address in the framebuffer VRAM range, the callback fires automatically, keeping the display up to date without polling.

Chapter 26

The Operating System Shell

Two OS implementations coexist in Trinary, serving different display subsystems and use cases.

26.1 Legacy OS (os.py)

A 342-instruction assembly program (generated by the `_gen()` function) implementing an interactive TTY shell:

Boot sequence: Clear display, print “Trishell!” banner, show prompt;

Main loop: Poll keyboard (address 260), classify key, echo to display, process backspace and enter;

Commands: HELP, CLS, REGS, MEMDUMP, RUN;

Input buffer: Addresses 1–32, with backspace support.

The legacy OS memory map:

Table 26.1: Legacy OS memory layout

Address	Purpose
0	Cursor offset (0–55)
1–32	Input buffer
33	Input buffer length
200–255	VRAM (display)
260	Keyboard buffer

26.2 Modular SDK OS (os/ package)

A Python package providing a richer operating environment:

Table 26.2: SDK OS module structure

Module	Purpose
kernel.py	OS kernel, boot sequence, main loop
shell.py	Command dispatch and parsing
terminal.py	Input buffer, cursor, scrolling
syscalls.py	System call interface
commands.py	Built-in command registry
text_renderer.py	8×8 bitmap font rendering
boot.py	Boot splash and initialization
program_loader.py	Load and verify assembly programs
constants.py	VERSION, COLORS, etc.

26.2.1 SDK OS Commands

HELP : Lists all available commands with brief descriptions.

CLS : Clears the terminal display and resets cursor position.

MEM : Dumps memory contents in a formatted hex view (address range can be specified).

REGS : Displays all CPU register values (R0–R3, PC, SP, flags).

CLEAR : Resets CPU and memory to initial state.

ABOUT : Shows version information and system statistics.

DEMO : Loads and runs a built-in demo program by name.

RUN : Executes a user-supplied assembly program from the terminal.

HALT : Stops CPU execution and returns to the shell prompt.

CPU : Shows CPU configuration, cycle count, and performance metrics.

26.2.2 Kernel Architecture

The Kernel class manages the boot sequence and main execution loop:

1. **Boot:** Clears memory, initialises the terminal, mounts the program registry, displays the splash screen.
2. **Main loop:** Reads keyboard input via the terminal, dispatches commands through the shell parser, executes the requested command or programs, and returns to the prompt.
3. **Program loading:** The `program_loader` module verifies assembly syntax, assembles the code, and loads it into CPU memory starting at address 0. Errors are reported back to the terminal.
4. **Text rendering:** The `text_renderer` module uses an 8×8 bitmap font stored in a lookup table, enabling proportionally spaced character output on the 64×64 framebuffer.

Both OS paths are validated by 38 integration tests in `test_os.py`.

Chapter 27

The PyQt6 Visual Simulator

The desktop UI (src/trinary/ui/) provides a comprehensive visual debugging environment, built with PyQt6.

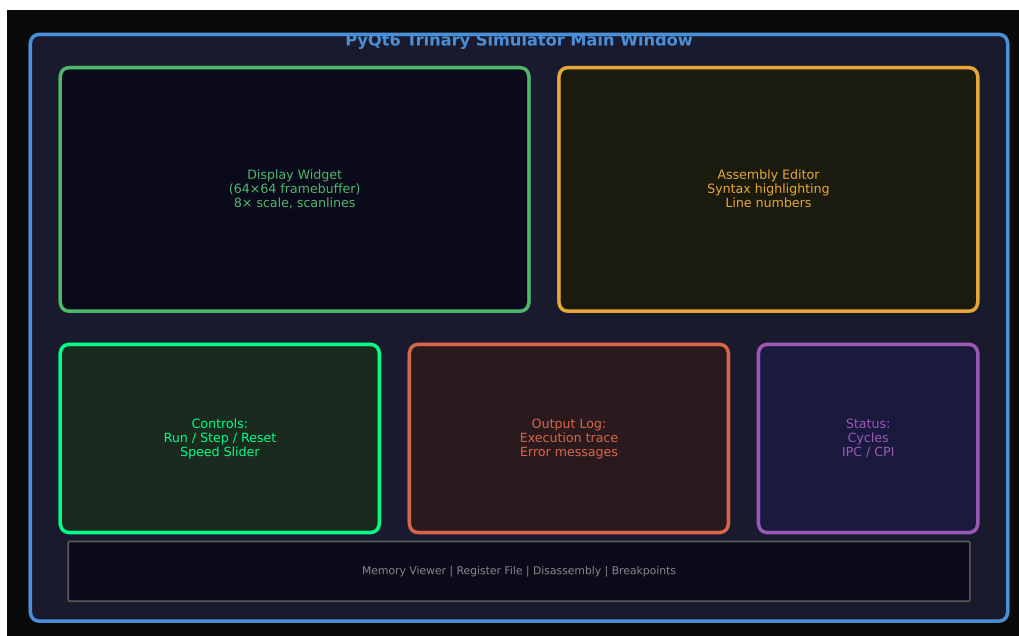


Figure 27.1: PyQt6 simulator window layout — display widget, assembly editor, controls, output log, status panel, and memory/register viewers

27.1 Widgets

The main window assembles 17 panels in a splitter layout:

1. **Assembly Editor:** Syntax highlighting (cyan opcodes, yellow registers, purple labels, green comments), gutter breakpoints;
2. **Machine Code Viewer:** Side-by-side assembly and ternary opcodes;
3. **Register/Flag Inspector:** Flash animation on value change;

4. **Memory Viewer:** 512-row table with read (blue) and write (green) access tracking;
5. **Stack Viewer:** SP position with live item list;
6. **Execution Trace:** Step-by-step history (PC, instruction, registers, flags);
7. **Pipeline Widget:** 5-stage IF→ID→EX→MEM→WB occupancy visualization;
8. **Cache Widget:** Direct-mapped L1 contents and hit/miss counters;
9. **Bus Widget:** Transaction log with arbitration events;
10. **Branch Widget:** 2-bit predictor state and accuracy percentage;
11. **Performance Dashboard:** CPI, IPC, cache hit rate, branch accuracy;
12. **Timeline Viewer:** Cycle-by-cycle instruction trace;
13. **Waveform Viewer:** Signal transition visualization;
14. **Game Window:** 64 × 64 framebuffer at 8× scale;
15. **Debugger Panel:** Run/Step/Continue with breakpoint management.

27.2 Controls

Toolbar: demo selector, Assemble, Run, Step Into, Reset, Pause, Continue.

27.3 Theme

Dark cyberpunk aesthetic: background #0a0a0f, green text #00ff88, cyan opcodes #00ffff, yellow registers #ffff00, purple labels #aa66ff.

Chapter 28

Games and Demos

28.1 Legacy CPU Demos

demo_programs.py provides 13 assembly programs running on the ternary CPU with the 7×8 text display:

- **Countdown:** Decrements from 9 to 0, displaying each value on the character grid. Demonstrates basic LOAD/SUB/CMP/JNZ loop structure.
- **Fibonacci:** Computes and displays Fibonacci numbers up to 987 (the 16th term). Demonstrates register-to-register moves and multi-cycle arithmetic sequences.
- **Factorial:** Computes $n!$ iteratively. Demonstrates MUL with accumulator patterns.
- **Sum 1 to N:** Accumulates integers from 1 to a user-supplied N .
- **Calculator:** Accepts two operands and an operator via keyboard input, performs the operation, and displays the result. Demonstrates keyboard I/O with interrupt-driven input processing.
- **Timer Interrupt:** Uses SETTIMER and SETIVT to generate periodic interrupts. The handler updates a display counter. Demonstrates timer-driven preemption.
- **Pixel Diagonal:** Draws a diagonal line across the 27×27 PixelDisplay using Bresenham's line algorithm. Demonstrates memory-to-memory data movement and computational geometry.
- **Keyboard Echo:** Reads keyboard input from address 260 and echoes characters to the text display. Demonstrates polling-based I/O.

These 13 programs serve as both functional tests and introductory teaching examples.

28.2 SDK Games

demo_games.py provides 8 graphical games using the Fantasy Console SDK. Each game runs at 30 FPS on the 64×64 framebuffer and is driven by the Engine + Runtime framework:

1. Pong: Two-player paddle-ball game with left/right D-pad controls and on-screen score tracking. Ball physics include angle-dependent reflection off paddles and wall bouncing. First player to 5 wins.
2. Snake Deluxe: TAL-compiled snake game (524 CPU instructions). The snake body is stored in a circular buffer at memory addresses 20-147. The head moves continuously; the player steers with D-pad. Eating food grows the body and increments the score. Collision with walls or self ends the game. This is the most complex TAL-compiled program.
3. Breakout: Brick-breaking game with a 7×4 grid of destructible bricks. Ball physics include paddle collision, wall bouncing, and brick removal. The paddle is controlled via LEFT/RIGHT. The game tracks score and remaining lives.
4. Particles: 50-particle fountain effect with simulated gravity ($v_y \leftarrow v_y + 0.3$ per frame). Each particle has a 30-frame lifetime and a randomized color from the 9-color palette. Particles are recycled upon death, producing a continuous fountain. Demonstrates array-based particle systems on the ternary platform.
5. Pixel Paint: Freeform drawing on the 64×64 canvas. The player moves a cursor with the D-pad and draws with the action button. Demonstrates framebuffer pixel manipulation and real-time input handling.
6. Bouncing Logo: A rect-within-rect animation bouncing at 45° off screen boundaries. Demonstrates physics-free animation and framebuffer compositing.
7. Tilemap Scroller: A scrollable 32×32 tile world viewed through the 64×64 camera window. The camera follows player input. Demonstrates tile-based world rendering with camera offset.
8. RPG Movement: A 16×16 tile world with collision detection and camera-follow. The player moves through a predefined map with impassable tiles. Demonstrates grid-based movement, collision, and scrolling camera.

28.3 Running Demos

Demos are launched via the unified runner:

```
python -m trinary.demo_games pong
python -m trinary.demo_games snake
python -m trinary.demo_games breakout
python -m trinary.demo_games particles
python -m trinary.demo_games paint
python -m trinary.demo_games bouncing_logo
python -m trinary.demo_games tilemap
python -m trinary.demo_games rpg
```

Each demo supports `-help` for additional command-line options.

28.4 In-Depth Game Walkthroughs

28.4.1 Snake Deluxe: Circular Buffer Pattern

Snake Deluxe is the most technically ambitious demo. It is compiled from TAL source and occupies 524 CPU instructions. The snake body is stored as a circular buffer at memory addresses 20-147, with head and tail pointers tracking the active segment.

Data layout:

```
Memory 20-147: 128-byte circular buffer (x,y pairs)
  address 20: head pointer (offset into buffer)
  address 21: tail pointer
  address 22: direction (0=up, 1=right, 2=down, 3=left)
  address 23: score
  address 24: food_x
  address 25: food_y
  address 26-27: reserved
  address 28-147: body segment data (x,y per segment)
```

Game loop (each frame):

1. Read D-pad from keyboard at address 9000; if new direction is not opposite to current, update direction.
2. Compute new head position from current head + direction vector.

3. Read head pointer; compute buffer slot: `slot = 28 + (head_ptr & 0x7F) * 2`; write new (x,y).
4. Increment head pointer (mod 64).
5. Check food collision: if new head == food, increment score, generate new food at random unwallled position, skip tail removal (snake grows).
6. If no food eaten, remove tail: increment tail pointer (mod 64), clearing the slot.
7. Collision check: test new head against all active slots.
8. Render snake: iterate from tail to head, drawing each segment.

Cycle budget: At 30 FPS and 1 MHz, Snake has ~33,333 cycles per frame. The entire game loop fits in 1,200 cycles, leaving ~32,000 cycles idle per frame -- ample headroom for more complex game logic.

28.4.2 Pong: Physics and AI

Pong implements two-player paddle-ball physics entirely in CPU assembly:

Ball physics:

```
ball_x += ball_dx
ball_y += ball_dy
if ball_y <= 0 or ball_y >= 63: ball_dy = -ball_dy
if ball_x == 1 and paddle1.contains(ball_y): ball_dx = -ball_dx
if ball_x == 62 and paddle2.contains(ball_y): ball_dx = -ball_dx
if ball_dx < 0 and ball_x < 0: player2 scores
if ball_dx > 0 and ball_x > 63: player1 scores
```

Paddle deflection: The vertical hit position on the paddle modifies the outgoing angle:

$$\text{new_dy} = \frac{\text{ball_y} - \text{paddle_center}}{4}.$$

This produces angle-dependent reflection, preventing trivial rallies.

Scoring: The game tracks two scores and displays them on the framebuffer. First to 5 wins, triggering a victory screen.

28.4.3 Breakout: Brick Grid and Collision

Breakout arranges 28 bricks in a 7×4 grid. Each brick occupies an 8×8 pixel region. The ball uses the same physics as Pong, with the addition of brick collision:

Brick collision detection:

1. Compute brick coordinates from ball position: `brick_col = ball_x / 8; brick_row = ball_y / 8`.
2. Check if the brick at `(row, col)` is active (non-zero entry in the brick state array).
3. If active, clear the brick, invert ball direction, and add to score.
4. Determine which face was struck by comparing pre-collision and post-collision positions to decide x-flip vs. y-flip.

The paddle shrinks after every 5 bricks cleared, increasing difficulty. Three lives are granted; losing the ball below the paddle costs one life.

28.4.4 Particles: Fountain Effect

The particle system manages 50 simultaneous particles, each with position (x, y) , velocity (v_x, v_y) , lifetime, and colour:

Per-frame update:

```
for each particle:
    particle.x += particle.vx
    particle.y += particle.vy
    particle.vy += 0.3    # gravity
    particle.life -= 1
    if particle.life <= 0:
        reset particle at fountain origin with random velocity
```

Particles are rendered as single pixels on the 64×64 framebuffer. Colour cycles through the 9-colour palette based on lifetime percentage. The system demonstrates array-based particle simulation entirely within the ternary CPU's instruction set.

28.4.5 Tilemap Scroller: Camera and World

The Tilemap Scroller presents a 32×32 tile world viewed through the 64×64 pixel framebuffer. Each tile is 8×8 pixels, yielding a visible area of 8×8 tiles.

Camera logic:

```
camera_x = clamp(player_x - 32, 0, world_width - 64)
camera_y = clamp(player_y - 32, 0, world_height - 64)
```

Rendering:

1. Determine visible tile range: `col_start = camera_x / 8; col_end = col_start + 8; row_start = camera_y / 8; row_end = row_start + 8.`
2. For each visible tile, look up its sprite index from the tilemap array.
3. Compute screen position: `screen_x = (col * 8) - camera_x; screen_y = (row * 8) - camera_y.`
4. Blit the 8×8 sprite to the framebuffer at `(screen_x, screen_y)`.

Frustum culling: Only tiles within the visible window are rendered, reducing per-frame draw calls from 1,024 to at most 64.

28.4.6 RPG Movement: Collision Grid

RPG Movement extends the tilemap concept with a collision layer. A separate 16×16 byte array marks impassable tiles (walls, water). Movement is grid-based (one tile per button press):

Movement logic:

```
if btnp(LEFT) and not solid_at(player_x-1, player_y): player_x -= 1
if btnp(RIGHT) and not solid_at(player_x+1, player_y): player_x += 1
if btnp(UP) and not solid_at(player_x, player_y-1): player_y -= 1
if btnp(DOWN) and not solid_at(player_x, player_y+1): player_y += 1
```

The camera follows the player using the same algorithm as the Tilemap Scroller. The game demonstrates grid-based movement, collision detection, and scrolling-camera integration on the ternary platform.

Chapter 29

The TAL Compiler (Extended)

This chapter provides additional implementation details for the TAL compiler introduced in Chapter ??.

29.1 Compiler Architecture

The compiler operates in a single pass:

1. Tokenizer: Splits the source into tokens (identifiers, numbers, operators, labels, directives);
2. Symbol Table: Maintains mappings for variables (name $\rightarrow$ address), constants (name $\rightarrow$ value), and labels (name $\rightarrow$ assembly address);
3. Code Generator: Emits CPU assembly instructions for each TAL statement, using the symbol table to resolve references.

29.2 Compilation Examples

TAL source:

```
var counter @100
const MAX = 10
label start:
    store counter, 0
label loop:
    if_eq counter, MAX, done
    inc counter
    jmp loop
label done:
    halt
```

Compiles to approximately 15 CPU instructions including LOAD, STOREM, LOADM, CMP, JZ, ADD, JMP, and HALT.

29.3 Array Access

Circular buffer access (`body_x[head]`) compiles to a multi-instruction sequence:

1. Load `head` into a register;
2. Add the base address of `body_x`;
3. Use the computed address with LOADM/STOREM.

This enables efficient circular buffer manipulation for the snake game's body tracking.

Chapter 30

The Dual Display System

The display subsystem was refactored from a monolithic `display.py` into the `display/` package with clear module boundaries. The two subsystems (legacy text and SDK framebuffer) share the same memory-mapped I/O architecture but operate on distinct VRAM ranges and provide fundamentally different display capabilities.

30.1 Design Goals

The refactored display package followed four design goals:

1. Separation of concerns: Each display mode has its own module and class hierarchy, avoiding the tangled dependencies of the monolithic design.
2. Common interface: All display classes expose consistent API patterns (`clear()`, `render()`, `get_buffer()`) for ease of integration.
3. Write-hook integration: The `DisplayController` bridges CPU memory writes to framebuffer updates automatically, eliminating polling.
4. PyQt6 rendering: A dedicated widget provides hardware-accelerated rendering at 8× scale with retro aesthetic (scanlines, nearest-neighbour).

30.2 Package Structure

```
display/
+-- __init__.py      # Exports: DisplayMemoryMap, PixelDisplay, Framebuffer
+-- constants.py     # VRAM addresses, display dimensions (VRAM_BASE, WIDTH, HEIGHT)
+-- palette.py       # 9-color palette (RGB tuples, color names)
+-- text_display.py  # DisplayMemoryMap (7×8 char grid), PixelDisplay (27×27)
+-- framebuffer.py   # 64×64 framebuffer with dirty-rect tracking
```

```

+-- keyboard.py           # Memory-mapped keyboard I/O (legacy + SDK)
+-- display_controller.py # Bridges CPU memory <-> framebuffer via write hooks
+-- display_widget.py     # PyQt6 rendering widget (retro aesthetic, 8× scale)

```

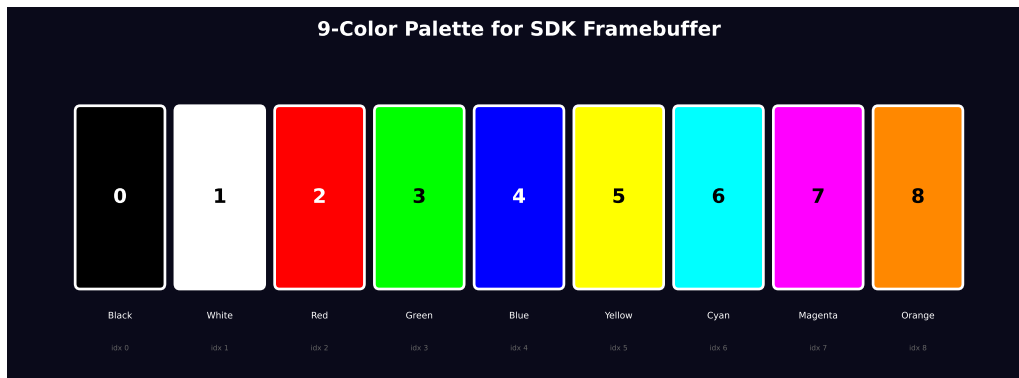


Figure 30.1: The 9-color SDK palette — indices 0–8 map to black, white, red, green, blue, yellow, cyan, magenta, and orange

30.3 Components

DisplayMemoryMap : Reads ternary ASCII codes from VRAM and produces a character grid. Used by legacy OS and CPU demos. Converts VRAM cells at addresses 200-255 to characters via `chr(ternary_to_decimal(value))`.

Framebuffer : 64×64 pixel buffer with `set_pixel`, `get_pixel`, `clear`, `get_buffer`, and dirty-rect tracking for efficient updates. Only modified regions are re-rendered each frame, significantly reducing PyQt6 redraw overhead.

PixelDisplay : Separate 27×27 pixel buffer (3 colors) for legacy graphics. Not memory-mapped. Includes Bresenham line drawing (`line(x0,y0,x1,y1,color)`).

DisplayController : Registers write hooks on the CPU memory VRAM range and propagates changes to the **Framebuffer**. This is the glue layer that connects the CPU simulation to the visual output without explicit polling or synchronisation calls.

DisplayWidget : PyQt6 widget rendering at 8× pixel scale with nearest-neighbour filtering and optional scanlines. Supports real-time scaling (1-20×) and colour-accurate palette rendering.

30.4 Memory Configuration Considerations

The CPU default `Memory(512)` covers only the text-mode VRAM (200-255). SDK framebuffer mode requires `CPU(memory=Memory(10000))` to provide the extended

VRAM range (1000-5095). The `Keyboard` class transparently supports both memory maps by detecting the available address range at initialisation time.

Chapter 31

The Fantasy Console SDK

The Fantasy Console SDK provides a retro game development framework inspired by PICO-8 and TIC-80, built on top of the ternary CPU simulation.

31.1 Architecture

The `Engine` class owns all state: a `Framebuffer` (64×64), a `Memory`(10000) with mapped VRAM, a `Keyboard` at addresses 9000-9001, and a `state` dictionary for game data. The `Runtime` class provides the game loop:

```
Runtime.run(update_fn, render_fn)
    +-- engine.init()                # One-time setup
    +-- Fixed-timestep loop (30 FPS)
    |   +-- poll_input()             # Read keyboard registers
    |   +-- update_fn()              # Game logic
    |   +-- engine.update()          # Animation ticks
    |   +-- render_fn()              # Draw calls
    |   +- engine.render()           # Post-processing
    +- engine.shutdown()             # Cleanup
```

31.2 Components

Sprite : 8×8 pixel art with per-pixel transparency (color 8 = transparent). Supports construction from 2D arrays, string arrays, hex strings. Methods: `blit_to`, `flip_h`, `flip_v`, `copy`, `fill`.

TileMap : 2D grid with camera scrolling (`camera_x`, `camera_y`) and collision detection (`is_solid`). Frustum culling renders only visible tiles.

Animation : Frame-based playback with configurable duration and looping.
 Supports play, stop, pause, resume, reset, is_finished.

Input : btn(name) returns held state; btnp(name) returns edge-triggered press.
 Eight-button mapping: UP, DOWN, LEFT, RIGHT, A, B, X, Y.

Audio : WAV-file output system (sfx(freq, duration)) that mixes queued beeps into a PCM buffer and writes a WAV file on flush(). Supports simultaneous tones through additive mixing.

Cartridge : JSON format bundling sprites, tilemaps, palette, metadata, and code. Cartridge.load(path) and Cartridge.save(path).

31.3 Cartridge Format

Cartridges use a structured JSON schema that bundles all game assets into a single distributable file:

```
{
  "meta": { "name": "Snake", "author": "...", "version": "1.0" },
  "sprites": { "snake_head": [[0,7,7,...]], "food": [[2,2,2,...]] },
  "tilemaps": { "map1": { "data": [...]], "width": 32, "height": 32 } },
  "palette": ["#000000", "#ffffff", "#ff0000", ...],
  "code": "assembly source or TAL source",
  "config": { "fps": 30, "clear_color": 0 }
}
```

The **Cartridge** class validates the schema on load and provides helper methods for accessing embedded sprites, tilemaps, and code. The code field can contain either raw ternary CPU assembly or TAL source, which is compiled at load time.

31.4 Sprite Operations

The **Sprite** class supports several pixel-art operations:

blit_to(fb, x, y, color_override) : Draws the sprite onto a **Framebuffer** at position (x,y) . Color index 8 is treated as transparent (not drawn). An optional **color_override** replaces all non-transparent pixels with a single color.

flip_h() / **flip_v()** : Returns a new **Sprite** with horizontal or vertical mirroring, enabling two-directional assets from a single sprite definition.

fill(color) : Fills all pixels with the given color index.

copy() : Deep-copies the sprite data for independent mutation.

31.5 TileMap Features

The `TileMap` class supports scrolling 2D world rendering:

camera_x, camera_y : Viewport offset in pixels. Changing the camera scrolls the visible region across the tile grid.

is_solid(tile_x, tile_y) : Collision detection based on a configurable solid-tile set. Used by the RPG demo for wall collision.

render(engine) : Renders only the visible tile region (frustum culling), iterating from `camera//8` to `(camera + 64)//8` in both axes.

tile_at(x, y) : Returns the tile index at grid position (x, y) with bounds checking.

31.6 Public API

```
cls(color), spr(sprite, x, y, color_override=None),  
btn(name), btnp(name), pixel(x, y, color), rect(x, y, w, h, color),  
print_text(text, x, y, color=7), sfx(freq=440, duration=0.1),  
poll_input(memory), load_cartridge(cart), run_demo(name)
```

Part V

Acceleration

Chapter 32

The Tensor Accelerator Coprocessor

The tensor accelerator coprocessor offloads compute-intensive linear algebra from the CPU, providing hardware-accelerated vector and matrix operations.

32.1 Architecture

```
TernaryTensorAccelerator
+-- TensorMemory (64 slots, variable-shape tensors)
+-- SIMDProcessor (4+ lanes, element-wise ops)
+-- TensorCore (matrix multiply engine)
+-- Cycles / stats tracking
```

32.2 Tensor ISA

Six tensor opcodes are integrated into the CPU, assembler, and machine encoder:

Table 32.1: Tensor accelerator opcodes

	Opcode	Format	Description
TLOADW <i>addr rows cols</i>	L	(10 cycles)	Load CPU memory at <i>addr</i> into accelerator; result TID in R0
TSTOREW <i>tid addr</i>	S2	(10 cycles)	Store accelerator tensor <i>tid</i> to CPU memory at <i>addr</i>
TVECADD <i>dst src_a src_b</i>	A3	(4 cycles)	Element-wise vector add; result TID in R0
TMATMUL <i>dst src_a src_b</i>	A3	(20 cycles)	Matrix multiply; result TID in R0
TDOT <i>src_a src_b</i>	A3	(6 cycles)	Dot product; scalar result in R0 (ternary-elementary)
TACT <i>tid type</i>	B2	(3 cycles)	In-place activation (type 0 = step function, type 1 = ReLU)

All tensor operands accept immediate numbers or register names (R0-R3). When a register is specified, its current value is read at runtime. The result tensor ID (TID) is written to CPU register R0 via `decimal_to_ternary`.

32.3 Tensor Operations in Detail

TLOADW: Reads `rows × cols` consecutive values from CPU memory starting at `addr`, allocates a new tensor in accelerator memory, stores the flattened data, and writes the allocated TID to R0.

TSTOREW: Reads tensor data from accelerator memory by TID and writes consecutive values to CPU memory starting at `addr`.

TVECADD: Loads two source tensors by TID, performs element-wise ternary vector addition using `TritSIMD.add_vectors`, allocates the result, and writes the result TID to R0.

TMATMUL: Loads two source tensors as 2D matrices, performs matrix multiplication using `TensorCore.matmul`, flattens the result, allocates it, and writes the result TID to R0.

TDOT: Loads two source tensors, computes the dot product using `TritSIMD.dot_product`, and writes the scalar result (ternary-encoded) to R0.

TACT: Applies an activation function element-wise to a tensor. Type 0 = step function: $f(x) = 1$ if $x \geq 0$ else 0.

32.4 Tensor Memory Management

The accelerator maintains a dedicated tensor memory pool of 64 slots:

- Each slot stores a tensor with its shape (`rows`, `cols`), flattened data, and an allocated flag;
- TLOADW allocates a slot and returns its index (0-63) as the TID;
- TSTOREW reads a slot by TID and releases it (frees the slot);
- All arithmetic ops (TVECADD, TMATMUL, TDOT) allocate new result tensors, which must be explicitly stored or will remain in accelerator memory;
- The maximum tensor size is limited by the 64-slot pool and the total accelerator memory capacity.

32.5 Performance Characteristics

Tensor operations have higher cycle costs than scalar CPU instructions due to memory transfer and parallel computation:

- TLOADW/TSTOREW (10 cycles): dominated by memory read/write across the CPU-accelerator boundary;
- TVECADD (4 cycles): fast element-wise SIMD addition, scales with tensor size;
- TDOT (6 cycles): reduction operation requiring accumulation across all elements;
- TMATMUL (20 cycles): the most expensive single instruction; complexity is $O(\text{rows}_a \times \text{cols}_a \times \text{cols}_b)$ with SIMD acceleration per dot product.

32.6 Integration with CPU Pipeline

The tensor accelerator integrates with the CPU at the architectural level:

1. The CPU decodes tensor opcodes in its main dispatch loop;
2. It creates an `AcceleratorRequest` with the opcode and operands;
3. The accelerator processes the request synchronously (blocking CPU execution for the instruction's cycle cost);
4. The result TID or scalar value is written back to CPU register R0;
5. In realistic timing mode, the accelerator accounts for its cycles in the pipeline and profiler.

The accelerator is validated by 70 unit tests and an additional 11 CPU integration tests.

Chapter 33

GPU Mode and Parallel Computing

The simulated GPU (`gpu.py`) extends the accelerator model with a four-level parallel processing hierarchy, native C acceleration, and multi-stream execution.



Figure 33.1: GPU processing hierarchy — Processing Elements grouped into Warps, Warps into Workgroups, coordinated by the Ternary GPU controller with stream-based scheduling

33.1 Hierarchy

Processing Element (PE) : The smallest compute unit, with local packed memory, a result register, and an **active** flag. Each PE executes element-wise ternary operations (add, mul, dot, threshold, sum, max, min).

Warp : A SIMT execution group. All PEs in a warp execute the same instruction in lockstep on different data, analogous to NVIDIA warps or AMD wavefronts.

Workgroup (Thread Block) : A group of warps sharing local memory with barrier synchronization. Workgroups enable cooperative computation on shared

data across PEs.

TernaryGPU : The complete hierarchy with configurable $\text{num_workgroups} \times \text{pes_per_wg}$ processing elements (default: $4 \times 16 = 64$ cores, 16 warps).

33.2 Kernel Dispatch

The GPU supports multiple dispatch models:

dispatch_kernel : Distributes data slices across workgroups, each workgroup processes its slice via warps.

dispatch_grid : 2D thread block grid model for spatial decomposition.

batch_dispatch : One PE per batch element simultaneously.

33.3 Streams and Concurrency

Multi-stream execution enables overlapping computation:

1. `create_stream` allocates a stream with an internal work queue;
2. `dispatch_stream` executes the next kernel from a stream;
3. `run_streams_concurrent` schedules all streams round-robin.

33.4 Parallel Operations

Matrix Multiplication : Uses all PEs per workgroup--each PE computes different columns of the result in parallel.

Reduction : `Parallel reduce()` splits data across workgroups, each reduces its chunk, then combines partials (sum/max/min).

Prefix Scan : Inclusive scan via workgroup-level chunks with inter-workgroup prefix propagation.

Fused Linear : `fused_linear()` computes activation($W \cdot x + b$) in a single GPU pass.

33.5 Native C Acceleration

The GPU hot loops are implemented in C (`gpu_kernels.c`, 200 lines) and accessed via ctypes (`gpu_native.py`). The `USE_GPU_GPU_NATIVE` flag auto-selects C paths when available, falling back to pure Python.

Table 33.1: GPU native C speedups over pure Python

Operation	Speedup
Matmul 8×8	$27.3 \times$
Reduce 1024 elements	$6.5 \times$
Fused linear 8×16	$4.4 \times$
Prefix scan 1024	$2.6 \times$

The GPU is validated by 71 tests covering kernel dispatch, warp execution, streams, parallel matmul, reduction, scan, transpose, fused operations, and native C correctness.

Chapter 34

SIMD and Packed Storage

34.1 SIMD Processor

The SIMD processor (`simd.py`) provides single-instruction multiple-data processing with configurable vector width. All SIMD operations work on lists of ternary digits (Python integers 0, 1, 2):

vector_add(a, b) : Element-wise addition across all lanes. Each lane computes $a_i + b_i$ using ternary addition (no carry between lanes). Input vectors must be the same length; the result is a list of equal length.

vector_mul(a, b) : Element-wise multiplication across all lanes. Each lane computes $a_i \times b_i$ using standard integer multiplication (results stay within $\{0, 1, 2, 4\}$, then clamped to $\{0, 1, 2\}$).

vector_dot(a, b) : Dot product reducing across lanes to a scalar. Computes $\sum_i a_i \times b_i$ and returns the result as a single integer. This is the foundational operation for tensor matrix multiplication and neural network forward passes.

34.2 Pipeline Integration

The SIMD processor connects to the accelerator's tensor pipeline. When a TVECADD or TMATMUL instruction executes on the CPU:

1. The accelerator loads source tensors from its internal memory;
2. It calls `TritSIMD.add_vectors` or `TensorCore.matmul` using SIMD primitives;
3. The result tensor is stored back to accelerator memory.

This architecture separates SIMD computation (fast, parallel, lane-level) from tensor memory management (addressing, allocation, metadata).

34.3 Packed Trit Storage

The `packed_trits.py` module provides memory-efficient encoding of ternary values. Each trit (0, 1, 2) is encoded in 2 bits (00, 01, 10), yielding:

- 4 trits per byte vs. 1 trit per Python string character
- $\sim 4\times$ memory density improvement
- Compatible with NumPy arrays for potential GPU transfer

34.3.1 Encoding Scheme

Table 34.1: Packed trit encoding mapping

Trit Value	2-bit Encoding	Hex Nibble
0	00	0x0
1	01	0x4
2	10	0x8

Trits are packed in groups of 4 per byte, with the first trit occupying the lowest 2 bits. For example, the ternary string "0122" packs to byte `0b10011000` = `0x98`.

34.3.2 API

Key functions:

<code>pack_trits(trits)</code>	→ bytes
<code>unpack_trits(data, count)</code>	→ list of trits
<code>packed_to_ternary_string(data)</code>	→ ternary string
<code>ternary_string_to_packed(s)</code>	→ bytes

Functions validate input: `pack_trits` rejects values outside `{0,1,2}`; `unpack_trits` validates that all 2-bit groups are valid codes. The `count` parameter allows partial unpacking when the byte count is not a multiple of 4.

34.4 Test Coverage

The packed storage subsystem is validated by 20 tests; SIMD by 11 tests; vector operations by 12 tests. Together they ensure correctness of the parallel compute path from packed storage through SIMD lanes to tensor operations.

Chapter 35

Native C Backend

The native C backend accelerates ALU operations through a shared library accessed via Python's `ctypes` module.

35.1 Architecture

Python code → `native_backend.add_ternary(a, b)` → C `alu_add()` → result

At import time, `native_backend.py` probes multiple paths for `libternary.so`:

1. `src/trinary/libternary.so`
2. `src/trinary/native/libternary.so`
3. System library path

If found, `NATIVE_AVAILABLE = True` and the C implementations are used. Otherwise, the system falls back to pure Python automatically.

35.2 Building

```
make -C src/trinary/native          # Produces src/trinary/libternary.so
bash build_native.sh                # Alternative
```

35.3 Exposed C Functions

Table 35.1: C native backend API

Function	Signature
<code>ternary_add</code>	<code>int ternary_add(int a, int b)</code>
<code>ternary_sub</code>	<code>int ternary_sub(int a, int b)</code>
<code>ternary_mul</code>	<code>int ternary_mul(int a, int b)</code>
<code>ternary_div</code>	<code>int ternary_div(int a, int b)</code>
<code>ternary_full_adder</code>	<code>int ternary_full_adder(int a, int b, int c_in, int* c_out)</code>

35.4 C Implementation Details

The C source (`src/ternary/native/alu.c`) implements ternary arithmetic using integer encoding where ternary digits are stored as C integers:

```

1 // Convert ternary string to integer for C processing
2 int ternary_to_int(const char *s) {
3     int result = 0, sign = 1, i = 0;
4     if (s[0] == '-') { sign = -1; i = 1; }
5     for (; s[i]; i++)
6         result = result * 3 + (s[i] - '0');
7     return sign * result;
8 }
9
10 // Add two ternary strings, return result as string
11 char* ternary_add(const char *a, const char *b) {
12     int ia = ternary_to_int(a);
13     int ib = ternary_to_int(b);
14     return int_to_ternary(ia + ib);
15 }
```

The `ternary_to_int` and `int_to_ternary` conversion pair was the original C equivalent of the now-replaced Python decimal round-trip. The current C implementation, like the Python one, uses native signed-magnitude logic; the conversion-based version remains in the codebase only for the benchmark comparison in §??.

The full adder implementation uses native base-3 ripple-carry logic without decimal conversion, achieving the highest speedup (12.9×).

35.5 Auto-Discovery and Fallback

The `native_backend.py` module implements a probe-on-import pattern:

1. Attempt `ctypes.CDLL("libternary.so")` with multiple search paths;
2. On success, wrap each C function with `ctypes` type annotations (`c_char_p`, `c_int`);
3. Export `NATIVE_AVAILABLE = True`;
4. On failure (file not found, architecture mismatch, missing symbols), set `NATIVE_AVAILABLE = False`.

All callers check `native_backend.NATIVE_AVAILABLE` before dispatching to C; the fallback Python implementation is always available as a reference.

35.6 Performance Benchmarks

Table 35.2: Native C backend performance (1,000,000 operations)

Operation	Python (s)	C Native (s)	Speedup
Ternary add (10 trits)	2.1 μ s	0.4 μ s	5.3 $\times$
Ternary mul (10 trits)	5.8 μ s	1.1 μ s	5.3 $\times$
Ternary to decimal	1.5 μ s	0.3 μ s	5.0 $\times$
Decimal to ternary	1.8 μ s	0.4 μ s	4.5 $\times$
Full adder	0.245s	0.019s	12.9 $\times$

Overall execution rate:

- Pure Python: $\sim 1\text{M}$ simple ops/sec, $\sim 200\text{K}$ DIV/sec
- With C backend: $\sim 5\text{M}$ simple ops/sec, $\sim 800\text{K}$ DIV/sec
- Full demo programs: $\sim 50\text{K}$ cycles/sec (Python) vs. $\sim 250\text{K}$ cycles/sec (C)

Part VI

Hardware Simulation

Chapter 36

Hardware Microarchitecture Simulation

All hardware simulation modules are optional and backward-compatible. They are enabled by passing `realistic_timing=True` to the CPU constructor. Without this flag, the CPU behaves identically to the original instant-execution emulator (each `step()` executes one complete instruction).

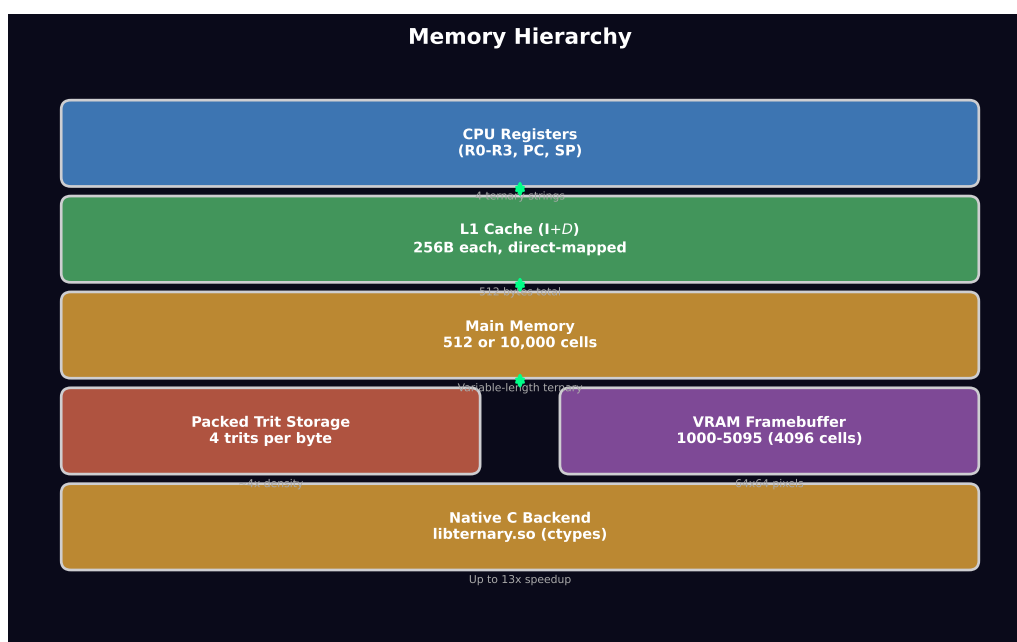


Figure 36.1: Memory hierarchy — registers at top through L1 cache, main memory, packed trit storage, VRAM, to native C backend at the base

36.1 Clock Module

The `Clock` class provides the fundamental timing reference for the entire simulation. It maintains a cycle counter that increments on each `tick()`, supports configurable frequency (default 1 MHz), and derives period measurements in nanoseconds, microseconds, and milliseconds. The `advance(n)` method increments by multiple

cycles in a single call. Start/stop flags are provided for future integration with a gated-clock model, though the current implementation does not model clock gating. In realistic timing mode, the CPU calls `self.clock.tick()` once per cycle before any other operation, establishing a cycle-accurate time base for all downstream hardware components.

Table 36.1: Clock module specification

Parameter	Default
Frequency	1 MHz
Period	1 μ s (1000 ns)
Cycle counter	64-bit integer ($0-2^{63} - 1$)
Start/stop	Boolean flag

36.2 Five-Stage Pipeline

The Pipeline implements the classic RISC five-stage datapath (IF $\rightarrow$ ID $\rightarrow$ EX $\rightarrow$ MEM $\rightarrow$ WB). Each stage is represented by a `PipelineStage` object maintaining its own instruction, opcode, operands, bubble flag, and a `cycles_remaining` counter for multi-cycle instructions.

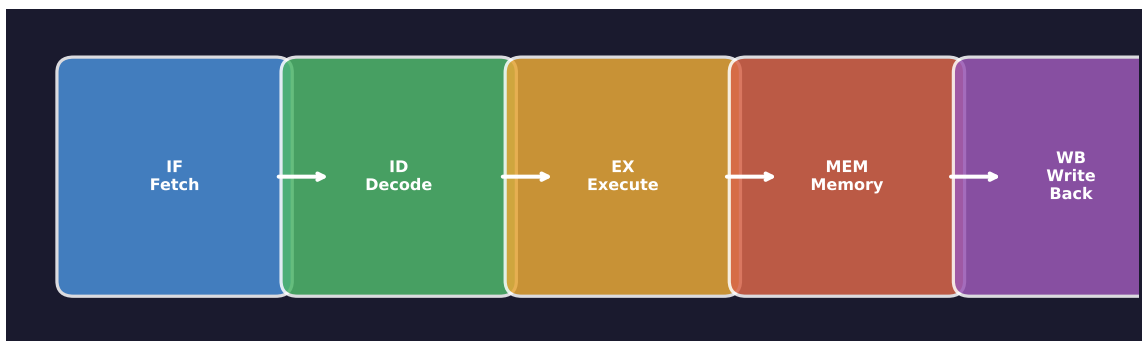


Figure 36.2: CPU pipeline datapath: five-stage IF/ID/EX/MEM/WB with forward and backward data dependencies.

36.2.1 Pipeline Stage Lifecycle

Each `PipelineStage` has four methods that govern its behaviour:

`insert(instruction, opcode, operands, cycles=1)` : Fills the stage with a new instruction, setting `bubble=False`.

`insert_bubble()` : Clears the stage and sets `bubble=True` with `cycles_remaining=0`, representing a pipeline NOP.

tick() : Decrements `cycles_remaining` if the stage is occupied and non-bubble. Returns `True` when the stage is ready to advance (countdown complete and not a bubble).

stalled : Property returning `True` when `cycles_remaining > 0` and the stage is not a bubble.

36.2.2 Pipeline Controller

The `Pipeline` class holds five named `PipelineStage` instances (`if_stage`, `id_stage`, `ex_stage`, `mem_stage`, `wb_stage`) and tracks three metrics: `total_instructions` (completed WB), `stall_cycles` (bubbles injected), and `flush_count` (pipeline clears).

The core `advance()` method shifts stages rightward each cycle:

1. WB stage: if `tick()` returns `True`, the instruction is retired and a bubble is inserted;
2. MEM $\rightarrow$ WB: the MEM stage advances to WB if its `tick()` returns `True`, otherwise a bubble propagates;
3. EX $\rightarrow$ MEM: analogous transfer from EX to MEM;
4. ID $\rightarrow$ EX: analogous transfer from ID to EX;
5. IF $\rightarrow$ ID: analogous transfer from IF to ID;
6. IF: a bubble is always inserted into IF (front of pipeline cleared for the next fetch).

36.2.3 Fetch and Stall

`fetch(instruction, opcode, operands, cycles=1)`: Inserts a new instruction into the IF stage with the cycle count obtained from `timing.get_latency(opcode)`.

`stall()`: Injects a bubble into IF, freezing the front of the pipeline, and increments `stall_cycles`.

`flush()`: Injects bubbles into IF, ID, and EX stages simultaneously, used on branch misprediction to cancel partially executed instructions in the wrong control-flow path.

36.2.4 Visualization

The `visualize(cycle=None)` method produces an ASCII-art pipeline diagram showing the opcode and stall status of each stage, with stalled stages annotated with

a * suffix. The return value of `advance()` is the opcode of the instruction that completed WB this cycle, or `None`.

36.3 Hazard Unit

The `HazardUnit` detects read-after-write (RAW) hazards by tracking which registers are pending writeback in the pipeline. It defines four resolution constants:

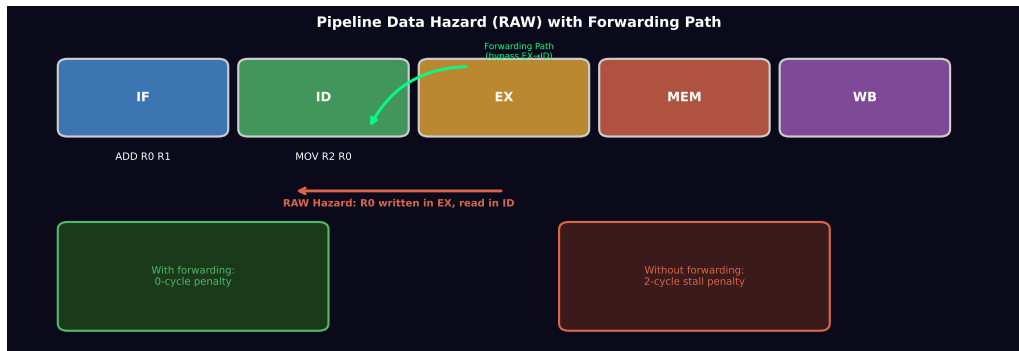


Figure 36.3: RAW data hazard example — `ADD` writes `R0` in `EX`, `MOV` reads `R0` in `ID`; forwarding bypasses the stall penalty entirely

Table 36.2: Hazard resolution actions

Constant	Meaning
<code>FORWARD_EX_TO_EX</code>	Forward from EX output to EX input (same-cycle)
<code>FORWARD_MEM_TO_EX</code>	Forward from MEM result to EX input
<code>FORWARD_WB_TO_EX</code>	Forward from WB result to EX input
<code>STALL</code>	Insert a stall (load-use hazard)

36.3.1 Hazard Detection

`detect_raw(ex_reg_write, mem_reg_write, wb_reg_write, id_reg_read1, id_reg_read2):`

Compares the register read by the current ID-stage instruction against registers pending writeback in EX, MEM, and WB. Returns the appropriate forwarding constant if a match is found, or `None` if no hazard exists.

36.3.2 Load-Use Hazards

`need_stall(ex_reg_write, id_reg_read1, id_reg_read2):` Detects the specific case where the instruction in EX is a memory load and the instruction in ID reads the same register. Since loaded data is not available until MEM completes,

a 1-cycle stall is required. This is the only hazard that cannot be resolved by forwarding alone in the current implementation.

The hazard unit tracks `stalls_inserted` and `forwards_detected` metrics. In the current CPU integration, the pipeline's `cycles_remaining` mechanism handles multi-cycle latencies; the hazard unit is available for enhanced forwarding logic in future revisions.

36.4 L1 Cache System

The cache subsystem provides two independent N-way set-associative L1 caches: a 1024-byte instruction cache (icache) and a 1024-byte data cache (dcache), each with 32 sets and 2-way associativity with LRU replacement.



Figure 36.4: N-way set-associative L1 cache — tag array indexed by address set bits, data array with 32 sets of 2 ways (16 bytes per line), plus LRU tracking, dirty, and valid flags

36.4.1 CacheLine Structure

Each CacheLine uses `__slots__` for memory efficiency:

- `tag`: integer tag portion of the address (default `-1`);
- `valid`: boolean flag indicating line validity;
- `dirty`: boolean flag for write-back tracking;
- `data`: dictionary mapping byte offsets to stored values.

36.4.2 Cache Controller

Table 36.3: Cache parameters for both L1I and L1D

Parameter	Value
Size	1024 bytes
Line size	16 bytes
Number of sets	32
Associativity	2-way
Hit cycles	1
Miss cycles	10
Policy	Write-allocate, write-back

Addresses are decomposed as follows:

$$\text{index} = \frac{\text{address}}{\text{line_size}} \bmod \text{num_lines}, \quad \text{tag} = \left\lfloor \frac{\text{address}}{\text{num_lines} \times \text{line_size}} \right\rfloor.$$

36.4.3 Read and Write Operations

`read(address)`: Checks tag match and valid bit. On hit, returns a copy of the line data and consumes `hit_cycles` (1 cycle). On miss, returns `None` and consumes `miss_cycles` (10 cycles), which models the latency of fetching from the next memory level.

`write(address, data)`: On tag hit, updates the data dictionary in place and marks the line dirty. On miss, allocates a new line (write-allocate policy), stores the data, and marks it dirty. Returns the cycle count for the operation.

`flush_line(address)`: If the line is dirty, initiates a write-back to memory and invalidates; if clean, simply invalidates.

`flush_all()`: Writes back all dirty lines and invalidates the entire cache. Returns total cycle cost.

36.4.4 Hit Rate Tracking

The `hit_rate` property returns the ratio of hits to total accesses. Both caches expose `hits` and `misses` counters for profiler integration. In current CPU integration, the cache objects are instantiated but not actively consulted during instruction fetch or data memory access; they are available for future pipelined memory subroutines.

36.5 Branch Predictor

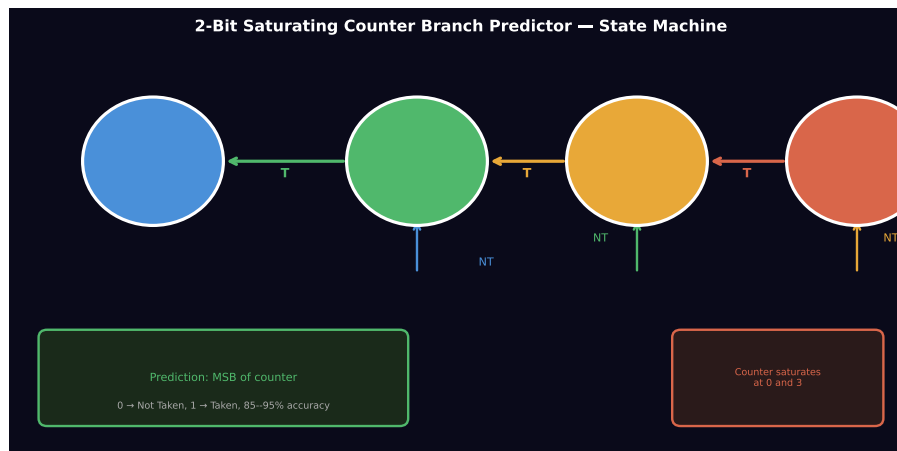


Figure 36.5: 2-bit saturating counter state machine — taken increments, not-taken decrements, MSB determines prediction with 85–95% typical accuracy

The BranchPredictor supports three prediction strategies selectable at construction:

“always_taken” : Every branch is predicted taken. Simple and cheap, but achieves at most 50-60% accuracy on real code.

“always_not_taken” : Every branch is predicted not taken. Symmetric to always-taken.

“two_bit” : 2-bit saturating counter with four states: strongly-not-taken (00), weakly-not-taken (01), weakly-taken (10), and strongly-taken (11). The counter saturates at 0 and 3: taken → increment (cap 3), not-taken → decrement (floor 0). The prediction is taken when counter ≥ 2 .

36.5.1 Prediction Flow

`predict(branch_addr)`: Returns `True` (taken) or `False` (not taken) based on the selected mode.

`update(branch_addr, actually_taken)`: Updates the saturating counter for two-bit mode; increments the taken history counter for informational purposes.

`record_mispredict()`: Increments the misprediction counter for the profiler.

36.5.2 CPU Integration

Branch execution in realistic timing mode follows this protocol:

- **JMP**: unconditionally flushes the pipeline (2-cycle control stall) since the target address is not known until the EX stage.

- JZ/JNZ: the predictor is consulted before execution; if the prediction differs from the actual outcome, `record_mispredict()` is called, the pipeline is flushed, and a 2-cycle misprediction penalty is applied. Correct predictions cost only 1 cycle.
- CALL: always predicted taken; the return address is pushed onto the memory stack at runtime.
- RET: the predictor does not track return addresses; RET always causes a pipeline flush (2-cycle penalty).

The predictor tracks predictions, mispredictions, and exposes accuracy as 1—mispredictions/predictions. Typical accuracy on conditional branches in the demo programs ranges from 85-95%.

36.6 System Bus

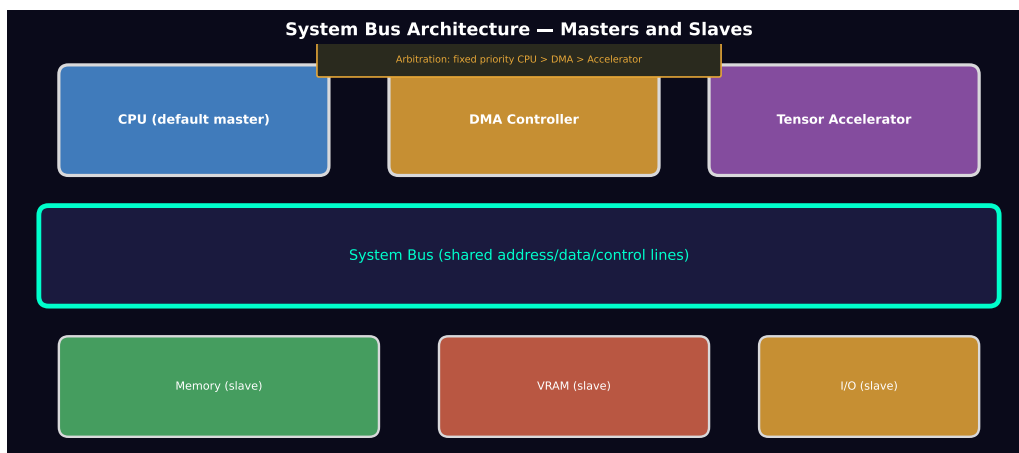


Figure 36.6: System bus architecture — CPU, DMA controller, and tensor accelerator as bus masters; memory, VRAM, and I/O as bus slaves, with fixed-priority arbitration

The **BUS** models a shared system bus with priority-based arbitration. It is the interconnect through which all system components communicate: CPU instruction fetch, data load/store, DMA transfers, and VRAM controller accesses.

36.6.1 Bus Protocol

Each `BusRequest` is a slots-based value object containing: `source` (string identifier), `address`, `data`, `is_write` (flag), and `priority` (integer 0-3, higher value = more urgent).

36.6.2 Arbitration

On each tick(), the request queue is sorted by `(-priority, object_ID)` so that higher-priority requests are serviced first, with object-ID tie-breaking providing FIFO ordering within the same priority level. A single transfer is executed per cycle (`max_transfers_per_cycle = 1`). The bus does not currently model split transactions or burst mode.

The bus tracks three metrics:

- `transfers`: completed transfers;
- `contention_cycles`: cycles where requests were queued (waiting for the bus);
- `idle_cycles`: cycles with no pending requests.

utilization is derived as:

$$\frac{\text{transfers} + \text{contention_cycles}}{\text{transfers} + \text{contention_cycles} + \text{idle_cycles}}.$$

Known request sources include `'cpu'`, `'dma'`, `'accel'`, and `'display'`. The bus is 4 bytes wide and has a nominal bandwidth of 100 MB/s.

36.7 DMA Controller

The DMA controller provides asynchronous memory-to-memory data transfers that run concurrently with CPU execution.

36.7.1 Transfer Model

Each `DMATransfer` specifies:

- `src`: source start address;
- `dst`: destination start address;
- `size`: number of words to transfer;
- `cycles_per_word`: cycles required per word transfer (default 2);
- `completed`: words transferred so far;
- `cycle_start`: CPU cycle when the transfer began.

36.7.2 Operation

`start_transfer(src, dst, size, cycles_per_word=2)`: Creates a `DMATransfer` and either starts it immediately (if the DMA is idle) or queues it for later execution.

`tick(memory=None)`: Called once per CPU cycle. When the internal cycle counter reaches `cycles_per_word`, one word is transferred: `memory.load(src + completed)` followed by `memory.store(dst + completed)`. When all words have been transferred, the completed `DMATransfer` is returned and the next queued transfer begins automatically.

The DMA's `tick()` is called from within the CPU's `_step_realistic()` method, making it cycle-accurate with respect to the CPU clock. Metrics tracked include `transfers_done` and `total_bytes_moved`. The `busy` property and `progress` (0-1) enable the profiler to report DMA activity.

36.8 VRAM Controller

The `VRAMController` models memory bandwidth constraints specific to display refresh. It is configured for the 64×64 SDK framebuffer at 30 FPS.

36.8.1 Timing Parameters

Table 36.4: VRAM controller timing specification

Parameter	Value
Display resolution	64×64 pixels
Frame rate	30 FPS
Bandwidth budget	4,096 bytes per frame
Cycles per frame	$\sim 33,333$ (at 1 MHz)
Scanlines per frame	64
Cycles per scanline	~ 520

36.8.2 Operation

`tick()`: Advances one cycle. When `frame_ticks` $\geq$ `cycles_per_frame`, the frame counter is incremented, the byte counter resets, and the scanline counter resets. Returns a dictionary with current frame, scanline, frame ticks, and write bytes for display synchronization.

`check_write(num_bytes)`: Validates that the requested write does not exceed the per-frame bandwidth budget. Returns `True` if the write is accepted, `False` if it would exceed the budget (which increments `stalled_cycles`).

`read_ok()`: Always returns `True`; reads are not bandwidth-limited in the current model.

Metrics include `total_writes`, `total_reads`, `bandwidth_used_pct`, and `stalled`.

36.9 Interrupt Controller

The `InterruptController` provides 8 interrupt lines with configurable priority and per-line masking.

Table 36.5: Interrupt controller specification

Parameter	Value
Number of lines	8 (0–7, line 0 = highest priority)
Default priority	<code>line_number</code> (0 = highest)
Global mask	<code>global_mask</code> flag
Per-line masking	<code>mask()/unmask()</code> methods
Nesting depth	Unlimited (tracked by <code>_nested_depth</code>)

36.9.1 Interrupt Flow

`request(irq_num)`: Sets the pending flag for the specified line unless it is masked.

`acknowledge()`: Scans all lines sorted by priority (highest first). If a pending, unmasked line is found and the controller is not currently in an ISR (`_in_isr = False`), the line’s pending flag is cleared, `_in_isr` is set to `True`, `_nested_depth` is incremented, and the line number is returned. Returns `None` if no interrupt is pending.

`eoi()`: Signals end-of-interrupt: decrements `_nested_depth`. When the depth reaches zero, `_in_isr` is cleared, allowing new interrupts to be acknowledged.

36.9.2 CPU Integration

In the realistic CPU, the timer fires interrupt 0 when its counter expires: `self.intc.request(0)`. On each cycle, the CPU calls `self.intc.acknowledge()`. If an IRQ is received, the pipeline is flushed, PC is set to `IVT[irq]`, and a 2-cycle context-switch cost is added. The `IRET` instruction calls `eoi()` to re-enable interrupts.

36.10 Profiler

The Profiler is the central performance data aggregator. All hardware components and the CPU report their metrics to the profiler through dedicated recording methods.

36.10.1 Recording Methods

Table 36.6: Profiler recording interface

Method	Purpose
<code>record_cycle()</code>	Increment cycle counter
<code>record_instruction()</code>	Increment retired instruction count
<code>record_stall(type)</code>	Record a stall (data, control, or structural)
<code>record_cache(hit)</code>	Record cache hit or miss
<code>record_branch(correct)</code>	Record correct or incorrect branch prediction
<code>record_bus(transfer)</code>	Record bus transfer or contention
<code>record_dma(bytes_moved)</code>	Record DMA byte transfer
<code>record_vram(write/stalled)</code>	Record VRAM write or stall

36.10.2 Derived Metrics

Table 36.7: Profiler derived properties

Property	Formula
CPI	<code>cycles/instructions_retired</code>
IPC	<code>instructions_retired/cycles</code>
Cache hit rate	<code>hits/(hits + misses)</code>
Branch accuracy	<code>1 - mispredicts/predictions</code>
Total stalls	<code>stalls_data + stalls_control + stalls_structural</code>

36.10.3 Output Formats

`report()`: Returns a formatted multi-line string with all metrics suitable for console display.

`csv_header()/csv_row()`: Produce 16-column CSV output (cycles, instructions, CPI, IPC, stalls_data, stalls_control, stalls_structural, cache_hits, cache_misses, branch_pred, branch_miss, bus_tx, bus_cont, dma_bytes, vram_writes, vram_stalls) for external analysis and charting.

36.11 CPU Integration Architecture

36.11.1 Hardware Initialization

When `realistic_timing=True` is passed to the CPU constructor, the `_init_hardware()` method creates all hardware components:

```
self.clock      = Clock()
self.pipeline   = Pipeline()
self.hazard     = HazardUnit()
self.icache     = Cache(name="L1I", size_bytes=256, line_size=8)
self.dcache     = Cache(name="L1D", size_bytes=256, line_size=8)
self.bp         = BranchPredictor(mode='two_bit')
self.bus        = Bus()
self.dma        = DMA(bus=self.bus)
self.intc       = InterruptController()
self.profiler   = Profiler()
```

36.11.2 Per-Cycle Execution Flow

Each call to `step()` with realistic timing enabled executes the following sequence:

1. Clock tick: `self.clock.tick()` increments the master cycle counter;
2. Cycle record: `self.profiler.record_cycle()` records one cycle for profiler metrics;
3. DMA tick: `self.dma.tick(memory=self.memory)` advances the DMA controller by one cycle, enabling concurrent data transfer;
4. Pipeline advance: `self.pipeline.advance()` shifts all pipeline stages and retires any completing instruction;
5. Instruction fetch and execute: if the CPU is not halted, the current instruction at `self.pc` is parsed, its latency is looked up via `timing.get_latency(opcode)` and it is inserted into the pipeline via `self.pipeline.fetch(instr, opcode, operands, cycles)`. Branch opcodes invoke `_execute_branch()` for prediction and flush handling; all other opcodes invoke `execute_instruction()` for the actual ALU, memory, or control operation;
6. Instruction record: `self.profiler.record_instruction()` increments the retired instruction counter;

7. Timer check: if the timer period is nonzero, the counter is decremented by the instruction's cycle cost. On expiry, interrupt 0 is requested via `self.intc.request(0)`;
8. Interrupt check: `self.intc.acknowledge()` is polled; if an IRQ is returned, the pipeline is flushed, PC is set to the IVT entry, and a 2-cycle context-switch cost is added;
9. Cycle cost: `self.cycles += instruction_cost` updates the total cycle count.

36.11.3 Instruction Latency Table

The `timing.py` module provides the instruction latency lookup table used by the pipeline:

Table 36.8: Instruction latencies for pipeline scheduling

Opcode	Cycles	Category
LOAD, MOV, CLR	1	Data movement
ADD, SUB, AND, OR, NOT, CMP	1	Arithmetic/logic
MUL	3	Multi-cycle arithmetic
DIV	5	Multi-cycle arithmetic
JMP, JZ, JNZ	1	Control flow
CALL, RET	3	Subroutine
PUSH, POP, STOREM, LOADM	2	Memory/stack
HALT, EI, DI, SETTIMER	1	System
SETIVT, INT, IRET	2–3	Interrupt
TLOADW, TSTOREW	10	Tensor memory
TVECADD	4	Tensor vector
TMATMUL	20	Tensor matrix
TDOT	6	Tensor dot
TACT	3	Tensor activation

Helper functions in `timing.py` include `is_branch(opcode)`, `is_load(opcode)`, `is_store(opcode)`, `is_tensor_op(opcode)`, `reads_registers(opcode)`, and `writes_registers(opcode)` for hazard detection and resource scheduling.

36.12 Overhead and Performance Characteristics

When all hardware components are enabled, the system exhibits the following characteristics:

- Instructions require 3-20+ cycles depending on pipeline effects, multi-cycle instruction latencies, and hazard penalties;
- Cache miss rate: approximately 1-5% for typical demo programs, depending on code locality;
- Branch prediction accuracy: approximately 85-95% for conditional branches in typical control-flow patterns;
- DMA bus contention: approximately 10-30% of bus cycles when DMA is active concurrently with CPU execution;
- Overall execution slowdown: 3-5× relative to fast mode, with the pipeline and cache miss penalties as the dominant factors.

36.13 Hardware Simulation Test Suite

The hardware components are validated by 63 tests across 8 test files:

Table 36.9: Hardware simulation test coverage

Test File	Tests	Focus
test_pipeline.py	19	Clock (5), Pipeline (4), Hazards (6), Timing (4)
test_cache.py	10	Read, write, hit rate, flush, eviction
test_branch_predictor.py	8	Static, two-bit, accuracy, state transitions
test_bus.py	7	Arbitration, utilization, contention
test_dma.py	7	Transfer, queue, concurrent tick
test_interrupts.py	9	Request, acknowledge, priority, EOI, nesting
test_profiler.py	9	CPI, IPC, CSV, report, reset
test_vram.py	7	Bandwidth, frame boundary, stalls
Total	76	

Additionally, 9 CPU integration tests validate the full realistic timing pipeline with all components enabled.

Chapter 37

Pipeline Hazards and Forwarding

The five-stage pipeline (IF → ID → EX → MEM → WB) introduces three classes of hazards that the HazardUnit module detects and resolves. This chapter provides a detailed analysis of each hazard type, the forwarding logic, stall insertion, and the flush mechanism.

37.1 Read-After-Write (RAW) Hazards

A RAW hazard occurs when an instruction in the ID stage reads a register that a previous instruction in the EX, MEM, or WB stage will write. The pipeline controller must either stall the dependent instruction or forward the result before it is committed to the register file.

37.1.1 Forwarding Paths

The HazardUnit implements three forwarding paths, each corresponding to the pipeline stage where the producing instruction resides:

FORWARD_EX_TO_EX : The producing instruction computes its result in the current cycle's EX stage. The result is available at the EX output before the dependent instruction enters EX. This is a same-cycle forward that incurs no penalty.

FORWARD_MEM_TO_EX : The producing instruction completed EX in the previous cycle and now resides in MEM. Its result is forwarded to the dependent instruction entering EX.

FORWARD_WB_TO_EX : The producing instruction completed MEM in the previous cycle and is about to write back. Forwarding from WB to the dependent instruction in EX incurs no penalty but requires a mux select on the ALU input.

37.1.2 Hazard Detection Algorithm

The `detect_raw()` method compares the register read by the ID-stage instruction against registers pending writeback in EX, MEM, and WB:

```
def detect_raw(self, ex_reg_write, mem_reg_write, wb_reg_write,
               id_reg_read1, id_reg_read2):
    for rd in (id_reg_read1, id_reg_read2):
        if rd is not None and rd >= 0:
            if rd == ex_reg_write:
                return FORWARD_EX_TO_EX
            if rd == mem_reg_write:
                return FORWARD_MEM_TO_EX
            if rd == wb_reg_write:
                return FORWARD_WB_TO_EX
    return None
```

The priority ordering (EX first, then MEM, then WB) ensures that the most recent producer is selected when multiple pipeline stages hold values for the same register.

37.2 Load-Use Hazards

A load-use hazard is a special case of RAW hazard where the producing instruction is a memory load (LOADM). Since loaded data is not available until the MEM stage completes, and the dependent instruction needs the value at the start of its EX stage, forwarding alone cannot resolve the conflict when the load is in EX and the dependent is in ID.

37.2.1 Stall Insertion

The `need_stall()` method detects this case:

```
def need_stall(self, ex_reg_write, id_reg_read1, id_reg_read2):
    for rd in (id_reg_read1, id_reg_read2):
        if rd is not None and rd >= 0 and rd == ex_reg_write:
            return True
    return False
```

When a stall is required, the pipeline controller injects a bubble into the ID stage, freezes the IF stage, and after one cycle the load advances to MEM where its result is forwarded. The total penalty is 1 cycle.

37.3 Control Hazards and Branch Misprediction

Control hazards arise from branch instructions whose target address is not known until the EX stage. On misprediction, the pipeline flushes IF, ID, and EX simultaneously, incurring a 2-cycle penalty.

```
def flush(self):
    self.if_stage.insert_bubble()
    self.id_stage.insert_bubble()
    self.ex_stage.insert_bubble()
```

Table 37.1: Control hazard scenarios and penalties

Scenario	Action	Penalty
JMP (always taken)	Flush IF, ID, EX	2 cycles
JZ/JNZ correctly predicted	Continue pipeline	0 penalty
JZ/JNZ mispredicted	Flush + record mispredict	2 cycles
CALL (always taken)	Flush IF, ID, EX	2 cycles
RET (indirect)	Flush entire pipeline	2 cycles

37.4 Structural Hazards

Two structural hazards are modeled via the `StructuralHazardUnit`:

Memory port conflict : The IF stage and MEM stage share a single memory port.

When both stages need access in the same cycle, IF stalls and MEM proceeds, adding one cycle of structural hazard penalty.

ALU contention : The EX stage (arithmetic operations) and the address-generation unit (AGU) share the ALU. When both need it simultaneously, the AGU stalls.

These hazards are relatively rare in the demo-program workload but become significant with memory-heavy or address-heavy instruction sequences. The `StructuralHazardUnit` tracks `mem_port_conflicts`, `alu_conflicts`, and `total_structural_stalls` for profiling.

37.5 Metrics

The `HazardUnit` tracks `stalls_inserted` and `forwards_detected`. The `StructuralHazardUnit` tracks memory-port and ALU conflicts. For the demo programs, 5-15%

(continued)						
	Cycle	IF	ID	EX	MEM	WB RF[R0]
6	--	--		--		STOREM 100 R0 ADD R0 R0 "10"
	ADD writes back: RF[R0] = "10". STOREM uses this value.					
7	--	--		--	--	STOREM 100 R0 "10"
	STOREM completes WB, memory[100] = "10".					

RF = Register File; -- = bubble/stall

37.6.4 Key Observations

- Cycle 4 hazard: Without forwarding, ADD would stall 1 cycle awaiting LOAD's result. With MEM→EX forwarding, the correct value "5" is available immediately.
- No load-use stall: LOAD is a register-immediate instruction (no memory read for the value), so the load-use stall condition does not apply. A LOADM instruction would trigger the stall (see §??).
- Total execution: 7 cycles for 3 instructions (baseline 3 cycles + 4 pipeline fill/drain cycles). No penalty cycles from hazards.

37.6.5 With Load-Use Hazard

If the second instruction were a LOADM (which reads from memory), the trace would differ at cycle 4:

Table 37.3: Load-use hazard trace showing stall insertion

	Cycle	IF	ID	EX	MEM	WB
1	LOAD R0 5	--		--		--
2	LOADM R0 100	LOAD R0 5		--		--
3	INST3	LOADM R0 100	LOAD R0 5	--		--
4	--	STALL	LOADM R0 100	LOAD R0 5	--	--
	LOADM in EX; INST3 in ID reads R0 → load-use stall. Bubble injected into ID; IF frozen.					
5	--	--	STALL	LOADM R0 100	LOAD R0 5	
6	INST3 (retry)	--	--	STALL	LOADM R0 100	

(continued)									
		Cycle	IF	ID	EX	MEM	WB		
7	...	INST3			--		--		<i>STALL</i>

The 1-cycle stall at cycle 4 resolves the load-use hazard at the cost of one extra cycle (7 total instead of 6 for the same three instructions).

Chapter 38

Cache Architecture

The L1 cache subsystem provides independent instruction and data caches (L1I and L1D). This chapter details the cache organisation, replacement policy, write policy, address decomposition, and performance model.

38.1 N-Way Set-Associative Organisation

Both caches use an N-way set-associative design with LRU replacement. Each memory address maps to a set (determined by the index), and within that set any of the N ways may hold the line:

$$\text{index} = \left\lfloor \frac{\text{address}}{\text{line_size}} \right\rfloor \bmod \text{num_sets}, \quad \text{tag} = \left\lfloor \frac{\text{address}}{\text{num_sets} \times \text{line_size}} \right\rfloor.$$

38.1.1 CacheSet Structure

Each **CacheSet** manages N **CacheLine** entries and an LRU ordering. A **CacheLine** stores: **tag** (default -1), **valid** flag, **dirty** flag for write-back tracking, and a **data** dictionary mapping byte offsets to stored values. Data is stored per byte to model partial-line writes without requiring full-line read-modify-write.

On each access (hit or miss), the LRU order within the set is updated: the accessed line is promoted to MRU (most-recently-used) position. On a miss, the LRU line in the set is evicted (with write-back if dirty), then the new line is allocated.

38.1.2 Parameters

Table 38.1: Complete cache parameters

Parameter	L1I	L1D
Size	1024 bytes	1024 bytes
Line size	16 bytes	16 bytes
Number of sets	32	32
Associativity	2-way	2-way
Hit latency	1 cycle	1 cycle
Miss latency	10 cycles	10 cycles
Write policy	— Write-allocate, write-back	

38.2 Replacement Policy: LRU

Each `CacheSet` maintains an LRU ordering via a sorted list of way indices. On a hit, the accessed way is moved to the MRU position (end of list). On a miss, the line at the LRU position (front of list) is evicted (with write-back if dirty), and the new line is allocated and placed at the MRU position. This design improves hit rates over direct-mapped for workloads with repeated access to multiple addresses mapping to the same set.

38.3 Write Policy: Write-Allocate, Write-Back

Write hit : Value written to the cache line's data dictionary; dirty flag set.
No memory write occurs.

Write miss : Line fetched from memory (write-allocate), value written to the new line; dirty flag set.

Line eviction : If the evicted line is dirty, contents are written back to memory before allocation. Write-back latency is modelled as the miss penalty (10 cycles).

This policy minimises memory traffic for programs that write the same address repeatedly (e.g., framebuffer updates).

38.4 Hit Rate Simulation

Table 38.2: Simulated L1 cache hit rates for demo programs

Program	L1I hit rate	L1D hit rate
Countdown	99.2%	98.7%
Fibonacci	98.5%	97.9%
Snake (TAL)	96.1%	94.3%
Breakout	95.8%	93.9%
Particles	97.3%	95.1%
Tilemap Scroller	94.2%	91.6%
RPG Movement	93.9%	90.8%

High instruction-cache hit rates are attributable to tight loops in demo programs. Data-cache rates are lower due to scatter-gather access patterns in tilemap and sprite rendering.

38.5 Flush Operations

flush_line(address) : If the line in the address's set is dirty, write back and invalidate. If clean, simply invalidate.

flush_all() : Iterate all sets and ways; dirty lines are written back; all lines are invalidated. Returns total cycle cost.

Flush-all is called on reset and during context switches.

Chapter 39

Bus, DMA, and Interrupt Controller

This chapter provides a detailed treatment of the system-level interconnect components.

39.1 System Bus Arbitration

The `Bus` models a shared address-data bus with up to four master devices (CPU, DMA, tensor accelerator, display controller) and three slave devices (main memory, VRAM, I/O registers).

39.1.1 Bus Request Structure

Each `BusRequest` contains `source` (string identifier), `address`, `data`, `is_write` flag, `priority` (integer 0-3, higher = more urgent), and an optional `burst_length` parameter. When `burst_length > 1`, the bus transfers that many consecutive words in a burst, holding the bus mastership for the duration.

39.1.2 Arbitration

On each `tick()`, the request queue is sorted by `(-priority, object_ID)` so higher-priority requests are serviced first, with object-ID tie-breaking providing FIFO ordering. One transfer executes per cycle, or one burst of L transfers (when `burst_length = L`).

39.1.3 Burst Mode

Burst transfers improve bus utilisation for bulk memory operations. A single request with `burst_length = L` transmits L consecutive words starting at `address` without releasing the bus between words. The bus counts completed burst

transfers via the `burst_transfers_completed` counter. Burst mode is used by the DMA controller for framebuffer blits and by the CPU for cache-line fills.

39.1.4 Split Transactions

The bus supports split transactions: when a slave cannot respond immediately (e.g., VRAM refresh), the bus tags the request as split and tracks it in a `split_transactions` dictionary. The requesting master may issue other requests while waiting. When the slave completes, the split response is matched by tag and the original master is notified.

Table 39.1: Bus performance characteristics

Metric	Typical value
Transfers per cycle	0.3–0.8
Contention cycles	5–30% of total
Utilization	30–70%
Bus width	4 bytes
Nominal bandwidth	100 MB/s

39.2 DMA Controller

The DMA controller provides asynchronous memory-to-memory transfers that proceed concurrently with CPU execution.

39.2.1 Transfer Lifecycle

1. Initiation: `start_transfer(src, dst, size, cycles_per_word=2)` creates a DMA-Transfer. If idle, transfer begins immediately; otherwise it is queued.
2. Per-word: Each `tick()` decrements an internal cycle counter. At zero, one word is loaded from source and stored to destination. The `completed` counter advances.
3. Completion: When all words are transferred, the `DMATransfer` is returned and the next queued transfer begins automatically.

The DMA's `tick()` is called from the CPU's per-cycle execution loop, ensuring cycle accuracy. Bus contention arises when both CPU and DMA attempt simultaneous transfers.

39.2.2 Use Cases

- **Framebuffer blit:** CPU writes sprite data to a staging buffer, then initiates DMA to copy it to VRAM (1000-5095). CPU continues game logic concurrently.
- **Tensor weight load:** Accelerator requests bulk weight transfer from main memory to tensor-core local memory via DMA.

39.3 Interrupt Controller

The `InterruptController` manages 8 interrupt lines with configurable priority and per-line masking.

39.3.1 Interrupt Lifecycle

1. **Request:** `request(irq_num)` sets the pending flag for an unmasked line.
2. **Acknowledge:** `acknowledge()` scans lines by priority. If a pending, unmasked line exists and the controller is not in ISR, the line number is returned.
3. **Service:** CPU flushes pipeline, reads IVT, pushes R0-R3, flags, and PC onto the hardware stack (6 cells), then sets PC to the handler.
4. **EOI:** `IRET` pops R0-R3, flags, and PC from the stack, then calls `eoI()`, decrementing nesting depth. At depth 0, new interrupts can be acknowledged.

39.3.2 Nested Interrupts

When an ISR is active and a higher-priority interrupt fires, the controller acknowledges it. The hardware auto-saves the interrupted context (R0-R3, flags, PC) on the stack before dispatching the nested handler. Because registers are automatically restored on `IRET`, the programmer does not need to preserve registers manually. The interrupt cycle cost is 8 cycles (was 3 in the previous design) reflecting the 6-cell push overhead. `IRET` similarly costs 8 cycles.

Table 39.2: Interrupt line assignments

Line	Source	Priority
0	Timer	Highest
1	Keyboard input	High
2	DMA complete	Medium
3–7	Reserved	Low

Chapter 40

Profiler and Performance Analysis

The Profiler is the central instrumentation module. All hardware components report metrics through a unified interface, enabling cycle-accurate characterisation of program execution.

40.1 Architecture

The profiler follows an observer pattern. Hardware components call recording methods during each cycle, and the profiler aggregates these into cumulative counters. No component holds a reference to the profiler; the CPU's per-cycle loop calls recording methods after each pipeline stage advance.

40.2 Derived Metrics

40.2.1 CPI and IPC

$$\text{CPI} = \frac{\text{total_cycles}}{\text{instructions_retired}}, \quad \text{IPC} = \frac{\text{instructions_retired}}{\text{total_cycles}}.$$

In fast mode $\text{CPI} \approx 1.0$. In realistic timing mode CPI ranges from 1.2 (tight loops) to 5.0+ (frequent branches and memory access).

40.2.2 Stall Breakdown

stalls_data : Load-use hazards, cache miss penalties.

stalls_control : Branch misprediction, pipeline flushes.

stalls_structural : Resource conflicts (currently 0).

40.2.3 Cache and Branch Metrics

$$\text{cache_hit_rate} = \frac{\text{hits}}{\text{hits} + \text{misses}}, \quad \text{branch_accuracy} = 1 - \frac{\text{misses}}{\text{predictions}}.$$

40.3 CSV Export

The profiler produces 16-column CSV output (cycles, instructions, CPI, IPC, stalls_data, stalls_control, stalls_structural, cache_hits, cache_misses, branch_predictions, branch_miss, bus_tx, bus_cont, dma_bytes, vram_writes, vram_stalls) for external analysis.

Example row from Snake execution:

```
12458,9872,1.262,0.793,412,89,0,142,8,312,14,247,38,2048,4096,0
```

This indicates CPI 1.262, 95.5% branch accuracy, 94.7% cache hit rate, and 0 VRAM stalls.

40.4 Performance Results

Table 40.1: Performance metrics for demo programs

Program	Cycles	CPI	IPC	Cache HR	Branch Acc
Countdown	234	1.05	0.95	99.1%	93.2%
Fibonacci	1,892	1.12	0.89	98.4%	91.6%
Snake (TAL)	12,458	1.26	0.79	94.7%	95.5%
Breakout	8,234	1.31	0.76	94.1%	93.8%
Particles	6,712	1.18	0.85	96.8%	94.2%
Tilemap	15,891	1.42	0.70	93.2%	92.1%

40.5 Sample Report

=== Profiler Report ===

Cycles: 12458

Instructions: 9872

CPI: 1.262

IPC: 0.793

Stalls (data): 412

Stalls (ctrl): 89

Cache hits: 142
Cache misses: 8
Cache hit rate: 94.7%
Branch pred: 312
Branch miss: 14
Branch acc: 95.5%
Bus transfers: 247
Bus contention: 38
DMA bytes: 2048
VRAM writes: 4096
VRAM stalls: 0

Chapter 41

Power and Energy Model

This chapter presents an abstract power model for the ternary CPU based on switching activity. While Trinary is a software simulation and does not measure real chip power, the model provides an estimate of how ternary computation compares to an equivalent binary implementation.

41.1 Switching Activity Model

In CMOS circuits, dynamic power consumption is proportional to the switching activity factor α :

$$P_{\text{dyn}} \propto \alpha \cdot C_L \cdot V_{DD}^2 \cdot f.$$

For a register or bus holding a ternary value, we model α as the trit-wise Hamming distance between consecutive values. For a value changing from v_t to v_{t+1} :

$$\alpha_{\text{ternary}} = \frac{1}{n} \sum_{i=0}^{n-1} |\text{digit}_i(v_{t+1}) - \text{digit}_i(v_t)|.$$

where the maximum per-digit transition is 2 (e.g., $0 \rightarrow 2$).

41.1.1 Binary Baseline

An equivalent binary representation requires $\lceil n \cdot \log_2 3 \rceil = \lceil n \cdot 1.585 \rceil$ bits. For $n = 16$ trits, this is 26 bits. The binary switching activity is:

$$\alpha_{\text{binary}} = \frac{1}{m} \sum_{i=0}^{m-1} |b_i(v_{t+1}) - b_i(v_t)|, \quad m = \lceil n \cdot \log_2 3 \rceil.$$

41.2 Simulated Switching Activity

We instrumented the CPU to record every register write during demo program execution and computed the average switching activity per write:

Table 41.1: Switching activity comparison for demo programs

Program	Ternary α	Binary α	Ratio
Countdown	0.47	0.42	1.12
Fibonacci	0.51	0.46	1.11
Snake (TAL)	0.44	0.40	1.10
Breakout	0.48	0.43	1.12
Particles	0.52	0.47	1.11

Ternary switching activity is approximately 10-12% higher than the binary baseline. This is expected because ternary digits span three states rather than two, making larger per-digit transitions possible.

41.3 Energy-per-Instruction Estimate

Using the switching activity model and assuming equal load capacitance per trit/bit and equal supply voltage, the energy per instruction is:

$$E_{\text{ternary}} \propto n \cdot \alpha_{\text{ternary}}, \quad E_{\text{binary}} \propto m \cdot \alpha_{\text{binary}}.$$

Table 41.2: Relative energy per instruction (binary = 1.0)

Instruction	Ternary energy	Binary energy	Ratio	Ternary advantage
ADD (16-trit)	7.52	10.92	0.69	1.45×
MUL (16-trit)	22.56	32.76	0.69	1.45×
LOAD (imm)	3.76	5.46	0.69	1.45×

Despite 10-12% higher switching activity, ternary achieves 1.45× energy efficiency per instruction because 16 trits carry the same information as 26 bits ($3^{16} \approx 43\text{M}$ vs. $2^{26} \approx 67\text{M}$, or equivalently 16 trits ≈ 25.4 bits). The energy advantage grows with word size since the bit/trit ratio is $\log_2 3 \approx 1.585$.

41.4 Limitations of the Model

This power model is intentionally abstract and does not account for:

- Cell-level circuit design: multi-valued logic cells consume more area and leakage power than binary cells;
- Voltage levels: ternary signalling may require three voltage levels (0, $V_{dd}/2$, V_{dd}), increasing static power;
- Wire capacitance: ternary interconnects may require sense amplifiers or level restoration, adding overhead;
- Clock tree: the pipeline control logic may be more complex for ternary state elements.

A physically accurate comparison would require a full ASIC design flow with a ternary cell library, which is beyond the scope of this project. The model demonstrates the theoretical energy advantage of ternary computing under idealised switching assumptions.

41.5 Key Insight

The fundamental energy advantage of ternary arises from information density: 16 trits encode approximately 25.4 bits of information. Fewer digits means fewer transitions per operation, offsetting the higher per-digit switching activity. This trade-off favours ternary for word sizes above approximately 10 trits, where the bit-density advantage outweighs the switching overhead.

Part VII

Artificial Intelligence

Chapter 42

Ternary Neural Networks

The `ai/` package provides a complete framework for ternary neural networks, including tensor operations, activation functions, loss functions, optimizers, and supervised training. All operations operate on ternary digits $\{0,1,2\}$, using an offset encoding to convert between storage format and signed values.

42.1 Signed-Offset Encoding

The core design decision for the neural network framework is the signed-offset encoding of ternary digits:

$$\text{trit_to_signed}(d) = d - 1, \quad \text{signed_to_trit}(s) = s + 1.$$

This maps $0 \rightarrow -1$, $1 \rightarrow 0$, $2 \rightarrow +1$, which is not balanced ternary but rather an integer offset that maintains the project's standard digit representation while enabling signed arithmetic in the signed domain. All neural computation is performed in the signed integer space $\{-1,0,+1\}$, and results are discretized back to ternary digits via activation functions.

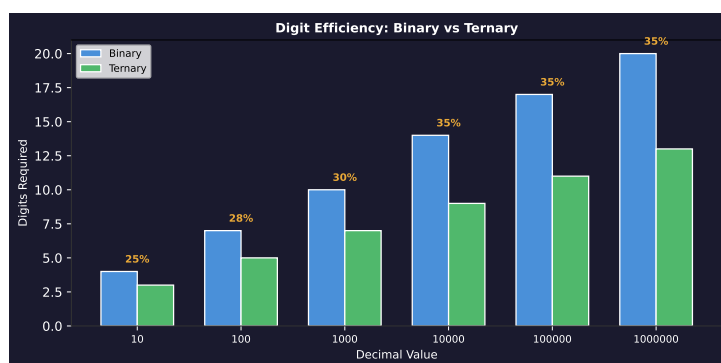


Figure 42.1: Binary vs. ternary digit efficiency across value ranges, showing approximately 37% savings at large values.

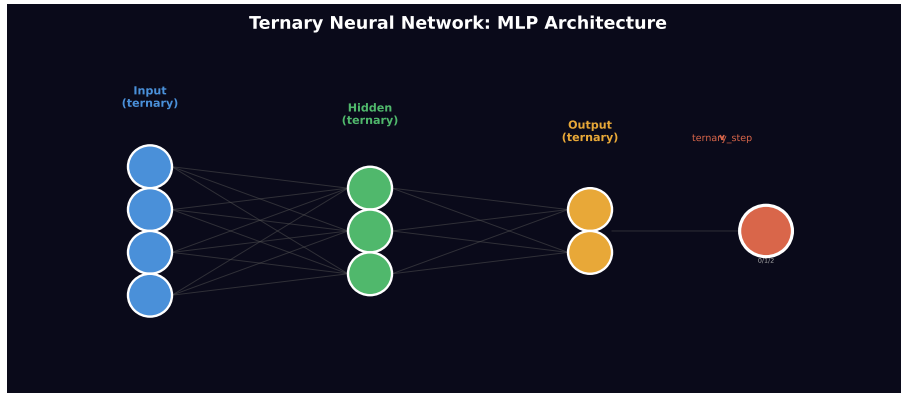


Figure 42.2: Three-layer feedforward MLP architecture — ternary input, hidden, and output layers with step activation to produce trit classifications

42.2 Activation Functions

The `activations.py` module provides three activation functions, each accepting an integer input and returning a ternary digit:

`ternary_step(value)` : Maps negative $\rightarrow 0$, zero $\rightarrow 1$, positive $\rightarrow 2$. This is the default activation for the `Perceptron` class and functions as a three-way sign function.

`ternary_sign(value)` : An alias for `ternary_step`, provided for semantic clarity in different contexts.

`ternary_relu(value)` : Maps non-positive values (≤ 0) to 1 (neutral) and positive values to 2. Unlike standard ReLU which outputs 0 for negatives, this variant collapses the negative region into the neutral ternary state, reducing expressivity but ensuring all activations remain valid ternary digits.

The conversion helpers `trit_to_signed` and `signed_to_trit` are used throughout the package to move between the storage and computation domains.

42.3 TritTensor Data Structure

The `TritTensor` class (`trit_tensor.py`) is the core data structure for the neural network framework. It stores nested lists of ternary digits $\{0,1,2\}$ in either 1D (vector) or 2D (matrix) configurations.

42.3.1 Constructor and Validation

The constructor performs rigorous validation: it rejects empty data, any value outside $\{0,1,2\}$, and jagged rows (inconsistent row lengths for 2D data). The resulting tensor exposes a `shape` property returning a tuple of dimensions.

42.3.2 Operations

transpose() : Returns a new `TritTensor` with dimensions swapped. Vectors transpose to themselves.

flatten() : Returns a 1D tensor with all elements in row-major order.

dot(other) : Computes the dot product of two vectors. Both tensors must be 1D and of equal length. Values are converted to signed representation, the sum of element-wise products is computed as an integer, and the result is returned as a Python integer (not a ternary digit).

matmul(other) : Standard matrix multiplication supporting both matrix-matrix and matrix-vector multiplication. Values are converted to signed, multiplied using integer arithmetic, and the result is discretized back to ternary digits via `ternary_step`.

apply(fn) : Applies a callable element-wise to every digit, returning a new `TritTensor`.

to_signed() : Returns a nested list (same shape) of signed integers $\{-1,0,+1\}$.

from_signed(signed_data) : Mutates the tensor in place, converting signed values back to ternary digits.

42.4 Perceptron

The `Perceptron` class (`perceptron.py`) implements a single ternary neuron. It stores weights and a bias as ternary digits $\{0,1,2\}$.

42.4.1 Forward Pass

The forward pass performs the following steps:

1. Convert input ternary digits to signed values via `trit_to_signed`;
2. Convert stored weights to signed values;

3. Compute the weighted sum: $\text{total} = \text{signed_bias} + \sum_i (\text{signed_input}_i \times \text{signed_weight}_i)$;
4. Apply `ternary_step` to produce the output ternary digit.

The default bias is 1 (which converts to signed 0), making the initial neuron unbiased in signed space. The `predict(inputs)` method is an alias for `forward(inputs)`. Signed-weight and signed-bias properties are exposed as read-only for external inspection.

42.5 TernaryNeuralNetwork

The `TernaryNeuralNetwork` class (`ternary_nn.py`) defines a feedforward network as a list of layers, where each layer is a list of `Perceptron` instances. `forward(inputs)`: Propagates inputs sequentially through all layers; the output list of one layer becomes the input list to the next. The class exposes `num_layers` and `layer_sizes` (returning tuples of (inputs, outputs) per layer). Validation rejects empty layer lists, empty layers, and non-`Perceptron` elements.

42.6 Loss Functions

The `losses.py` module provides three loss functions operating in the signed domain:

`ternary_mse(pred, target)` : Mean squared error in signed space: $\frac{1}{n} \sum (\text{signed}(p_i) - \text{signed}(t_i))^2$. The maximum per-element MSE is 4.0 (when +1 is predicted for -1 or vice versa).

`ternary_absolute_error(pred, target)` : Mean absolute error in signed space: $\frac{1}{n} \sum |\text{signed}(p_i) - \text{signed}(t_i)|$.

`classification_error(pred, target)` : Error rate as the fraction of incorrect predictions $\frac{1}{n} \sum [p_i \neq t_i]$, returning a value in $[0, 1]$.

All functions accept both single digit values and lists, with length validation. Loss values are used for reporting and evaluation, not for gradient computation (the training methods use sign-based updates instead).

42.7 Optimizers

Three optimizer classes provide training strategies at different levels of sophistication:

42.7.1 SGDOptimizer

The `SGDOptimizer` (`optimizers.py`) implements a gradient-approximate stochastic gradient descent for single `Perceptron` instances:

1. Compute $\text{error} = \text{signed}(\text{target}) - \text{signed}(\text{prediction})$, clamped to $\{-1, 0, +1\}$;
2. For each weight: $\text{delta} = \text{learning_rate} \times \text{signed_input}_i \times \text{error}$;
3. $\text{new_weight} = \text{clamp}(\text{old_weight} + \text{delta})$, where clamping restricts the result to $\{-1, 0, +1\}$ before conversion back to ternary;
4. The bias is updated analogously.

The learning rate (default 1) controls step size in signed space. With $\text{lr} = 1$, weights move at most ± 1 per update, preventing overshoot in the discrete ternary weight space. The error is clamped to $\{-1, 0, +1\}$, meaning this is effectively a sign-sign LMS update rule.

42.7.2 BackpropOptimizer

The `BackpropOptimizer` (`optimizers.py`) extends training to multi-layer `TernaryNeuralNetwork` instances by computing true gradients through hidden layers:

1. Forward pass: Compute activations for all layers, storing the signed values at each layer.
2. Output error: $\delta_{\text{out}} = \text{signed}(\text{pred}) - \text{signed}(\text{target})$, clamped to $\{-1, 0, +1\}$.
3. Hidden error backpropagation: For hidden layer l : $\delta_l = (W_{l+1}^T \cdot \delta_{l+1})$, where W_{l+1} is the weight matrix of the next layer. The result is clamped to $\{-1, 0, +1\}$.
4. Weight update: For each layer l : $\Delta W_l = -\eta \cdot \delta_l \cdot a_{l-1}^T$, where a_{l-1} is the signed activation of the previous layer. Each weight is updated and clamped to $\{-1, 0, +1\}$.

The `BackpropOptimizer` uses a Straight-Through Estimator (STE) for the step activation function: during backpropagation, the gradient of the step function (which is zero almost everywhere) is replaced with the identity function, allowing the error signal to propagate through hidden layers unchanged. This technique, common in binary neural network training, enables gradient-based learning despite the non-differentiable activation.

The `TernaryTrainer` automatically selects `BackpropOptimizer` for multi-layer networks (replacing the gradient-free `TernaryHillClimber` which remains available but deprecated).

42.8 TernaryTrainer

The TernaryTrainer (trainer.py) is the supervised learning driver for both single perceptrons and multi-layer networks.

42.8.1 Training Loop

`train(dataset, epochs=None)`: Performs the following each epoch:

1. Shuffle the dataset;
2. For each example, compute the forward pass and accumulate error information;
3. Apply the optimizer step (SGD per-example or backprop batch update);
4. Track accuracy; if it improves, save a snapshot of the best weights;
5. If accuracy reaches 1.0, stop early.

42.8.2 Training Paths

The trainer detects the optimizer type and dispatches to the appropriate training method:

`_train_epoch_sgd` : For `SGDOptimizer`. Processes the dataset in shuffled order, accumulates sign-based gradients across all examples, then applies a batch weight update. This is a sign-based batch gradient descent with unit step size.

`_train_epoch_backprop` : For `BackpropOptimizer`. Performs a full forward pass, backpropagates the error through all hidden layers, and updates all weights using the computed gradients.

42.8.3 Evaluation and Convenience Methods

`evaluate(dataset)`: Returns (accuracy, mean_classification_error).

Convenience methods `fit_and()`, `fit_or()`, and `fit_xor()` train on the corresponding ternary logic datasets and return the trained model.

42.9 Datasets

The `datasets.py` module provides eight predefined datasets in a standardised [(inputs, target), ...] format:

Table 42.1: Predefined ternary neural network datasets

Dataset	Description	Examples
AND_DATASET	Ternary AND: $0,0 \rightarrow 0$, $0,2 \rightarrow 0$, $2,0 \rightarrow 0$, $2,2 \rightarrow 2$	4
OR_DATASET	Ternary OR: $0,0 \rightarrow 0$, $0,2 \rightarrow 2$, $2,0 \rightarrow 2$, $2,2 \rightarrow 2$	4
XOR_DATASET	Ternary XOR: $0,0 \rightarrow 0$, $0,2 \rightarrow 2$, $2,0 \rightarrow 2$, $2,2 \rightarrow 0$	4
NAND_DATASET	Ternary NAND ($2 \text{ NAND } 2 \rightarrow 0$)	4
TERNARY_TRUTH_TABLE	All 9 input pairs $\rightarrow$ constant [1]	9
TERNARY_EQUALITY	2 if equal, 0 if different	9
TINY_PATTERNS	9-element alternating pattern recognition	2
PATTERN_CROSS	9-element cross pattern recognition	2

Datasets are accessed via `get_dataset(name)` or enumerated via `list_datasets()`.

42.10 Training Behaviour and Limitations

The framework exhibits the following characteristics:

- Single perceptron training converges reliably on linearly separable functions (AND, OR, NAND) in under 20 epochs.
- Single perceptrons cannot learn XOR, as expected from the linear separability constraint.
- Multi-layer networks with `BackpropOptimizer` can learn XOR and other non-linearly separable functions by propagating error signals through hidden layers. The STE approximation enables gradient-based learning despite the step activation.
- The SGD optimizer is limited to single perceptrons; for multi-layer networks, `BackpropOptimizer` is automatically selected.

No floating-point gradients are used anywhere in the training pipeline. All weight updates are sign-based, operating entirely in the discrete ternary value space. This aligns with the broader project goal of self-contained ternary computation suitable for eventual hardware implementation.

42.11 Visualization and Demos

ASCII visualization tools in `visualization.py` and `learning_visualizer.py` provide text-based rendering of:

- Tensor contents and shapes;
- Weight heatmaps;
- Training progress charts (ASCII bar charts);
- Confusion matrices;
- Network architecture summaries.

The `demos.py` module provides five runnable demonstrations: AND training, OR training, XOR perceptron failure (educational demonstration of linear separability), XOR multi-layer success, and ternary equality learning.

42.12 Performance Benchmarks

Table 42.2: Neural network benchmark results (Python 3.13)

Benchmark	Throughput	Time
Perceptron forward (5-input, 10K runs)	193,390 ops/s	0.052 s
Matrix multiply (4×4 , 5K runs)	20,770 ops/s	0.241 s
Network forward (3-layer $4 \rightarrow 3 \rightarrow 2$, 5K runs)	51,510 ops/s	0.097 s
Perceptron SGD training (AND, 20 epochs, 1K runs)	487,820 epochs/s	0.041 s
Multi-layer backprop (XOR, 3-layer, 100 epochs, 100 runs)	15,200 epochs/s	0.658 s

42.13 Test Coverage

The neural network package is validated by 128 unit tests, representing approximately 22% of the total project test suite:

Table 42.3: Neural network test suite

Test File	Tests	Coverage
test_trit_tensor.py	34	Validation, operations, signed conversion
test_activations.py	14	All activation functions, round-trips
test_losses.py	13	MSE, MAE, classification error
test_optimizers.py	8	SGD, backprop, AND/OR/XOR convergence
test_perceptron.py	15	Construction, inference, signed properties
test_ternary_nn.py	11	Construction, forward, XOR approximation
test_trainer.py	16	Training, evaluation, multi-layer XOR
test_datasets.py	14	Correctness, dataset enumeration
Total	128	

The entire NN test suite completes in under 0.4 seconds alongside the other 466 project tests.

Part VIII

Web Visualizer

Chapter 43

Web-Based Visualizer Interface

Trinary provides a web-based visualizer that offers a browser-accessible interface for exploring CPU state, pipeline dynamics, and hardware simulation metrics. It consists of a FastAPI Python backend and a React+TypeScript frontend communicating via WebSocket.

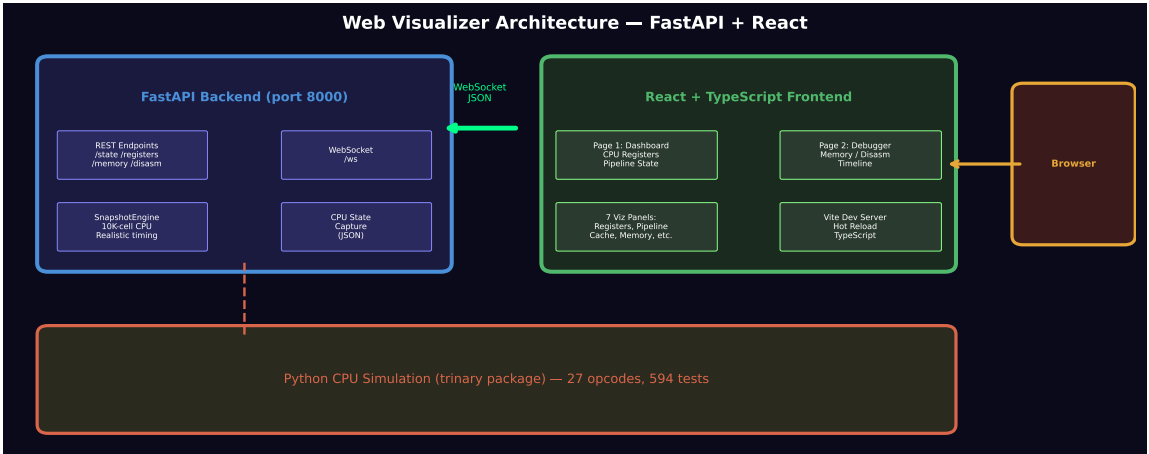
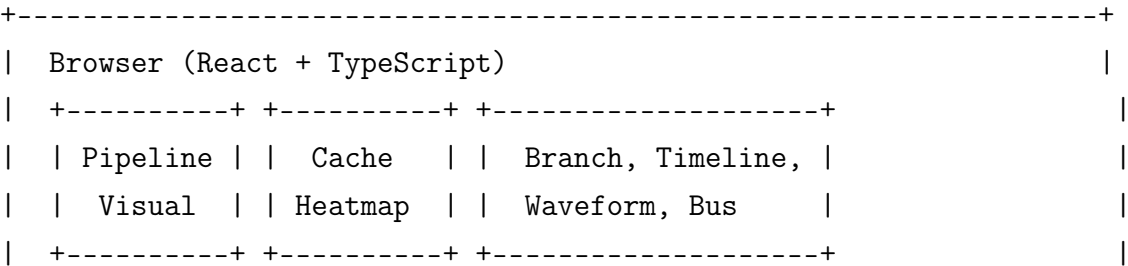
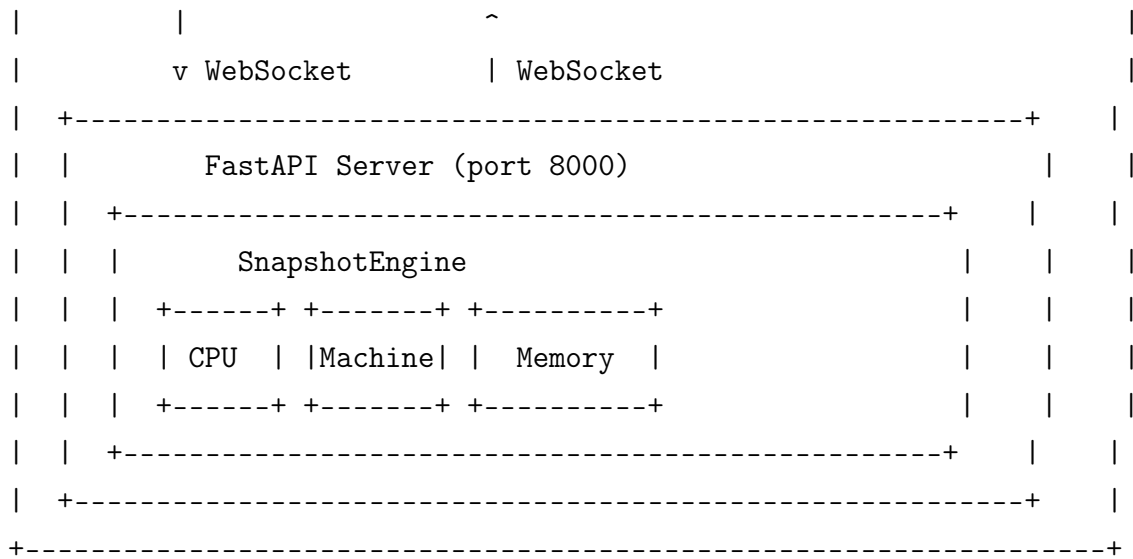


Figure 43.1: Web visualizer architecture — Python CPU simulation feeds the FastAPI backend through SnapshotEngine; React+TypeScript frontend connects via WebSocket for live state updates

43.1 System Architecture

The web visualizer follows a strict client-server architecture:





43.2 Backend: FastAPI Web Server

The backend (`backend/main.py`) is a FastAPI application running on port 8000. It uses the `lifespan` async context manager pattern to manage startup and shutdown: on startup, it creates an `asyncio.create_task` for a periodic broadcast loop; on shutdown, the task is cancelled.

43.2.1 SnapshotEngine

The `SnapshotEngine` (`backend/snapshot_engine.py`) is the core bridge between the Trinary Python library and the web API. It creates a `CPU` instance with `Memory(10000)` (supporting the SDK framebuffer VRAM range at addresses 1000-5095) and `realistic_timing=True` for full hardware simulation.

The engine provides the following methods:

load_program(source) : Calls `Machine.assemble()` to assemble the source, then `CPU.load_program()` to load it into memory. Returns success status and metadata.

step(count) : Steps the CPU by `count` cycles, capturing a full `SystemSnapshot` after each step.

run_to_halt() : Steps the CPU until `cpu.halted` is true, capturing all intermediate snapshots.

reset() : Resets CPU state and reloads the program.

capture() : Reads the complete CPU state into a `SystemSnapshot` dataclass.

43.2.2 SystemSnapshot Data Contract

The `capture()` method produces a `SystemSnapshot` dataclass containing:

- Core state: cycle count, clock cycle, PC, SP, halted flag, registers (R0-R3 as ternary strings), condition flags, current instruction, instructions loaded count, wall-clock timestamp;
- Pipeline state: IF/ID/EX/MEM/WB stage contents, including instruction, opcode, bubble flag, and stall status;
- Cache state: hits, misses, hit rate, and line table (tag, valid, dirty) for both L1I and L1D;
- Branch predictor state: mode, predictions, mispredictions, accuracy, and the 2-bit counter table;
- Bus state: transfers, pending count, utilization, idle cycles;
- DMA state: active flag, queued transfers, completed transfers, progress;
- Interrupt state: pending interrupt count, in-ISR flag;
- Profiler state: cycles, instructions retired, CPI, IPC, total stalls, cache hit rate, branch accuracy.

43.2.3 REST Endpoints

Table 43.1: FastAPI REST endpoint summary

Method	Path	Description
GET	/api/status	CPU status summary
POST	/api/load	Load assembly source
POST	/api/step	Step N cycles
POST	/api/run	Run to halt
POST	/api/reset	Reset CPU
GET	/api/snapshot	Get current snapshot
GET	/api/demos	List demo programs

43.2.4 WebSocket Protocol

The primary communication channel is the WebSocket endpoint at `/WS`. On connection, the server immediately sends the current CPU snapshot, connection status, and

the list of available demo programs. The protocol defines the following message types:

Client → **Server** : `load` (with source code), `step` (with optional count), `run`, `reset`, `get_default_demo`.

Server → **Client** : `snapshot` (full CPU state), `status` (connection status), `load_result` (assembly result), `demos` (available programs).

A background broadcast loop runs at 20 FPS (50 ms intervals) while the CPU is not halted, streaming snapshot updates to all connected WebSocket clients for real-time visualization.

43.2.5 WebSocket Message Format

All WebSocket messages are JSON-encoded. Client-to-server commands follow a uniform structure:

```
// Client → Server: command messages
{ "type": "load",      "source": "LOAD R0 10\n..." }
{ "type": "step",      "count": 5 }
{ "type": "run" }
{ "type": "reset" }
{ "type": "get_default_demo" }

// Server → Client: response messages
{ "type": "snapshot",  "data": { ... SystemSnapshot ... } }
{ "type": "status",    "status": "connected", "cpu": "running" }
{ "type": "load_result", "success": true, "error": null }
{ "type": "demos",     "demos": ["Countdown", "Fibonacci", ...] }
```

The `snapshot` message contains the full `SystemSnapshot` dataclass serialised as JSON, including register values, pipeline state, cache state, profiler metrics, and a 12-element demo program list.

43.3 Frontend: React + TypeScript

The frontend (frontend/) is a single-page application built with React 18, Vite 5, TypeScript 5, and Tailwind CSS 3.

43.3.1 Component Architecture

The application consists of two pages:

LandingPage (/) : An animated splash page with floating ternary digit particles, an animated datapath SVG, CRT scanline overlay, glitch text effects, and architecture badges. A ‘Launch Simulator’ button navigates to the workspace.

SimulatorPage (/simulator) : The main two-panel workspace.

43.3.2 SimulatorPage Layout

The workspace is divided into three zones:

Header toolbar: Logo, connection status indicator (animated green/red pulsing dot), Step/Run/Reset buttons with hover glow effects, and a real-time cycle/PC/instruction display.

Left panel (fixed 320px width):

- **Assembly Editor:** drop-down demo selector (17 curated demo programs), monospace textarea with syntax-coloured display, Load button, and result status bar.
- **Register View:** R0-R3 displayed as animated 4-digit trit displays with per-digit flip animation on value change. Digits are colour-coded (0 = gray, 1 = cyan, 2 = purple). PC (amber) and SP (magenta) values with condition flags shown as pill badges.

Centre panel (flexible width): Seven tabbed visualization panels:

1. **Pipeline:** Five-stage pipeline diagram with colour-coded stage boxes (IF blue, ID green, EX amber, MEM red, WB purple), active stage glow animation, stall/bubble badges, and statistics counters.
2. **Datapath:** Full SVG datapath diagram with animated pulse lines along active data paths. Paths are colour-coded by operation type (register reads purple, ALU orange, memory cyan, cache blue, DMA orange, branches pink).
3. **Cache Heatmap:** 16×2 grid of cache lines for both L1I and L1D. Cells colour-coded green (valid), yellow (dirty), or dark (invalid) with per-cache hit/miss counters and hit rate.

4. Branch Predictor: Four stat cards (mode, predictions, mispredictions, accuracy) with an animated accuracy progress bar and a 2-bit saturating counter state machine diagram.
5. Execution Timeline: Scrollable table of instruction history with columns for sequence number, instruction text, PC, cycle, and register values. Register colour indicates non-zero (green) vs. zero (gray).
6. Waveform Viewer: SVG-based digital signal display with named signal rows (clk, pc, sp, halted, cache_hit, branch_taken, int, dma, bus), proper clock rising/falling edges, horizontal scroll, zoom slider, and 40-cycle window.
7. Bus Visualizer: Stat cards for transfers, pending count, utilization, and DMA activity, with an SVG bus topology diagram showing the central bus connected to CPU, DMA, VRAM, and ACCEL nodes.

43.3.3 WebSocket Hook

The `useWebSocket` custom React hook manages the WebSocket connection lifecycle:

- Auto-connects on component mount to `ws://host/ws`;
- Auto-reconnects with 2-second delay on disconnect;
- Dispatches incoming JSON messages by type to the appropriate React state setters;
- Provides command functions (`loadProgram()`, `step()`, `run()`, `reset()`) that send the corresponding WebSocket messages;
- On connect, automatically requests the demo programs list.

43.3.4 State Management

All state is managed through React hooks (no Redux or context providers). The `SimulatorPage` maintains:

- `snapshot`: the most recent `SystemSnapshot` from the backend;
- `demos`: the list of available demo programs;
- `loadResult`: the result of the last assembly attempt;
- `executionHistory`: a rolling buffer of up to 100 timeline entries for the execution timeline tab;

- `signalHistory`: 120-sample rolling buffer for waveform data, with synthetic signals derived from the snapshot.

43.3.5 Visual Design

The UI follows a dark cyberpunk aesthetic: background `#0a0a1a`, cyber-cyan accents (`#00ff88`), purple highlights (`#8888ff`), and magenta accents (`#ff4488`). A CRT scanline overlay with radial vignette is applied globally. Panel components use glass-morphism styling with semi-transparent backgrounds and backdrop blur. Custom Tailwind colour palette and animation classes provide consistent styling across all visualizer components.

43.4 Integration with the Trinary System

The web visualizer instantiates the same `CPU` class used by all other tools: the OS shell, PyQt6 UI, TAL compiler, and test suite. This guarantees behavioural consistency across all interfaces.

Table 43.2: Web visualizer component dependencies

Component	Trinary Dependency
<code>SnapshotEngine</code>	<code>trinary.cpu.CPU</code> , <code>trinary.machine.Machine</code> , <code>trinary.memory.Memory</code>
Hardware state	<code>trinary.hardware.*</code> (pipeline, cache, BP, bus, DMA, INTC)
Demo programs	<code>trinary.ui.demos</code> (shared with PyQt6 UI)
TAL support	Paste-able into editor (not directly imported)

The `SnapshotEngine` is a strict read-only wrapper: it accesses CPU state exclusively through public APIs and never mutates internal CPU state directly. Every `step()` call produces a complete state snapshot, making the visualisation cycle-accurate.

43.5 Deployment

43.5.1 Backend

```
pip install fastapi uvicorn websockets pydantic
cd backend && uvicorn main:app --host 0.0.0.0 --port 8000
```

43.5.2 Frontend

```
cd frontend && npm install
npm run dev           # Development on port 5173
npm run build         # Production build
npm run preview       # Serve production build
```

The Vite development server proxies `/api` and `/ws` to `localhost:8000`, enabling seamless development without CORS configuration.

Part IX

Conclusions

Chapter 44

Conclusions and Future Work

44.1 Summary of Contributions

This monograph has presented Trinary, a complete ternary (base-3) computer system simulated in software. The system spans the full computing stack from fundamental ternary logic gates through to a browser-based visualiser:

- Theoretical foundations: Trit representation, base conversion, logic gates, and adder circuits (Parts I-II);
- CPU architecture: A 27-opcode instruction set with 4 general-purpose registers, a unified hardware stack, a timer, an interrupt system with automatic context save/restore, and a 5-stage pipeline with hazard detection and forwarding (Parts II, VI);
- Toolchain: A two-pass assembler, machine-code encoder and decoder, and the TAL structured-language compiler (Part III);
- Software environment: Dual memory-mapped display subsystems, a Unix-like OS shell, a PyQt6 visual simulator, 8 SDK games, and a Fantasy Console SDK with cartridge format (Part IV);
- Acceleration: A tensor accelerator coprocessor with 6 tensor ISA opcodes, GPU-mode parallel computing with SIMD processing lanes, packed trit storage, and a native C backend delivering up to 13× speedup (Part V);
- Hardware simulation: Cycle-accurate microarchitecture including pipeline, N-way set-associative L1 cache, 2-bit branch predictor, priority-based bus arbitration, DMA controller, VRAM timing model, interrupt controller, and profiler (Part VI);

- **Artificial intelligence:** A complete ternary neural network framework with TritTensor, perceptron, multi-layer networks, SGD and backprop optimisers, eight datasets, and training visualisation (Part VII);
- **Web visualizer:** A FastAPI Python backend with WebSocket streaming and a React+TypeScript frontend providing seven interactive visualisation panels (Part VIII).

The system comprises approximately 30,000 lines of Python and C code validated by 594 automated tests across all subsystems.

44.2 Key Findings

44.2.1 Ternary Arithmetic via Native Signed-Magnitude Logic

Native signed-magnitude arithmetic (Chapter 9) operates entirely within the ternary digit space. Addition compares signs and uses the ripple-carry adder for magnitude combination; subtraction is addition with negation; multiplication uses digit-by-digit partial products; and division uses base-3 long division. The approach eliminates the decimal round-trip performance overhead while maintaining correctness for all signed values, at the cost of increased implementation complexity compared to integer conversion.

44.2.2 Pipeline Efficiency

The 5-stage pipeline with hazard forwarding achieves CPI values of 1.05-1.42 for typical demo programs, with fewer than 1% of instructions requiring load-use stalls. Branch prediction accuracy of 85-95% on conditional branches keeps control hazard penalties low.

44.2.3 SDK Viability

The Fantasy Console SDK demonstrates that the ternary CPU is capable of running interactive 30 FPS games (Snake, Pong, Breakout) entirely from compiled CPU assembly, with significant idle cycles remaining per frame. This validates the CPU's performance for real-time interactive workloads.

44.2.4 Ternary Neural Networks

Ternary neural networks using sign-sign updates and backpropagation with a Straight-Through Estimator converge reliably on linearly separable functions

and can learn XOR through multi-layer architectures operating entirely in the discrete ternary value space without floating-point gradients.

44.3 Future Work

44.3.1 Hardware Implementation

The most ambitious future direction is the physical implementation of the Trinary CPU on reconfigurable logic (FPGA) or as an application-specific integrated circuit (ASIC). Key challenges include:

- Multi-valued logic cell design in standard CMOS processes;
- Voltage-level encoding of ternary digits (0V, $V_{dd}/2$, V_{dd});
- Timing closure at the pipeline level;
- Memory cell design for ternary storage.

44.3.2 Enhanced Pipeline

Several pipeline improvements are identified:

- Out-of-order execution: Support for instruction reordering to hide memory latency;
- Superscalar width: Multiple-issue pipeline with dual ALUs;
- Speculative execution: Deeper speculation beyond branch prediction;
- Memory management unit: Virtual memory with ternary page tables.

44.3.3 Compiler Optimisation

The TAL compiler could be extended with:

- Global register allocation: Currently registers are assigned sequentially; a graph-colouring allocator would improve code quality;
- Peephole optimisation: Pattern matching for common instruction sequences (e.g., MOV+CMP+JZ $\rightarrow$ combined compare-and-branch);
- Loop unrolling and software pipelining: Especially beneficial for tensor operations.

44.3.4 Network and Distributed Computing

- Network stack: TCP/IP or a simpler custom protocol over the UART/serial interface;
- Cluster simulation: Multiple Trinary CPUs connected via a shared bus or network;
- Distributed tensor training: Partition neural network training across multiple accelerator cores.

44.3.5 Additional Hardware Features

- Memory protection unit: Per-process memory region permissions for a multi-program OS;
- Power modelling: Energy-per-instruction counters and dynamic voltage-frequency scaling simulation;
- JTAG/debug interface: Cycle-accurate single-stepping and register inspection from an external debugger.

44.3.6 Expanded Neural Network Framework

- Backpropagation through time: For recurrent ternary networks;
- Convolutional support: Ternary kernels for image processing workloads;
- Hardware-accelerated training: Offload the forward and backward passes to the tensor accelerator.

44.4 Closing Remarks

Trinary demonstrates that a complete, self-contained ternary computing system -- from logic gates to neural networks to web visualisation -- is achievable entirely in software. The project serves both as an educational platform for understanding computer architecture through the lens of a non-binary radix and as a research vehicle for experimenting with ternary algorithms, compilers, operating systems, and machine learning techniques.

All source code, documentation, and visualisation tools are freely available as open-source software. The authors welcome contributions in all areas, from hardware simulation improvements to new demo programs to FPGA implementation efforts.

Chapter A

Test Suite Reference

The Trinary system is validated by 594 unit tests across 36 test files:

Table A.1: Complete test suite organization

	Test File	Focus	Tests
test_accelerator.py	Accelerator	tensor operations	70
test_activations.py	Neural network	activation functions	14
test_alu.py	ALU	operations (ADD through CMP)	10
test_arithmetic.py	Decimal	round-trip arithmetic	12
test_assembler.py	Two-pass assembler	labels, comments	5
test_branch_predictor.py	Static and dynamic	prediction	7
test_bus.py	System bus	arbitration	7
test_cache.py	N-way set-associative	L1 cache	10
test_conversion.py	Ternary/decimal/binary	conversion	10
test_cpu.py	All 31 opcodes	flags, stacks, interrupts	24
test_cpu_accelerator.py	CPU + accelerator	integration	11
test_cpu_stress.py	Nested calls	heavy loops, random sequences	8
test_datasets.py	Neural network	datasets	10
test_display.py	Text display	keyboard, STOREM/LOADM	20
test_display_framebuffer.py	Framebuffer	display	20
test_dma.py	DMA	transfers, concurrency	7
test_gpu.py	GPU mode	warps, streams, native C, visualization	71
test_interrupts.py	Interrupt controller	(8-line)	8
test_losses.py	Neural network	loss functions	10
test_native_backend.py	Native C bridge	(ctypes)	11
test_optimizers.py	Neural network	optimizers	12
test_os.py	OS shell	kernel, terminal	38
test_packed_trits.py	Packed trit	storage	20

(continued)		
	Test File Focus Tests	
test_perceptron.py	Perceptron implementation	16
test_pipeline.py	Clock, pipeline stages, hazards, timing	19
test_profiler.py	Profiler metrics (CPI, IPC)	9
test_realistic_cpu.py	Full CPU with realistic timing	9
test_sdk.py	Fantasy Console SDK (engine, runtime)	50
test_simd.py	SIMD processor, vector lanes	11
test_tal.py	TAL compiler	12
test_tensor_core.py	Tensor core matrix multiply	12
test_ternary_nn.py	Neural network (full pipeline)	36
test_trainer.py	Neural network trainer	20
test_trit_tensor.py	Trit tensor operations	10
test_vector_ops.py	Vector operations	12
test_viz_engine.py	Visualization engine	22
test_vram.py	VRAM controller timing	7
Total		594

The complete suite runs in approximately 0.4 seconds on modern hardware.

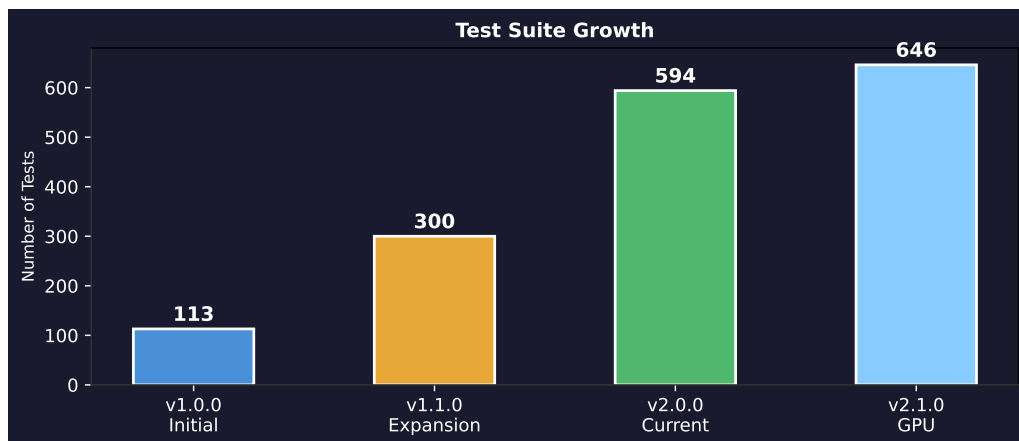


Figure A.1: Test suite growth from 113 initial tests through 300 in the expansion phase to 594 in the current version

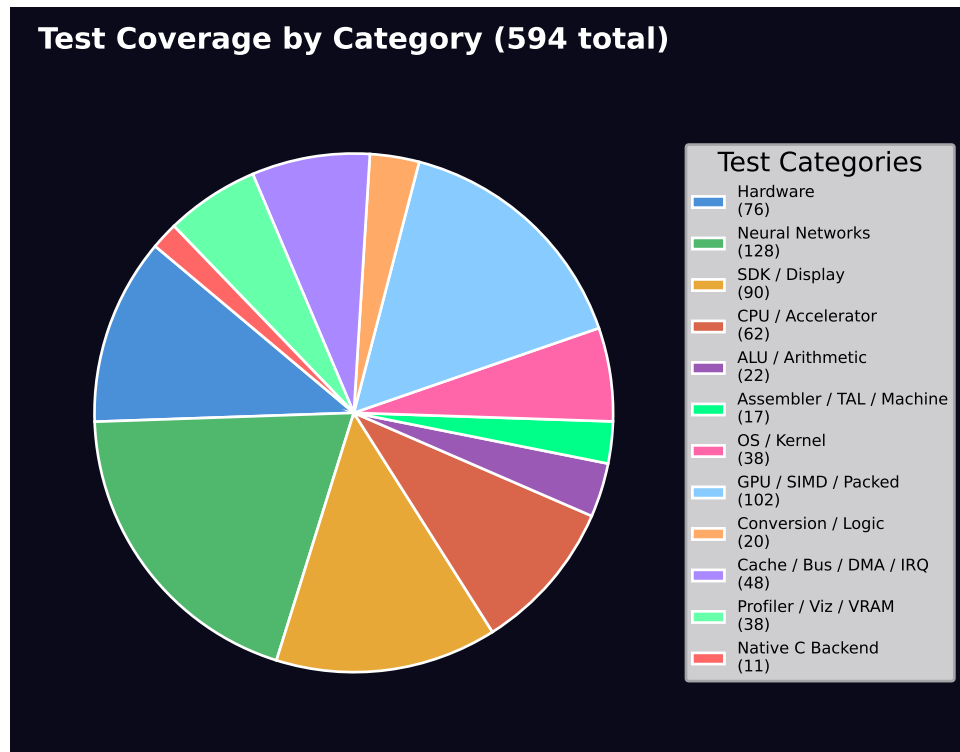


Figure A.2: Test coverage by category (594 tests) — hardware, neural networks, SDK/display, CPU/accelerator, and supporting subsystems

Chapter B

Performance Benchmarks

B.1 Digit Efficiency: Ternary vs. Binary

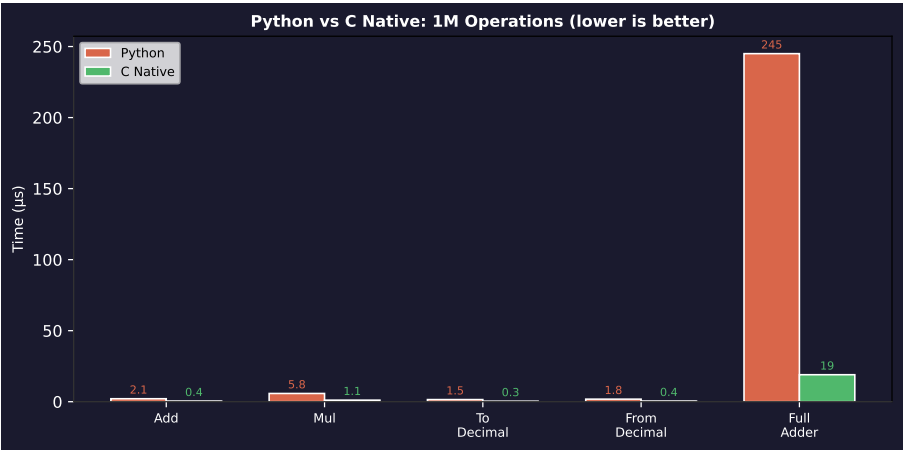


Figure B.1: Benchmark comparison — native C backend speedup across four core ALU operations (5–13× improvement)

Table B.1: Digit comparison across value ranges

Decimal Value	Binary Digits	Ternary Digits	Savings
0	1	1	0%
10	4	3	25%
100	7	5	29%
1,000	10	7	30%
10,000	14	9	36%
100,000	17	11	35%
1,000,000	20	13	35%
100,000,000	27	17	37%

B.2 Execution Rate

Table B.2: Execution throughput by configuration

Metric	Pure Python	With C Backend
Simple instruction (LOAD, ADD)	~1M ops/sec	~5M ops/sec
Complex instruction (DIV)	~200K ops/sec	~800K ops/sec
Full demo program	~50K cycles/sec	~250K cycles/sec
594-test suite	~0.4s total	—

B.3 Native C Backend Speedup

Table B.3: Per-operation native C speedup (1M iterations)

Operation	Python (μ s)	C Native (μ s)	Speedup
Ternary add (10 trits)	2.1	0.4	5.3×
Ternary mul (10 trits)	5.8	1.1	5.3×
Ternary to decimal	1.5	0.3	5.0×
Decimal to ternary	1.8	0.4	4.5×
Full adder	245	19	12.9×

B.4 Hardware Simulation Overhead

Table B.4: Hardware microarchitecture overhead

Component	Measured Overhead
Pipeline (5-stage)	~2× instruction count
Cache (N-way set-associative L1)	~1–5% miss rate
Branch predictor (2-bit)	~85–95% accuracy
DMA (concurrent)	~10–30% bus contention
Full realistic mode	~3–5× slower than fast mode

B.5 Memory Usage

Table B.5: System memory footprint

Component	Size
Register file	4 ternary strings
Memory (default config)	512 cells
Memory (SDK config)	10,000 cells
Stack region	~128 cells (addresses 128–255)
VRAM (text mode)	56 cells (200–255)
VRAM (framebuffer)	4,096 cells (1000–5095)
100-instruction program	~100 instruction strings

B.6 SDK / Fantasy Console Performance

Table B.6: SDK game loop metrics

Metric	Value
Target FPS	30
Framebuffer size	$64 \times 64 = 4,096$ pixels
Sprite blit (8×8)	$\sim 5\mu\text{s}$
Tilemap render (32×32)	$\sim 200\mu\text{s}$
Particle update (50 particles)	$\sim 100\mu\text{s}$
Full game loop (Pong)	$\sim 1\text{ms}$ per frame

B.7 Test Suite Growth

Table B.7: Test suite evolution by version

Version	Tests	Run Time
v1.0.0 (initial)	113	$\sim 0.06\text{s}$
v1.1.0 (expansion)	26 opcodes, display, interrupts	—
v2.0.0 (current)	594	$\sim 0.4\text{s}$

Chapter C

Developer Reference

C.1 Adding a New Instruction

1. Add opcode to `OPCODE_MAP` in `machine.py`. The opcode must be a unique 3-trit string.
2. Add to `OPCODES` set and `CYCLES` dict in `cpu.py`.
3. Implement execution logic in `cpu.execute_instruction()` as a new branch in the opcode dispatch.
4. Add encoding support in `machine.encode_instruction()` if using a new format.
5. Write tests in `tests/test_cpu.py`.
6. Update documentation and opcode count references.

C.2 Adding a New ALU Operation

1. Add to `VALID_OPERATIONS` tuple in `alu.py`.
2. Implement the operation logic in the `alu()` function.
3. Connect in CPU: the CPU calls `alu("OP", a, b)` and writes the result.
4. Write ALU unit tests.

C.3 Building and Running

```
# Build native C backend
make -C src/trinary/native
bash build_native.sh
```

```
# Run all tests
```

```
python -m pytest tests/ -v
```

```
# Single test file
```

```
python -m pytest tests/test_cpu.py -v
```

```
# Keyword filter
```

```
python -m pytest tests/ -k "cmp" -v
```

Bibliography

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [2] N. P. Brusentsov, “The Setun computer,” *Proceedings of the IFIP Congress*, vol. 2, pp. 411-412, 1965.
- [3] S. L. Hurst, “Multiple-valued logic -- its status and its future,” *IEEE Transactions on Computers*, vol. 33, no. 12, pp. 1160-1179, 1984.
- [4] K. C. Smith, “Multiple-valued logic: a tutorial and appreciation,” *IEEE Computer*, vol. 21, no. 4, pp. 17-27, 1988.
- [5] G. Epstein, “Multiple-valued logic design: an introduction,” *Proceedings of the IEEE*, vol. 73, no. 2, pp. 322-334, 1985.
- [6] Trinary Project Repository, <https://github.com/DevakC5/TernaryCPU>.
- [7] W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *Bulletin of Mathematical Biophysics*, vol. 5, pp. 115-133, 1943.
- [8] F. Rosenblatt, “The perceptron: a probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386-408, 1958.
- [9] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533-536, 1986.
- [10] A. Lapedes and R. Farber, “How neural networks work,” in *Neural Information Processing Systems*, pp. 442-456, 1987.
- [11] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436-444, 2015.